

**CENTRO UNIVERSITÁRIO MAURÍCIO DE NASSAU**

Bacharelado em Ciência da Computação

**GRUPO 18**

## **GROWTH INTELLIGENCE HUB (GIH)**

Plataforma de inteligência de crescimento para redes de parceiros  
em marketplaces regionais de delivery

### **SPRINT 06 — APRIMORAMENTO DO SISTEMA**

Documento acumulado: inclui as Sprints 01, 02, 03, 04 e 05

#### **EQUIPE**

Ingryd Vitoria de Araújo Barbosa — 01642893

João Pedro Nunes de França — 01626444

Marcio Maycom — 01607574

Pedro Miranda — 01607408

Thiago José Falcão de Freitas — 01597267

#### **Projeto Integrador**

Fábrica de Software · Prof.<sup>a</sup> Priscilla Gonçalves

Tópicos Avançados · Prof. Antenor Parnaíba

**2026.2**

Entrega final da disciplina: 05 de dezembro de 2026

# Sumário

## PARTE I — SPRINT 01 · PLANEJAMENTO E DESCOBERTA

1. Identificação da equipe
2. Tema
3. Definição do problema
4. Objetivos do sistema
5. Público-alvo
6. Requisitos Funcionais
7. Requisitos Não Funcionais
8. Casos de Uso
9. Product Backlog
10. Cronograma inicial
11. Repositório GitHub

## PARTE II — SPRINT 02 · ARQUITETURA E MODELAGEM

1. Arquitetura do sistema
2. Diagrama de classes
3. Modelo Entidade-Relacionamento
4. Modelo relacional
5. Protótipo das telas principais
6. Banco de dados criado
7. Projeto estruturado no GitHub

## PARTE III — SPRINT 03 · ESTRUTURA INICIAL FUNCIONANDO

1. A estrutura implementada
2. Banco de dados conectado
3. Login funcional
4. Cadastro de usuários
5. Controle de perfis
6. CRUD principal
7. Execução local
8. Repositório
9. Dificuldades encontradas
10. Ajustes no planejamento
11. Próximos passos

## PARTE IV — SPRINT 04 · PRIMEIRO MÓDULO COMPLETO

1. O módulo e suas funcionalidades
2. O módulo funcionando
3. Persistência
4. Validações
5. Mensagens de erro
6. Navegação entre telas
7. Repositório e commits
8. Execução e roteiro de demonstração
9. Dificuldades encontradas

10. Ajustes no planejamento, na arquitetura e na modelagem
11. Próximos passos

## **PARTE V — SPRINT 05 · SEGUNDO MÓDULO FUNCIONANDO**

1. O segundo módulo: Previsão
2. O módulo funcionando
3. Integração com o banco
4. Regras de negócio implementadas e atualizadas
5. Testes realizados e resultados
6. Bugs identificados e correções
7. Repositório e commits
8. Execução e roteiro de demonstração
9. Dificuldades encontradas
10. Ajustes no planejamento, na arquitetura e na modelagem
11. Próximos passos

## **PARTE VI — SPRINT 06 · APRIMORAMENTO DO SISTEMA**

1. Correções apontadas na Pré-Banca
2. O terceiro módulo: Campanha e otimizador
3. O módulo funcionando
4. A integração entre os módulos
5. Melhorias na interface
6. Ajustes de navegação
7. Testes e bugs
8. Repositório, commits e revisões
9. Execução e roteiro de demonstração
10. Dificuldades encontradas
11. Ajustes no planejamento, na arquitetura e na modelagem
12. Próximos passos

Documentação completa e versionada em: [github.com/PedroMiranda243/gih-fabrica-de-software](https://github.com/PedroMiranda243/gih-fabrica-de-software)

## 1. Identificação da equipe

|                 |                                                    |
|-----------------|----------------------------------------------------|
| Nome do projeto | Growth Intelligence Hub (GIH)                      |
| Turma           | CC8MB                                              |
| Disciplinas     | Fábrica de Software · Tópicos Avançados            |
| Repositório     | github.com/PedroMiranda243/gih-fabrica-de-software |

### Integrantes

Equipe de 5 integrantes, dentro do limite de 3 a 5 estabelecido pela disciplina. Os papéis organizam o trabalho, mas todos os integrantes contribuem com código e aparecem no histórico de commits do repositório.

| Integrante                       | Matrícula | GitHub           | Papel                                 |
|----------------------------------|-----------|------------------|---------------------------------------|
| Ingryd Vitoria de Araújo Barbosa | 01642893  | @ingrydaraujob   | Desenvolvedora Frontend               |
| João Pedro Nunes de França       | 01626444  | @joaopfranca04   | Banco de Dados, Documentação e Testes |
| Marcio Maycom                    | 01607574  | @mihaeldatoman   | Product Owner                         |
| Pedro Miranda                    | 01607408  | @PedroMiranda243 | Dev. Backend / Núcleo Computacional   |
| Thiago José Falcão de Freitas    | 01597267  | @Thiagofalcao    | Scrum Master                          |

### Papéis previstos e responsabilidades

| Papel                                 | Responsabilidade principal                                     |
|---------------------------------------|----------------------------------------------------------------|
| Scrum Master                          | Cerimônias, board e acompanhamento das entregas                |
| Product Owner                         | Backlog, requisitos, priorização e critérios de aceite         |
| Dev. Backend / Núcleo Computacional   | API, regras de negócio, otimizador C++/CUDA e modelo preditivo |
| Desenvolvedor Frontend                | Telas, painel, gráficos, responsividade e acessibilidade       |
| Banco de Dados, Documentação e Testes | Modelagem, migrações, suíte de testes e documentação           |

*Os papéis organizam o trabalho; todos os integrantes contribuem com código e aparecem no histórico de commits. As histórias do núcleo em OpenMP e CUDA exigem programação em par, para que ninguém seja a única pessoa a entender o motor do projeto.*

## 2. Tema

**Growth Intelligence Hub (GIH) — plataforma de inteligência de crescimento para redes de parceiros em marketplaces regionais de delivery.**

O sistema recebe os relatórios periódicos de desempenho de uma rede de comércios parceiros e devolve decisões priorizadas: quem está crescendo, quem está prestes a cair, e — dado um orçamento e uma equipe limitados — exatamente em quais parceiros investir a próxima rodada de ações comerciais.

### Domínio de aplicação

Marketplaces regionais de delivery operam por franquia ou licença local. Uma unidade típica intermedeia pedidos entre consumidores de uma cidade ou microrregião e uma rede de 50 a 500 comércios parceiros. A unidade recebe da plataforma nacional um relatório periódico com o desempenho de cada parceiro, e é responsável por desenvolver comercialmente essa base.

### Por que este tema atende aos critérios da disciplina

| Critério                                   | Como o tema atende                                                                                                                                     |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Resolver um problema real                  | Redes regionais gerenciam a base comercial sem qualquer ferramenta analítica                                                                           |
| Complexidade compatível com um TCC         | Ingestão, modelagem temporal, regras de segmentação, aprendizado de máquina, otimização combinatória, paralelismo, GPU, controle de acesso e auditoria |
| Permitir evolução no semestre              | Sete sprints com entregas independentes, cumulativas e executáveis                                                                                     |
| Ser viável no prazo                        | Escopo fatiado por módulo; o núcleo pesado tem investigação técnica antecipada para a Sprint 3                                                         |
| Funcionalidades suficientes                | 43 requisitos funcionais em 6 módulos integrados                                                                                                       |
| Componente de IA/otimização não decorativo | O motor de decisão é o produto — sem ele o sistema vira um painel passivo                                                                              |

## 3. Definição do problema

### 3.1 A situação

Numa operação regional de delivery, a atenção comercial se concentra nos parceiros de maior faturamento — o Top 15. É uma escolha racional a curto prazo: são eles que sustentam a receita. Mas produz três efeitos que se realimentam:

- Os parceiros fora do topo — a cauda longa, tipicamente 85% da base — passam meses sem contato.
- Sem estímulo, a cauda longa não cresce, e o ranking permanece imóvel: os mesmos nomes no topo, período após período.
- Como o topo é sempre o mesmo, ele também não é desafiado — e estagna.

Ao mesmo tempo, os dados chegam em formato que não sustenta análise. O painel da plataforma mostra o número do período atual em texto corrido. Não há série histórica, não há comparação automática com o período anterior, não há alerta. O gestor vê o retrato, nunca o filme.

### 3.2 Os sub-problemas

| ID | Sub-problema                                        | Consequência operacional                                                                             |
|----|-----------------------------------------------------|------------------------------------------------------------------------------------------------------|
| P1 | Dados sem visualização analítica                    | Tendências passam despercebidas; a leitura depende de esforço manual repetido a cada período         |
| P2 | Queda de parceiro detectada tarde                   | Quando a perda fica evidente no faturamento, o parceiro já pode ter migrado ou reduzido a operação   |
| P3 | Cauda longa sem processo de relacionamento          | Ranking imóvel; potencial de crescimento não explorado em cerca de 85% da base                       |
| P4 | Alocação do esforço comercial decidida por intuição | Verba de cupom e agenda da equipe são distribuídas sem critério explícito nem verificação de retorno |
| P5 | Comunicação manual e sem segmentação                | Ações por categoria exigem esforço proporcional ao número de parceiros, e por isso não acontecem     |

### 3.3 O problema central

#### P4 é o núcleo técnico do projeto.

Escolher em quais parceiros aplicar quais ações, respeitando orçamento, capacidade da equipe e cotas por categoria, para maximizar o retorno esperado, é um problema de otimização combinatória com restrições.

O espaço de busca cresce exponencialmente: com  $N$  parceiros e  $A$  tipos de ação, existem  $(A+1)^N$  planos possíveis. Para 200 parceiros e 4 tipos de ação, isso já ultrapassa  $10^{139}$  combinações — inviável por força bruta e igualmente inviável de resolver "no olho".

É exatamente o tipo de problema que justifica metaheurística paralelizada e aceleração em GPU — e é a ponte entre as duas disciplinas: a Fábrica de Software entrega o produto completo em torno dele, Tópicos Avançados entrega o motor.

### 3.4 Por que este problema é relevante

**Para a operação.** A cauda longa concentra a maior parte do potencial de crescimento não explorado: são 85% dos parceiros recebendo uma fração da atenção comercial. Um parceiro que sai da base não é apenas receita perdida — é um comércio que passa a operar só no concorrente, e reconquistá-lo custa muito mais do que teria custado mantê-lo. O alerta que chega um período antes é a diferença entre uma conversa de retenção e uma perda consumada.

**Para os parceiros.** São comércios pequenos, quase sempre sem estrutura de marketing. Entrar numa campanha bem escolhida é acesso a demanda que não conseguiriam gerar sozinhos. Quando a alocação é feita por intuição, quem já é grande recebe mais, e a assimetria se aprofunda sozinha.

**Para a decisão.** Verba de incentivo e agenda da equipe são recursos escassos e disputados. Hoje a escolha é feita sem critério explícito e, principalmente, sem verificação posterior: não se sabe se a campanha da semana passada funcionou, porque nada foi medido contra uma previsão. Sem linha de base não existe aprendizado, apenas repetição.

**Para o campo técnico.** O problema não é uma desculpa para usar tecnologia; ele exige a tecnologia. Priorizar ações sob restrições de orçamento, capacidade e cotas é otimização combinatória, e estimar o retorno de cada ação é um problema de previsão. São duas classes de problema com solução conhecida e mensurável, o que permite avaliar objetivamente se a solução funcionou, comparando speedup e erro de previsão contra baselines, em vez de opinar sobre a interface.

3.5 O que já se sabe que não resolve

| Alternativa                | Por que não basta                                                                                                                                          |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Planilha                   | Não sustenta série histórica multi-período de centenas de parceiros, nem otimização com restrições                                                         |
| Assistente de IA genérico  | Responde sobre os dados colados, mas sem rastreabilidade, sem persistência, sem painel e sem garantia de consistência numérica entre duas perguntas iguais |
| BI genérico                | Mostra o passado. Não prevê, não otimiza e não fecha o ciclo até a ação                                                                                    |
| Ampliar a equipe comercial | Custo recorrente; não resolve o critério de priorização, apenas aumenta o volume de contatos feitos sem critério                                           |

## 4. Objetivos do sistema

### 4.1 Objetivo geral

Transformar os dados brutos de desempenho de uma rede de parceiros em um plano de ação comercial priorizado, previsto por modelo treinado a partir do histórico e otimizado sob as restrições reais de orçamento e capacidade de operação.

### 4.2 Objetivos específicos

| ID | Objetivo                                                | Verificação                                                                                  |
|----|---------------------------------------------------------|----------------------------------------------------------------------------------------------|
| O1 | Ingerir e normalizar relatórios periódicos por parceiro | Importação de 500 parceiros gera 500 registros consistentes, com período obrigatório         |
| O2 | Calcular métricas derivadas e séries históricas         | Ticket médio, variação e tendência disponíveis para qualquer parceiro com 2 ou mais períodos |
| O3 | Segmentar automaticamente por regra determinística      | A mesma base reprocessada produz exatamente a mesma segmentação                              |
| O4 | Prever faturamento e risco de queda com modelo próprio  | MAPE inferior ao de um baseline ingênuo em conjunto de teste separado                        |
| O5 | Otimizar a alocação de ações sob restrições             | O plano respeita 100% das restrições e supera a heurística "investir nos maiores"            |
| O6 | Acelerar o otimizador por paralelismo em CPU e GPU      | Speedup mínimo de 5x em GPU, com qualidade de solução equivalente                            |
| O7 | Gerar mensagens com aprovação humana obrigatória        | Nenhuma mensagem é aprovada sem ação explícita de usuário autorizado                         |
| O8 | Controlar acesso por perfil, com trilha de auditoria    | Toda ação sensível registra autor, data e parâmetros                                         |

### 4.3 Indicadores de sucesso

#### Do produto

| Indicador                       | Definição                                                     | Meta                       |
|---------------------------------|---------------------------------------------------------------|----------------------------|
| Mobilidade do Top N             | Parceiros que entram ou saem do Top 15 entre dois períodos    | Tornar visível e crescente |
| Cobertura de relacionamento     | Percentual da base que recebeu ao menos uma ação no período   | Cobertura planejada        |
| Antecedência do alerta de risco | Períodos de antecedência do sinal antes da queda se confirmar | Mínimo de 1 período        |

#### Técnicos

| Indicador                                | Meta                                  |
|------------------------------------------|---------------------------------------|
| Speedup do otimizador (GPU sobre serial) | Mínimo de 5x no cenário de referência |
| Tempo de resposta do otimizador em GPU   | Até 5 s no cenário de referência      |
| MAPE do modelo preditivo                 | Inferior ao baseline ingênuo          |
| Cobertura de testes no núcleo de regras  | Mínimo de 70%                         |

## 5. Público-alvo

### 5.1 Perfil da organização usuária

Unidade regional de marketplace de delivery — franquia ou licenciada local — com:

- 50 a 500 comércios parceiros ativos
- 1 a 3 pessoas na equipe comercial
- Relacionamento predominantemente remoto, por aplicativo de mensagens
- Sem ferramenta de BI, sem analista de dados e sem equipe de tecnologia própria
- Verba de incentivo limitada e disputada entre parceiros

### 5.2 Usuários do sistema

| Perfil              | Quem é                                         | O que faz no sistema                                      | Necessidade central                                               |
|---------------------|------------------------------------------------|-----------------------------------------------------------|-------------------------------------------------------------------|
| Gestor (primário)   | Responsável pela unidade; decide onde investir | Consulta o painel, executa o otimizador, aprova mensagens | Decidir rápido, com critério defensável, sem depender de analista |
| Analista comercial  | Executa o relacionamento no dia a dia          | Importa relatórios, analisa parceiros, redige mensagens   | Saber quem procurar hoje e com qual argumento                     |
| Administrador       | Responsável técnico da operação                | Gerencia usuários e perfis, ajusta parâmetros, audita     | Manter o sistema seguro e configurado sem mexer em código         |
| Parceiro (opcional) | Comércio cadastrado na rede                    | Consulta apenas o próprio desempenho                      | Enxergar o próprio resultado e a própria evolução                 |

### 5.3 Restrições impostas pelo público

O público-alvo condiciona decisões técnicas que aparecem nos requisitos não funcionais:

- Interface em português e autoexplicativa — não há treinamento formal nem suporte de TI interno (RNF20)
- Instalação em um comando — não há equipe para configurar ambiente (RNF07)
- Funcionar sem GPU — a máquina do usuário pode ser um notebook comum (RNF06)
- Rastreabilidade das análises — o gestor precisa justificar a decisão para a franqueadora e para os parceiros (RNF17)
- Aprovação humana antes de qualquer envio — a relação com o parceiro é ativo estratégico (RF39)

## 6. Requisitos Funcionais

43 requisitos funcionais distribuídos em 6 módulos. Prioridade em MoSCoW: M (indispensável), S (importante), C (desejável). Perfis: ADM Administrador, GES Gestor, ANL Analista, PAR Parceiro.

### Módulo 1 — Autenticação, perfis e auditoria

| ID   | Requisito                                                                                                         | Pri | Perfis |
|------|-------------------------------------------------------------------------------------------------------------------|-----|--------|
| RF01 | Autenticar o usuário por login e senha, criando uma sessão identificada.                                          | M   | todos  |
| RF02 | Permitir o encerramento da sessão, invalidando-a no servidor.                                                     | M   | todos  |
| RF03 | Permitir cadastrar, editar, desativar e listar usuários.                                                          | M   | ADM    |
| RF04 | Atribuir a cada usuário exatamente um perfil entre Administrador, Gestor, Analista e Parceiro.                    | M   | ADM    |
| RF05 | Restringir o acesso a cada funcionalidade conforme o perfil, validando a permissão no servidor a cada requisição. | M   | todos  |
| RF06 | Registrar em trilha de auditoria as ações sensíveis, com autor, data, hora e parâmetros.                          | M   | todos  |
| RF07 | Permitir que o usuário altere a própria senha, exigindo a senha atual.                                            | S   | todos  |
| RF08 | Permitir consultar a trilha de auditoria, com filtro por autor, tipo de ação e datas.                             | S   | ADM    |

### Módulo 2 — Ingestão e gestão de dados

| ID   | Requisito                                                                                              | Pri | Perfis        |
|------|--------------------------------------------------------------------------------------------------------|-----|---------------|
| RF09 | Importar um relatório de desempenho por período, aceitando texto colado e arquivo CSV.                 | M   | GES, ANL      |
| RF10 | Exigir a data inicial e final do período na importação e recusar a importação sem esse dado.           | M   | GES, ANL      |
| RF11 | Validar o conteúdo e exibir prévia com registros reconhecidos, rejeitados e o motivo, antes de gravar. | M   | GES, ANL      |
| RF12 | Impedir a importação de um período já registrado, oferecendo opção explícita de substituição.          | S   | GES, ANL      |
| RF13 | Manter o histórico das importações com autor, data, período coberto e total de registros.              | M   | GES, ANL, ADM |
| RF14 | Cadastrar, editar e desativar parceiros, com nome, categoria, status comercial e contato.              | M   | GES, ANL      |
| RF15 | Sugerir a categoria do parceiro pelo nome, sinalizando que é sugestão e exigindo confirmação.          | S   | GES, ANL      |
| RF16 | Oferecer gerador de dados sintéticos de 100 a 10.000 parceiros, para demonstração e benchmark.         | M   | ADM           |

### Módulo 3 — Inteligência de negócio e segmentação

| ID   | Requisito                                                                                                | Pri | Perfis        |
|------|----------------------------------------------------------------------------------------------------------|-----|---------------|
| RF17 | Exibir painel com indicadores consolidados do período e a variação frente ao período anterior.           | M   | GES, ANL      |
| RF18 | Exibir ranking por faturamento, com posição atual, posição anterior e variação percentual.               | M   | GES, ANL      |
| RF19 | Exibir a série histórica em gráfico, para a unidade e para o parceiro individual.                        | M   | GES, ANL, PAR |
| RF20 | Classificar cada parceiro em exatamente um segmento, por regra determinística com precedência explícita. | M   | GES, ANL      |

| ID   | Requisito                                                                                      | Pri | Perfis   |
|------|------------------------------------------------------------------------------------------------|-----|----------|
| RF21 | Permitir configurar os limiares da segmentação sem alteração de código.                        | S   | ADM      |
| RF22 | Exibir a mobilidade do ranking entre dois períodos, listando quem entrou e quem saiu do Top N. | M   | GES, ANL |
| RF23 | Permitir filtrar e ordenar a lista de parceiros por categoria, segmento e métricas.            | M   | GES, ANL |
| RF24 | Permitir buscar parceiro por nome, com correspondência parcial.                                | M   | GES, ANL |
| RF25 | Permitir exportar em CSV a visão atualmente filtrada.                                          | S   | GES, ANL |
| RF26 | Permitir que o perfil Parceiro consulte apenas o próprio desempenho, sem acesso a terceiros.   | C   | PAR      |

## Módulo 4 — Núcleo computacional: previsão e otimização

| ID   | Requisito                                                                                         | Pri | Perfis   |
|------|---------------------------------------------------------------------------------------------------|-----|----------|
| RF27 | Treinar modelo de previsão a partir do histórico, registrando data, volume de dados e métricas.   | M   | ADM, GES |
| RF28 | Exibir, por parceiro, o faturamento previsto e a probabilidade estimada de queda.                 | M   | GES, ANL |
| RF29 | Permitir configurar orçamento, número máximo de ações, catálogo com custos e cotas por categoria. | M   | GES      |
| RF30 | Executar o otimizador e retornar o plano de campanha, com uplift esperado e custo total.          | M   | GES      |
| RF31 | Garantir que todo plano respeite as restrições e sinalizar quando não existir solução viável.     | M   | GES      |
| RF32 | Permitir escolher o modo de execução entre serial, CPU paralelo e GPU, com seleção automática.    | M   | GES, ADM |
| RF33 | Exibir benchmark comparativo entre os modos: tempo, speedup e qualidade da solução.               | M   | GES, ADM |
| RF34 | Registrar o histórico das execuções do otimizador com parâmetros, modo, tempo e resultado.        | S   | GES, ADM |
| RF35 | Permitir comparar lado a lado dois planos de campanha gerados com parâmetros diferentes.          | C   | GES      |

## Módulo 5 — Central de comunicação

| ID   | Requisito                                                                                        | Pri | Perfis   |
|------|--------------------------------------------------------------------------------------------------|-----|----------|
| RF36 | Gerar mensagens personalizadas por segmento e categoria, a partir do plano ou de seleção manual. | M   | GES, ANL |
| RF37 | Manter as mensagens geradas em fila de aprovação, no estado pendente.                            | M   | GES, ANL |
| RF38 | Permitir aprovar, editar ou rejeitar cada mensagem antes de considerá-la pronta para envio.      | M   | GES      |
| RF39 | Impedir que qualquer mensagem seja aprovada sem ação explícita de um usuário com perfil Gestor.  | M   | GES      |
| RF40 | Manter o histórico das mensagens decididas, com autor, data e conteúdo final.                    | S   | GES, ANL |

## Módulo 6 — Assistente analítico

| ID   | Requisito                                                                                  | Pri | Perfis   |
|------|--------------------------------------------------------------------------------------------|-----|----------|
| RF41 | Responder a perguntas em linguagem natural sobre os dados armazenados.                     | S   | GES, ANL |
| RF42 | Citar o período e a origem dos dados em toda resposta, ou declarar insuficiência de dados. | M   | GES, ANL |

| ID   | Requisito                                                                                     | Pri | Perfis   |
|------|-----------------------------------------------------------------------------------------------|-----|----------|
| RF43 | Impedir que o assistente produza valores numéricos não calculados pelo núcleo determinístico. | M   | GES, ANL |

**Total: 43 requisitos funcionais — 31 Must, 9 Should, 3 Could.**

## 7. Requisitos Não Funcionais

29 requisitos não funcionais, cada um com métrica de aceitação verificável — nenhum enunciado subjetivo.

### Desempenho e escalabilidade

| ID    | Requisito                                                                                         | Métrica de aceitação                                  |
|-------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| RNF01 | O otimizador em GPU resolve o cenário de referência (2.000 parceiros, 5 ações) em uso interativo. | Até 5 s de ponta a ponta                              |
| RNF02 | O otimizador em GPU apresenta ganho sobre o baseline serial, com qualidade equivalente.           | Speedup mínimo de 5x, uplift dentro de 2% do baseline |
| RNF03 | O painel responde dentro do limiar de fluidez para bases realistas.                               | Até 2 s com 5.000 parceiros                           |
| RNF04 | O sistema suporta a carga máxima prevista sem degradação funcional.                               | 10.000 parceiros e 52 períodos                        |
| RNF05 | As consultas ao histórico usam índices, evitando varredura completa.                              | Plano sem varredura sequencial                        |

### Portabilidade e disponibilidade

| ID    | Requisito                                                               | Métrica de aceitação                          |
|-------|-------------------------------------------------------------------------|-----------------------------------------------|
| RNF06 | O sistema funciona sem GPU compatível, recorrendo ao modo CPU paralelo. | Fluxo completo executado sem GPU, com aviso   |
| RNF07 | O ambiente sobe com um único comando, sem etapa manual.                 | docker compose up a partir de clone limpo     |
| RNF08 | O sistema funciona nos navegadores de maior uso.                        | Chrome, Edge e Firefox, duas versões recentes |

### Segurança

| ID    | Requisito                                                                                              | Métrica de aceitação                     |
|-------|--------------------------------------------------------------------------------------------------------|------------------------------------------|
| RNF09 | Senhas armazenadas apenas como hash com derivação lenta e sal por usuário.                             | bcrypt ou Argon2; nenhuma senha em claro |
| RNF10 | Sessão em cookie HttpOnly e SameSite, com renovação do identificador no login.                         | Verificado por teste automatizado        |
| RNF11 | Bloqueio temporário após falhas sucessivas; resposta idêntica para usuário inexistente e senha errada. | Bloqueio após 5 falhas                   |
| RNF12 | Toda saída originada do usuário ou do modelo é escapada antes de chegar ao navegador.                  | Teste com carga de injeção de script     |
| RNF13 | Todo acesso ao banco usa consultas parametrizadas.                                                     | Revisão de código e teste de injeção SQL |
| RNF14 | A autorização por perfil é verificada no servidor em todos os endpoints.                               | Teste de cada endpoint com cada perfil   |
| RNF15 | O sistema limita o tamanho das entradas aceitas.                                                       | Limite para relatório e para pergunta    |

### Confiabilidade e rastreabilidade

| ID    | Requisito                                                                                      | Métrica de aceitação                     |
|-------|------------------------------------------------------------------------------------------------|------------------------------------------|
| RNF16 | Ranking, segmentação e métricas são determinísticos; o modelo de linguagem não calcula número. | Reprocessar produz resultado idêntico    |
| RNF17 | Toda resposta do assistente cita a origem ou declara insuficiência de dados.                   | Conjunto de perguntas de teste           |
| RNF18 | Erros retornam mensagem genérica; o detalhe fica no log do servidor.                           | Nenhum rastreamento de pilha na resposta |
| RNF19 | Log estruturado com identificador de correlação entre resposta e evento.                       | Identificador presente em ambos          |

## Usabilidade e acessibilidade

| ID    | Requisito                                                               | Métrica de aceitação                           |
|-------|-------------------------------------------------------------------------|------------------------------------------------|
| RNF20 | Interface integralmente em português e operável sem treinamento formal. | Usuário novo completa o fluxo sem documentação |
| RNF21 | Interface responsiva para desktop e tablet.                             | Layout funcional a partir de 768 px            |
| RNF22 | Texto atende ao contraste mínimo do nível AA.                           | Razão mínima de 4,5:1                          |
| RNF23 | Operações longas informam progresso.                                    | Indicador em importação e otimização           |

## Manutenibilidade e processo

| ID    | Requisito                                                                    | Métrica de aceitação                              |
|-------|------------------------------------------------------------------------------|---------------------------------------------------|
| RNF24 | O núcleo de regras tem cobertura de testes automatizados adequada.           | Cobertura mínima de 70%                           |
| RNF25 | Toda alteração chega ao ramo principal por Pull Request revisado.            | Proteção ativa; nenhum push direto                |
| RNF26 | A integração contínua executa análise estática e testes a cada Pull Request. | CI obrigatória e verde antes da mesclagem         |
| RNF27 | A documentação permite a um terceiro executar o sistema do zero.             | Validado por integrante que não escreveu o código |

## Privacidade

| ID    | Requisito                                                     | Métrica de aceitação                    |
|-------|---------------------------------------------------------------|-----------------------------------------|
| RNF28 | O sistema não coleta dados pessoais de consumidores finais.   | Modelo sem entidade de consumidor final |
| RNF29 | O repositório não contém dados reais; toda massa é sintética. | Verificação antes de cada entrega       |

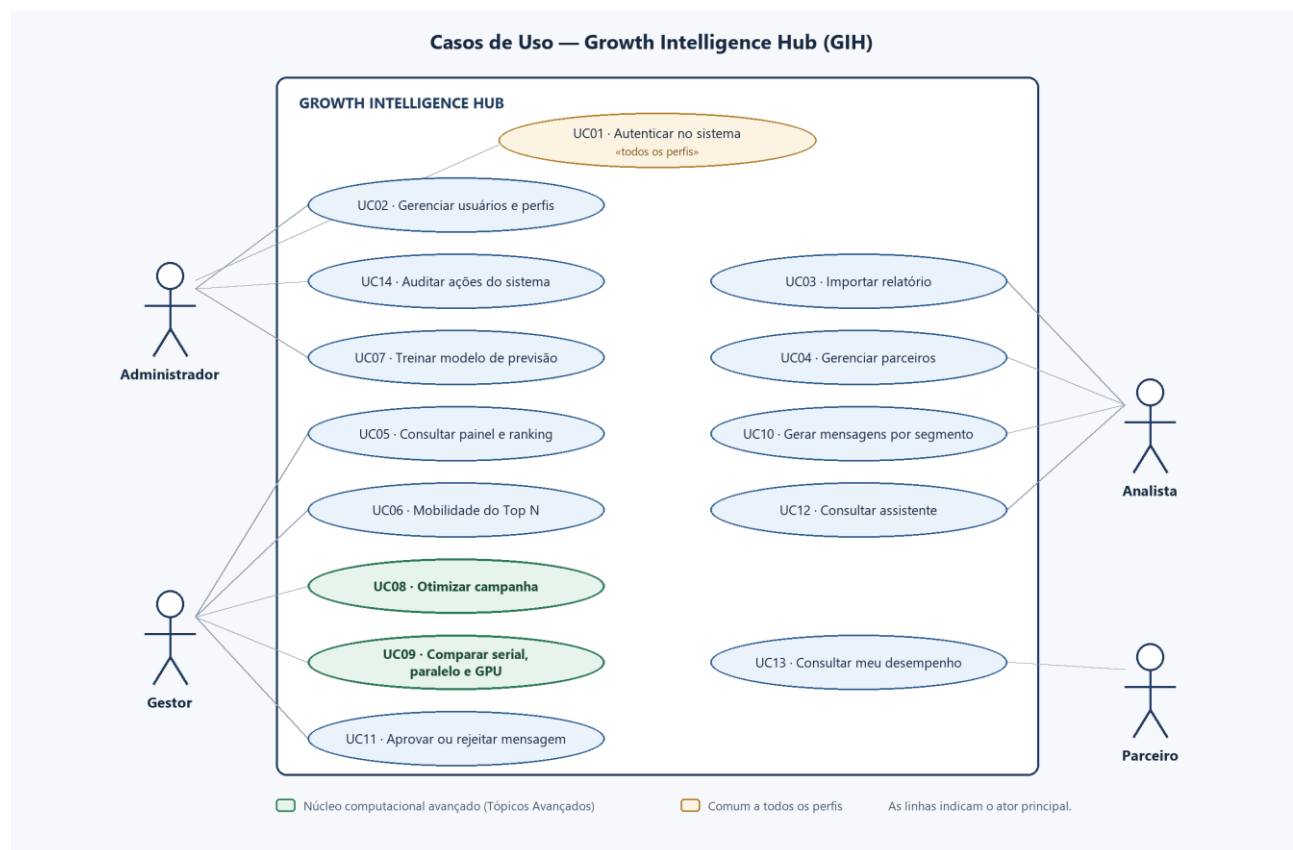
## 8. Casos de Uso

### 8.1 Atores

| Ator                      | Tipo       | Descrição                                                                                            |
|---------------------------|------------|------------------------------------------------------------------------------------------------------|
| Administrador             | Primário   | Responsável técnico da operação. Gerencia usuários, perfis e parâmetros; audita as ações registradas |
| Gestor                    | Primário   | Responsável pela unidade. Executa o otimizador, aprova mensagens e acompanha a mobilidade do ranking |
| Analista                  | Primário   | Executa o relacionamento. Importa relatórios, analisa e redige mensagens, mas não aprova envio       |
| Parceiro                  | Primário   | Comércio da rede. Acesso restrito ao próprio desempenho histórico                                    |
| Modelo de Linguagem Local | Secundário | Serviço interno que redige textos e responde ao assistente. Nunca calcula números                    |

A separação entre Gestor e Analista é o que materializa a exigência de diferentes perfis de usuários: os dois enxergam o mesmo painel, mas apenas o Gestor pode comprometer recursos (executar campanha) e comprometer a relação com o parceiro (aprovar mensagem).

### 8.2 Diagrama de casos de uso



### 8.3 Visão geral

| ID   | Caso de uso                      | Ator principal   | Requisitos cobertos |
|------|----------------------------------|------------------|---------------------|
| UC01 | Autenticar no sistema            | Todos            | RF01, RF02, RF07    |
| UC02 | Gerenciar usuários e perfis      | Administrador    | RF03, RF04, RF05    |
| UC03 | Importar relatório de desempenho | Analista, Gestor | RF09 a RF13         |
| UC04 | Gerenciar parceiros e categorias | Analista, Gestor | RF14, RF15, RF16    |

| ID   | Caso de uso                                  | Ator principal        | Requisitos cobertos      |
|------|----------------------------------------------|-----------------------|--------------------------|
| UC05 | Consultar painel e ranking                   | Gestor, Analista      | RF17 a RF20, RF23 a RF25 |
| UC06 | Analisar mobilidade do Top N                 | Gestor, Analista      | RF22                     |
| UC07 | Treinar modelo de previsão                   | Administrador, Gestor | RF27, RF28               |
| UC08 | Configurar e executar otimização de campanha | Gestor                | RF29 a RF31, RF35        |
| UC09 | Comparar desempenho serial, paralelo e GPU   | Gestor, Administrador | RF32, RF33, RF34         |
| UC10 | Gerar mensagens por segmento                 | Analista, Gestor      | RF36, RF37               |
| UC11 | Aprovar ou rejeitar mensagem                 | Gestor                | RF38, RF39, RF40         |
| UC12 | Consultar assistente analítico               | Gestor, Analista      | RF41, RF42, RF43         |
| UC13 | Consultar meu desempenho                     | Parceiro              | RF19, RF26               |
| UC14 | Auditar ações do sistema                     | Administrador         | RF06, RF08               |

## 8.4 Matriz de permissões

Legenda: X executa · L somente leitura · — sem acesso

| Caso de uso              | ADM | GES | ANL | PAR |
|--------------------------|-----|-----|-----|-----|
| UC01 Autenticar          | X   | X   | X   | X   |
| UC02 Gerenciar usuários  | X   | —   | —   | —   |
| UC03 Importar relatório  | —   | X   | X   | —   |
| UC04 Gerenciar parceiros | —   | X   | X   | —   |
| UC05 Painel e ranking    | L   | X   | X   | —   |
| UC06 Mobilidade do Top N | L   | X   | X   | —   |
| UC07 Treinar modelo      | X   | X   | —   | —   |
| UC08 Executar otimização | —   | X   | L   | —   |
| UC09 Benchmark           | X   | X   | —   | —   |
| UC10 Gerar mensagens     | —   | X   | X   | —   |
| UC11 Aprovar mensagem    | —   | X   | —   | —   |
| UC12 Assistente          | —   | X   | X   | —   |
| UC13 Meu desempenho      | —   | —   | —   | X   |
| UC14 Auditoria           | X   | —   | —   | —   |

## 8.5 Especificação detalhada — UC08: Executar otimização de campanha

| Ator principal | Gestor                                                                                            |
|----------------|---------------------------------------------------------------------------------------------------|
| Objetivo       | Obter o plano de ações que maximiza o retorno esperado dentro das restrições reais                |
| Pré-condições  | Usuário autenticado com perfil Gestor; modelo de previsão treinado; ao menos um período importado |
| Pós-condições  | Plano de campanha gerado e persistido; execução registrada no histórico e na auditoria            |
| Requisitos     | RF29, RF30, RF31, RF32, RF34, RF35                                                                |

### Fluxo principal

1. O usuário abre a tela de campanha.
2. Informa orçamento total, número máximo de ações, período de aplicação e o catálogo de ações com custo unitário.
3. Opcionalmente define cotas por categoria — por exemplo, ao menos 30% das ações destinadas à cauda longa.
4. Opcionalmente escolhe o modo de execução; se não escolher, o sistema seleciona o modo disponível mais rápido.
5. O usuário dispara a otimização.
6. O sistema recupera, para cada parceiro elegível, o faturamento previsto e o risco de queda.
7. O sistema executa o otimizador, exibindo indicador de progresso.
8. O sistema apresenta o plano de campanha: pares parceiro-ação, uplift esperado, custo total, folga por restrição e tempo de execução.
9. O usuário pode exportar o plano ou encaminhá-lo para a geração de mensagens.
10. O sistema registra a execução com autor, parâmetros, modo, tempo e resultado.

### Fluxos alternativos

- A1 — Restrições inviáveis. O sistema não retorna plano parcial: informa a inviabilidade e aponta qual restrição foi violada.
- A2 — Orçamento superior ao necessário. Retorna o plano completo, informa a sobra e sugere ampliar o catálogo de ações.
- A3 — Comparação de cenários. O usuário executa uma segunda otimização e solicita a comparação lado a lado.
- A4 — GPU indisponível. Executa em CPU paralelo e informa a substituição, sem interromper a operação.

### Exceções

- E1 — Modelo não treinado. O sistema interrompe e direciona o usuário para o caso de uso UC07.
- E2 — Execução excede o tempo limite. Preserva a melhor solução encontrada, sinaliza que é parcial e registra a ocorrência.

*Os demais casos de uso centrais — UC01, UC03, UC05, UC09 e UC11 — estão especificados no mesmo nível de detalhe no repositório, em docs/03-casos-de-uso.md.*

## 9. Product Backlog

Histórias no formato "Como <perfil>, quero <ação>, para <valor>", com prioridade MoSCoW, estimativa em story points (Fibonacci), sprint alocada e critérios de aceite. Os critérios de aceite completos estão nas issues do repositório.

### 9.1 Épicos

| Épico | Título                                | Pontos | Sprints |
|-------|---------------------------------------|--------|---------|
| E1    | Fundação, planejamento e ambiente     | 47     | 1–3     |
| E2    | Autenticação, perfis e auditoria      | 39     | 4–5     |
| E3    | Ingestão e modelo de dados            | 46     | 2–6     |
| E4    | Inteligência de negócio e segmentação | 47     | 6–9     |
| E5    | Núcleo preditivo                      | 34     | 8–9     |
| E6    | Otimização, paralelismo e GPU         | 82     | 3–11    |
| E7    | Central de comunicação                | 26     | 12      |
| E8    | Assistente analítico                  | 21     | 12–13   |
| E9    | Qualidade, documentação e entrega     | 47     | 2–13    |
|       | Total                                 | 389    |         |

### 9.2 Histórias

| ID  | História                                        | Épico | Pri | Pts | Spr |
|-----|-------------------------------------------------|-------|-----|-----|-----|
| H01 | Definir tema, problema e objetivos              | E1    | M   | 5   | 1   |
| H02 | Levantar requisitos funcionais e não funcionais | E1    | M   | 8   | 1   |
| H03 | Modelar os casos de uso                         | E1    | M   | 5   | 1   |
| H04 | Priorizar o product backlog                     | E1    | M   | 3   | 1   |
| H05 | Construir o cronograma até 05/12                | E1    | M   | 3   | 1   |
| H06 | Configurar o repositório GitHub                 | E1    | M   | 3   | 1   |
| H07 | Modelar o banco (entidade-relacionamento)       | E1    | M   | 8   | 2   |
| H08 | Prototipar as telas principais                  | E1    | M   | 5   | 3   |
| H09 | Subir o ambiente com um comando                 | E1    | M   | 5   | 2   |
| H10 | Configurar integração contínua                  | E1    | M   | 2   | 2   |
| H11 | Autenticar com login e senha                    | E2    | M   | 5   | 4   |
| H12 | Encerrar a sessão                               | E2    | M   | 2   | 4   |
| H13 | Renovar o identificador de sessão no login      | E2    | M   | 3   | 4   |
| H14 | Bloquear tentativas repetidas de login          | E2    | M   | 3   | 4   |
| H15 | Cadastrar e editar usuários                     | E2    | M   | 5   | 4   |
| H16 | Atribuir perfis de acesso                       | E2    | M   | 5   | 4   |
| H17 | Validar permissão no servidor a cada requisição | E2    | M   | 8   | 5   |
| H18 | Consultar a trilha de auditoria                 | E2    | S   | 5   | 4   |
| H19 | Alterar a própria senha                         | E2    | S   | 3   | 4   |
| H20 | Versionar o esquema do banco por migrações      | E3    | M   | 5   | 2   |
| H21 | Importar relatório colando o texto              | E3    | M   | 8   | 5   |
| H22 | Importar relatório por arquivo CSV              | E3    | M   | 3   | 6   |
| H23 | Recusar importação sem período informado        | E3    | M   | 3   | 5   |
| H24 | Ver prévia antes de gravar                      | E3    | M   | 5   | 5   |
| H25 | Alertar ao reimportar período existente         | E3    | S   | 3   | 6   |

| ID   | História                                      | Épico | Pri | Pts | Spr |
|------|-----------------------------------------------|-------|-----|-----|-----|
| H26  | Cadastrar e editar parceiros                  | E3    | M   | 5   | 3   |
| H27  | Sugerir categoria pelo nome                   | E3    | S   | 5   | 3   |
| H28  | Construir o gerador de dados sintéticos       | E3    | M   | 5   | 3   |
| H29  | Consultar o histórico de importações          | E3    | M   | 2   | 6   |
| H77  | Comando para limpar e repovoar o banco        | E3    | M   | 2   | 3   |
| H30  | Ver indicadores consolidados do período       | E4    | M   | 5   | 6   |
| H31  | Ver ranking com variação de posição           | E4    | M   | 5   | 6   |
| H32  | Ver a série histórica em gráfico              | E4    | M   | 5   | 6   |
| H33  | Segmentar por regra determinística            | E4    | M   | 8   | 7   |
| H34  | Configurar os limiares da segmentação         | E4    | S   | 3   | 7   |
| H35  | Ver quem entrou e saiu do Top N               | E4    | M   | 5   | 7   |
| H36  | Filtrar e ordenar a lista de parceiros        | E4    | M   | 5   | 7   |
| H37  | Buscar parceiro por nome                      | E4    | M   | 2   | 6   |
| H38  | Exportar a visão filtrada em CSV              | E4    | S   | 3   | 7   |
| H39  | Consultar o próprio desempenho (parceiro)     | E4    | C   | 3   | 9   |
| H40  | Painel rápido mesmo com base grande           | E4    | M   | 3   | 6   |
| H41  | Definir e extrair as variáveis preditivas     | E5    | M   | 8   | 8   |
| H42  | Treinar o modelo de previsão                  | E5    | M   | 8   | 8   |
| H43  | Estimar a probabilidade de queda              | E5    | M   | 5   | 8   |
| H44  | Ver previsão e risco na tela do parceiro      | E5    | M   | 5   | 8   |
| H45  | Disparar o retreino do modelo                 | E5    | S   | 5   | 9   |
| H46  | Comparar o modelo com baselines estatísticos  | E5    | M   | 3   | 8   |
| H47  | Validar a cadeia de compilação da GPU (spike) | E6    | M   | 5   | 3   |
| H48  | Formalizar o problema de otimização           | E6    | M   | 5   | 9   |
| H49  | Implementar o otimizador serial (baseline)    | E6    | M   | 8   | 9   |
| H50  | Configurar as restrições da campanha          | E6    | M   | 5   | 9   |
| H51  | Executar o otimizador e receber o plano       | E6    | M   | 8   | 10  |
| H52  | Recusar planos inviáveis                      | E6    | M   | 3   | 9   |
| H53a | Portar o otimizador para C++                  | E6    | M   | 8   | 10  |
| H53b | Paralelizar as partidas com OpenMP            | E6    | M   | 5   | 10  |
| H54a | Estruturas na GPU e transferência host-device | E6    | M   | 5   | 11  |
| H54b | Kernel CUDA de avaliação da população         | E6    | M   | 5   | 11  |
| H54c | Laço completo do otimizador na GPU            | E6    | M   | 3   | 11  |
| H55  | Escolher o modo de execução                   | E6    | M   | 3   | 10  |
| H56  | Cair para CPU quando não houver GPU           | E6    | M   | 3   | 10  |
| H57  | Ver o benchmark comparativo na interface      | E6    | M   | 8   | 11  |
| H58  | Consultar o histórico de execuções            | E6    | S   | 3   | 10  |
| H59  | Comparar dois planos lado a lado              | E6    | C   | 5   | 11  |
| H60  | Gerar mensagens por segmento e categoria      | E7    | M   | 8   | 12  |
| H61  | Manter fila de mensagens pendentes            | E7    | M   | 5   | 12  |
| H62  | Aprovar, editar ou rejeitar mensagem          | E7    | M   | 5   | 12  |
| H63  | Impedir aprovação sem ação humana             | E7    | M   | 5   | 12  |
| H64  | Consultar histórico de mensagens decididas    | E7    | S   | 3   | 12  |
| H65  | Perguntar sobre os dados em linguagem natural | E8    | S   | 8   | 13  |

| ID  | História                                       | Épico | Pri | Pts | Spr |
|-----|------------------------------------------------|-------|-----|-----|-----|
| H66 | Resposta citando o período usado               | E8    | M   | 5   | 13  |
| H67 | Impedir que o modelo produza números próprios  | E8    | M   | 5   | 13  |
| H68 | Assistente admite quando não sabe              | E8    | M   | 3   | 12  |
| H69 | Testes automatizados no núcleo de regras       | E9    | M   | 8   | 7   |
| H70 | Testes de segurança automatizados              | E9    | M   | 5   | 11  |
| H71 | Exigir Pull Request revisado em main           | E9    | M   | 2   | 2   |
| H78 | Teste de ponta a ponta contra a API no ar      | E9    | M   | 5   | 5   |
| H72 | Subir o sistema seguindo apenas o README       | E9    | M   | 5   | 13  |
| H73 | Documentação técnica final consolidada         | E9    | M   | 5   | 13  |
| H74 | Produzir o vídeo horizontal (até 10 min, 16:9) | E9    | M   | 8   | 13  |
| H75 | Produzir o vídeo vertical (9:16)               | E9    | M   | 5   | 13  |
| H76 | Validar acessibilidade e responsividade        | E9    | S   | 4   | 13  |

### 9.3 Primeiro corte, se o prazo apertar

Ordem de remoção acordada previamente, para que a decisão não seja tomada sob pressão:

- H59 — comparar planos lado a lado (Could)
- H39 — portal do parceiro (Could)
- H27 — sugestão de categoria (Should)
- H38 — exportação em CSV (Should)
- H65 — perguntas livres ao assistente (Should), mantendo H66, H67 e H68, que protegem a confiabilidade

**Nada dos épicos E6 e E9 entra nessa lista: são, respectivamente, o diferencial técnico avaliado e a condição de entrega.**

## 10. Cronograma

Treze sprints: a primeira já entregue e doze ciclos semanais, de segunda a sexta, até 04/12. A mudança de sprints quinzenais para semanais atende ao pedido da avaliação da Sprint 1 e traz três consequências práticas: as histórias de 13 pontos foram quebradas em etapas verificáveis, cada semana passa a ter um entregável demonstrável próprio, e um eventual atraso aparece em sete dias em vez de quatorze.

**Regra permanente: toda sprint termina com algo executável.**

| Spr | Período       | Tema                                | Entregável demonstrável                                                 |
|-----|---------------|-------------------------------------|-------------------------------------------------------------------------|
| 1   | até 05/09     | Planejamento e descoberta           | Documentação completa e repositório configurado                         |
| 2   | 14/09 – 18/09 | Modelagem e fundação                | Banco criado por migração; ambiente sobe com um comando                 |
| 3   | 21/09 – 25/09 | Dados, protótipo e spike de GPU     | Base populada por um comando; protótipo aprovado; kernel de GPU rodando |
| 4   | 28/09 – 02/10 | Autenticação e usuários             | Login com sessão; cadastro de usuários e perfis                         |
| 5   | 05/10 – 09/10 | Autorização e ingestão por texto    | Perfis barrados no servidor; relatório importado com prévia             |
| 6   | 12/10 – 16/10 | Ingestão completa e painel          | CSV, histórico e painel com indicadores e ranking                       |
| 7   | 19/10 – 23/10 | Segmentação e mobilidade do Top N   | Cada parceiro segmentado; quem entrou e saiu do Top N                   |
| 8   | 26/10 – 30/10 | Modelo preditivo                    | Modelo treinado, MAPE melhor que o baseline, previsão na tela           |
| 9   | 02/11 – 06/11 | Otimizador: formalização e baseline | Otimizador serial devolvendo plano válido                               |
| 10  | 09/11 – 13/11 | Otimizador integrado e CPU paralela | Plano na interface; versão C++ com OpenMP e ganho medido                |
| 11  | 16/11 – 20/11 | GPU e benchmark                     | Speedup medido e exibido na tela                                        |
| 12  | 23/11 – 27/11 | Central de comunicação              | Mensagens geradas e fila de aprovação funcionando                       |
| 13  | 30/11 – 04/12 | Assistente e fechamento             | Sistema completo, documentado e os dois vídeos                          |

**Carga:** 389 pontos no total, 27 já entregues na Sprint 1 e 362 distribuídos em doze semanas — média de 30,2 pontos por sprint, contra uma capacidade nominal de cerca de 40. A Sprint 13 está deliberadamente acima da média, e isso está tratado em 10.3.

### 10.1 Marcos

| Data  | Marco                            | Critério de verificação                                     |
|-------|----------------------------------|-------------------------------------------------------------|
| 05/09 | Planejamento entregue            | Documento no Teams e formulário de identificação preenchido |
| 25/09 | Risco de GPU retirado            | Kernel CUDA compilado e conferido contra a CPU              |
| 02/10 | Acesso controlado                | Login com 4 perfis, negação validada no servidor            |
| 16/10 | Dados entrando e painel no ar    | Importação completa e ranking com variação                  |
| 23/10 | Inteligência de negócio pronta   | Segmentação determinística e mobilidade do Top N            |
| 30/10 | Modelo batendo o baseline        | MAPE registrado e inferior ao baseline ingênuo              |
| 13/11 | Otimizador paralelo em CPU       | Plano válido e ganho medido sobre o serial                  |
| 20/11 | Componente avançado demonstrável | Speedup medido e exibido na interface                       |
| 27/11 | Produto completo                 | Fluxo inteiro, do dado bruto à mensagem aprovada            |
| 30/11 | Congelamento de escopo           | Nenhuma funcionalidade nova a partir desta data             |

| Data  | Marco         | Critério de verificação                               |
|-------|---------------|-------------------------------------------------------|
| 05/12 | Entrega final | Sistema, código, documentação, banco e os dois vídeos |

## 10.2 Riscos e mitigações

| #  | Risco                                                              | Prob. | Mitigação                                                                                                                                                                                          |
|----|--------------------------------------------------------------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| R1 | A cadeia de compilação de GPU não funcionar no ambiente disponível | Média | Spike antecipado para a Sprint 3, oito semanas antes de ser necessário. Alternativas em ordem: CuPy, Numba com destino CUDA, OpenCL                                                                |
| R2 | Escopo completo sem folga no calendário                            | Alta  | A média de 30 pontos por semana não deixa margem. Válvula de escape definida em 10.3: o assistente analítico sai primeiro. A velocidade real das Sprints 2 e 3 recalibra o plano antes da Sprint 8 |
| R3 | O speedup em GPU ficar abaixo da meta de 5x                        | Média | Ampliar a escala do cenário com o gerador sintético. Se ainda assim não atingir, reportar o resultado medido com a análise do porquê                                                               |
| R4 | Trilha do núcleo concentrada em uma pessoa                         | Alta  | E5 e E6 somam 116 pontos sob o mesmo responsável, sem par. Mitigação por transferência de conhecimento: decisões em ADR e apresentação do código do núcleo nas revisões das Sprints 10 e 11        |
| R5 | Modelo preditivo não superar o baseline ingênuo                    | Média | Baselines implementados antes do modelo, para saber cedo qual é o alvo; o otimizador segue com a melhor estimativa disponível                                                                      |
| R6 | Ausência ou queda de participação de um integrante                 | Média | O ciclo semanal expõe o problema em sete dias; acompanhamento por commits e board em cada orientação                                                                                               |
| R7 | Vídeos deixados para os últimos dias                               | Alta  | A Sprint 13 já nasce sobrecarregada. Roteiros escritos na Sprint 12; congelamento em 30/11; gravação distribuída entre 01/12 e 04/12                                                               |
| R8 | Escopo crescer durante o semestre                                  | Média | Toda ideia nova entra no backlog como história e disputa prioridade; não é incorporada direto à sprint em andamento                                                                                |

## 10.3 A semana que não fecha

A Sprint 13 está com 45 pontos contra uma média de 30, e é justamente a semana dos dois vídeos. Não é erro de distribuição: é o que sobra quando o escopo completo é dividido por doze semanas, com as outras onze já entre 22 e 32 pontos.

Registrar isso agora tem um propósito: quando a pressão chegar, a decisão já estará tomada em vez de improvisada. O assistente analítico é a válvula de escape — se qualquer semana entre a 8 e a 12 atrasar, ele é o que não se constrói. A justificativa é que ele não é o componente de inteligência avaliado: esse papel cabe ao modelo preditivo e ao otimizador paralelo.

**O que não pode sair em nenhuma hipótese: os dois vídeos, a documentação final e a reprodutibilidade a partir do README. Sem eles não há entrega.**

*As doze sprints semanais foram distribuídas a partir das duas datas oficiais divulgadas: entrega da Sprint 1 em 05/09 e entrega final em 05/12. O calendário de orientações da disciplina tem precedência: o cronograma será ajustado preservando a sequência de temas e os marcos técnicos.*

## 11. Repositório GitHub

Endereço: [github.com/PedroMiranda243/gih-fabrica-de-software](https://github.com/PedroMiranda243/gih-fabrica-de-software)

### 11.1 O que já está configurado

| Recurso                  | Situação                                                                       |
|--------------------------|--------------------------------------------------------------------------------|
| Visibilidade             | Público                                                                        |
| Documentação             | Sete documentos em docs/, cobrindo todos os entregáveis desta sprint           |
| Diagrama de casos de uso | Publicado em SVG, renderizando diretamente no GitHub                           |
| Labels                   | Por épico (E1 a E9), prioridade (MoSCoW) e tipo (feat, fix, docs, test, spike) |
| Milestones               | Sete, uma por sprint, com data de encerramento                                 |
| Issues                   | Histórias das Sprints 1 a 3 abertas, com critérios de aceite                   |
| Modelos                  | Issue (história e defeito) e Pull Request, com Definition of Done              |
| Proteção do ramo main    | Exige 1 aprovação em Pull Request; bloqueia push forçado e exclusão            |
| Contribuição             | CONTRIBUTING.md com padrão de ramos, commits e Definition of Done              |

### 11.2 Estrutura do repositório

- README.md — problema, solução, stack, execução e equipe
- CONTRIBUTING.md — fluxo de branches, commits e Pull Requests
- docs/01-visao-do-produto.md — tema, problema, objetivos e público-alvo
- docs/02-requisitos.md — requisitos funcionais e não funcionais, com rastreabilidade
- docs/03-casos-de-uso.md — diagrama e especificação dos casos de uso
- docs/04-product-backlog.md — épicos e histórias priorizadas
- docs/05-cronograma.md — sprints, marcos e riscos
- docs/06-equipe-e-processo.md — papéis, cerimônias e Definition of Done
- docs/07-arquitetura-preliminar.md — visão de contêineres, stack e decisões de arquitetura

### 11.3 Regras de dados e confidencialidade

O repositório é público, e por isso valem sem exceção:

- Nenhum dado real de nenhuma operação. Toda massa de demonstração é gerada por script.
- Nenhum segredo versionado: senhas e tokens ficam em .env, bloqueado pelo .gitignore.
- Nenhum nome de organização, pessoa ou parceiro real em código, documentação ou dados de exemplo.
- Verificação de conteúdo sensível executada antes de cada entrega.

*Projeto acadêmico. © 2026 os autores — todos os direitos reservados. Repositório público significa visível para avaliação e portfólio; não constitui licença de uso, cópia ou redistribuição.*

## Parte II — Sprint 02: Arquitetura e Modelagem

Esta parte apresenta a estrutura técnica que sustenta o desenvolvimento do sistema: como a solução está organizada, como as classes se relacionam, como os dados são modelados, como as telas se comportam e qual é o estado atual do banco e do repositório.

Tudo aqui decorre do que foi definido na Parte I. O problema, os objetivos e os requisitos não mudaram — o que muda é que agora existe uma arquitetura que responde a eles, e código rodando que a comprova.

### Duas numerações de sprint

A equipe trabalha em sprints semanais, e a disciplina avalia por sprints de entrega. Como os dois sistemas de numeração convivem na documentação do repositório, vale deixar a correspondência explícita antes de qualquer outra coisa.

| Entrega da disciplina                 | Prazo      | Sprints da equipe | O que cobre                                                              |
|---------------------------------------|------------|-------------------|--------------------------------------------------------------------------|
| Sprint 01 — planejamento e descoberta | 05/09/2026 | 1                 | Tema, problema, objetivos, requisitos, casos de uso, backlog, cronograma |
| Sprint 02 — arquitetura e modelagem   | 19/09/2026 | 2 a 5             | Arquitetura, classes, MER, modelo relacional, protótipo, banco criado    |

**A segunda entrega cobre quatro das nossas sprints, e não uma.** Na data desta entrega o banco já está criado e migrado, a API já autentica com perfis e já ingere relatórios, e o risco técnico do núcleo em GPU já foi retirado por medição.

# 1. Arquitetura do sistema

O sistema se organiza em cinco componentes com responsabilidades separadas. A separação não é estética: ela é o que permite medir o núcleo computacional isoladamente, trocar o modelo preditivo sem tocar na interface, e garantir que o assistente de linguagem nunca produza um número.

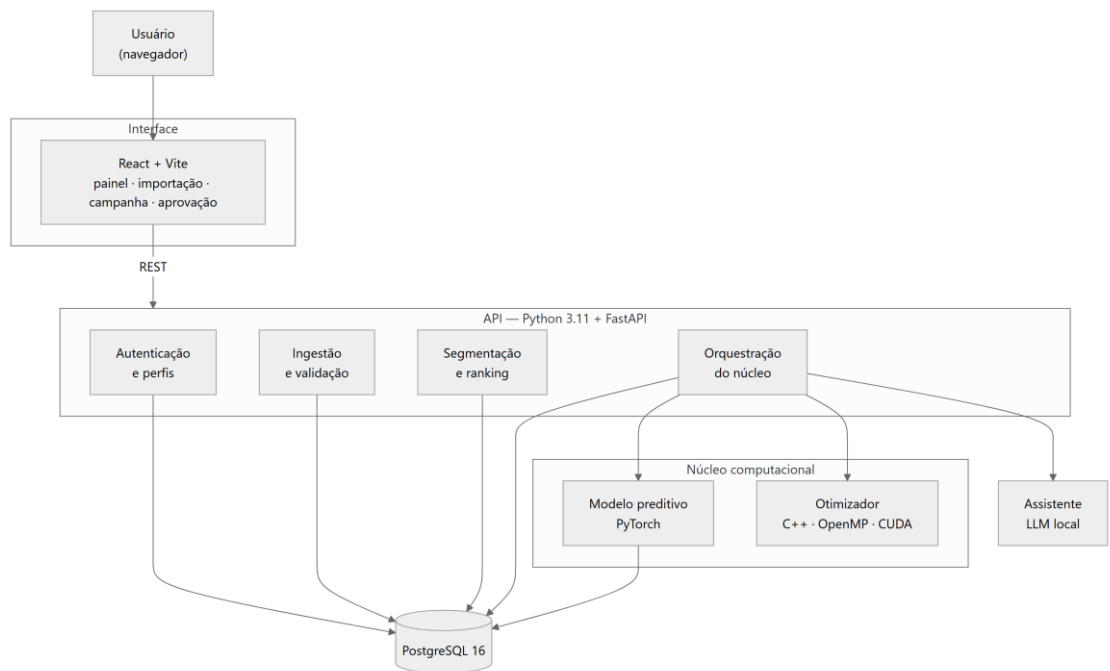


Figura 1 — Arquitetura geral: interface, API, banco, núcleo computacional, modelo preditivo e assistente.

## 1.1 Componentes e responsabilidades

| Camada     | Responsabilidade                                           | O que não faz                       |
|------------|------------------------------------------------------------|-------------------------------------|
| web/       | Exibe, formata e coleta entrada                            | Nenhuma regra de negócio            |
| api/       | Regras de negócio, autorização, persistência, orquestração | Cálculo pesado dentro da requisição |
| nucleo/    | Otimizador combinatório: serial, OpenMP e CUDA             | Não conhece autenticação nem banco  |
| modelo/    | Previsão de faturamento e probabilidade de queda           | Não decide ação comercial           |
| assistente | Redige texto sobre fatos já apurados                       | Não calcula número                  |

## 1.2 Tecnologias previstas

| Camada     | Tecnologia                           | Por quê                                                                       |
|------------|--------------------------------------|-------------------------------------------------------------------------------|
| Interface  | React + Vite                         | Componentização e recarga rápida no desenvolvimento                           |
| API        | Python 3.11 + FastAPI                | Tipagem em tempo de execução e especificação da API gerada automaticamente    |
| Banco      | PostgreSQL 16 + Alembic              | Restrições de integridade no banco e evolução do esquema versionada           |
| Núcleo     | C++17 + OpenMP + CUDA                | Paralelismo real em CPU e GPU — é o componente avaliado por Tópicos Avançados |
| Modelo     | PyTorch + NumPy                      | Treino local, sem serviço externo                                             |
| Assistente | Modelo de linguagem local via Ollama | Roda na própria máquina: sem custo por uso e sem enviar dados para fora       |

| Camada        | Tecnologia     | Por quê                                |
|---------------|----------------|----------------------------------------|
| Empacotamento | Docker Compose | O ambiente inteiro sobe com um comando |

A disciplina orienta que JavaScript não seja a tecnologia do núcleo computacional. A arquitetura respeita isso integralmente: JavaScript existe apenas na interface. Inteligência artificial, processamento intensivo, paralelismo, otimização e GPU estão em Python, C++ e CUDA.

### 1.3 Como os componentes se comunicam

- A interface conversa com a API por REST, em JSON, sobre HTTP. Nenhum outro componente é acessível a partir do navegador.
- A API é o único componente que fala com o banco. Toda leitura e escrita passa por consulta parametrizada.
- A API entrega ao núcleo um cenário já montado — uplift, custo e categoria por parceiro — e recebe de volta um plano e o tempo de execução.
- A API entrega ao modelo preditivo a série histórica e recebe previsão por parceiro.
- A API entrega ao assistente um bloco de fatos já calculados; o assistente devolve texto, nunca número.

### 1.4 A integração com o componente computacional avançado

O otimizador resolve um problema combinatório: dados N parceiros e A ações possíveis, escolher no máximo uma ação por parceiro de forma a maximizar o uplift esperado sem violar o orçamento, o número máximo de parceiros e a cota por categoria. O espaço de busca tem  $(A+1)^N$  combinações — inviável por força bruta na escala de referência de 2.000 parceiros.

A solução é uma metaheurística populacional escrita pela equipe, com três implementações da mesma interface: serial, paralela em CPU com OpenMP e paralela em GPU com CUDA. As três produzem o mesmo tipo de resultado, o que é o que torna o benchmark comparável.

**O risco técnico desse componente já foi retirado, por medição.** Uma investigação técnica dedicada validou a cadeia de compilação e mediu o ganho real oito semanas antes de ser necessário. O kernel CUDA chega a 100x sobre o baseline serial, e o resultado confere com o cálculo em CPU com erro relativo zero.

A medição também produziu um achado que mudou o projeto: comparada com a CPU paralela, e não com a serial, a GPU só compensa se a população permanecer na memória dela entre as gerações. Com transferência a cada geração o ganho cai para 1,1x; sem ela, chega a 11x. Isso deixou de ser otimização e virou requisito de projeto do componente.

### 1.5 Decisões de arquitetura registradas

Cada decisão não óbvia está registrada com a situação que a motivou, as alternativas consideradas e as consequências assumidas. São nove até aqui:

| ADR | Decisão                                                            | Consequência assumida                                                   |
|-----|--------------------------------------------------------------------|-------------------------------------------------------------------------|
| 001 | Python + C++/CUDA como stack do núcleo                             | Duas linguagens no projeto, em troca de paralelismo real                |
| 002 | Metaheurística paralela, não solver pronto                         | Mais trabalho; é o componente que a disciplina avalia                   |
| 003 | Segmentação e ranking determinísticos, fora do modelo de linguagem | Reprocessar a mesma base dá o mesmo resultado                           |
| 004 | Degradação para CPU quando não houver GPU                          | O sistema nunca deixa de funcionar por falta de hardware                |
| 005 | CUDA via NVRTC, sem instalar o toolkit                             | Superada pela 006 no mesmo dia; mantida como registro                   |
| 006 | Toolchain nativo: compilar com nvcc e MSVC                         | Quem mexe no núcleo precisa do compilador instalado                     |
| 007 | Sessão com estado no servidor, não token autocontido               | Uma consulta a mais por requisição; encerrar sessão tem efeito imediato |

| ADR | Decisão                                                        | Consequência assumida                                     |
|-----|----------------------------------------------------------------|-----------------------------------------------------------|
| 008 | A auditoria grava em transação própria                         | Registro a mais é ruído; registro a menos é ponto cego    |
| 009 | Formato do relatório importado tolerante, guiado por cabeçalho | Tolerância esconde erro; mitigada pela mensagem de recusa |

## 2. Diagrama de classes

O sistema tem três camadas com naturezas diferentes: entidades de domínio, serviços com a regra de negócio, e o núcleo computacional em C++ e CUDA. Apresentá-las num único desenho produziria uma figura de oitenta caixas, ilegível impressa. São oito diagramas, cada um legível sozinho, e o primeiro mostra como eles se ligam.

### 2.1 Visão de integração

Onde cada camada vive e o que atravessa a fronteira entre elas.

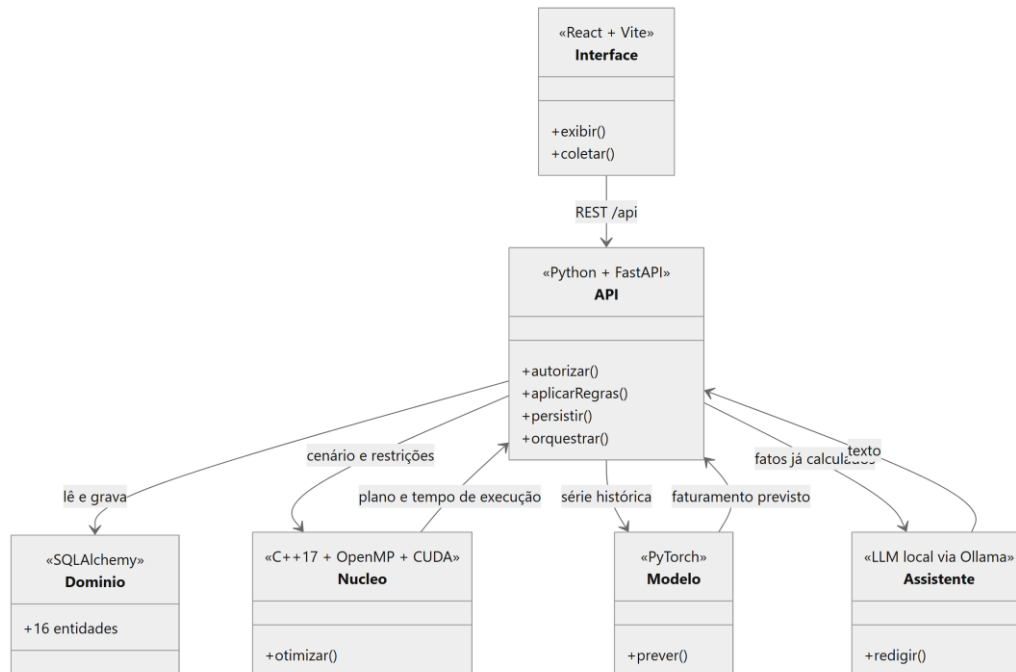


Figura 2 — Visão de integração entre as camadas, com as tecnologias de cada uma.

Três fronteiras não se atravessam: a interface não conhece regra de negócio, o núcleo não conhece autenticação nem banco, e o assistente não calcula número.

## 2.2 Domínio — acesso, sessões e auditoria

Quem entra no sistema, por quanto tempo, e o rastro que fica.

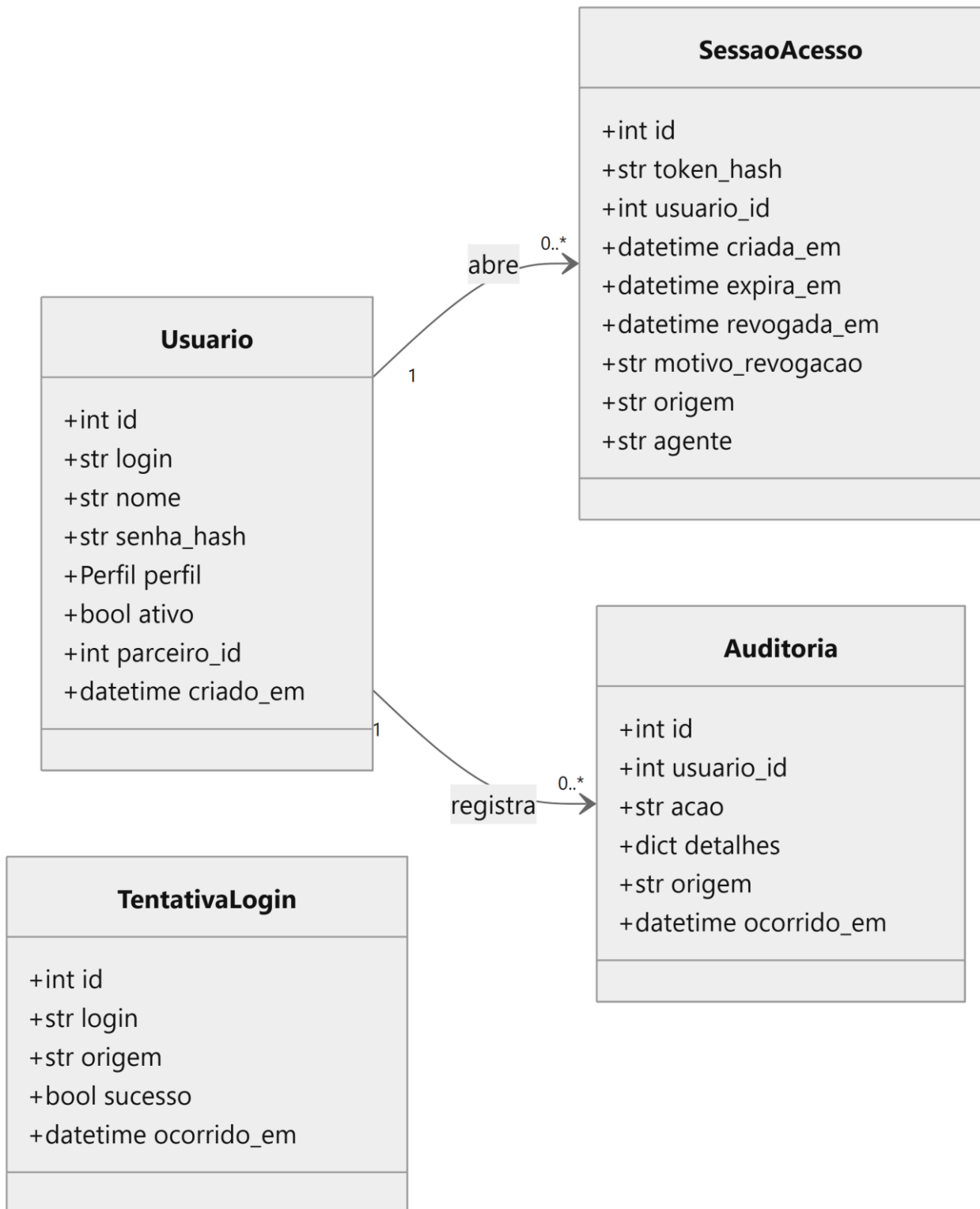


Figura 3 — Entidades de controle de acesso: usuário, sessão, tentativas de login e trilha de auditoria.

A sessão guarda o hash do identificador, nunca ele próprio: quem conseguir ler a tabela não consegue se passar por ninguém. E TentativaLogin não se liga a Usuario de propósito — o login é texto livre, porque tentativa contra usuário inexistente também precisa ser contada. É justamente o login desconhecido que o ataque por dicionário usa.

## 2.3 Domínio — parceiros e dados de desempenho

O núcleo informacional do produto: quem são os parceiros, o que faturaram, e como foram classificados.

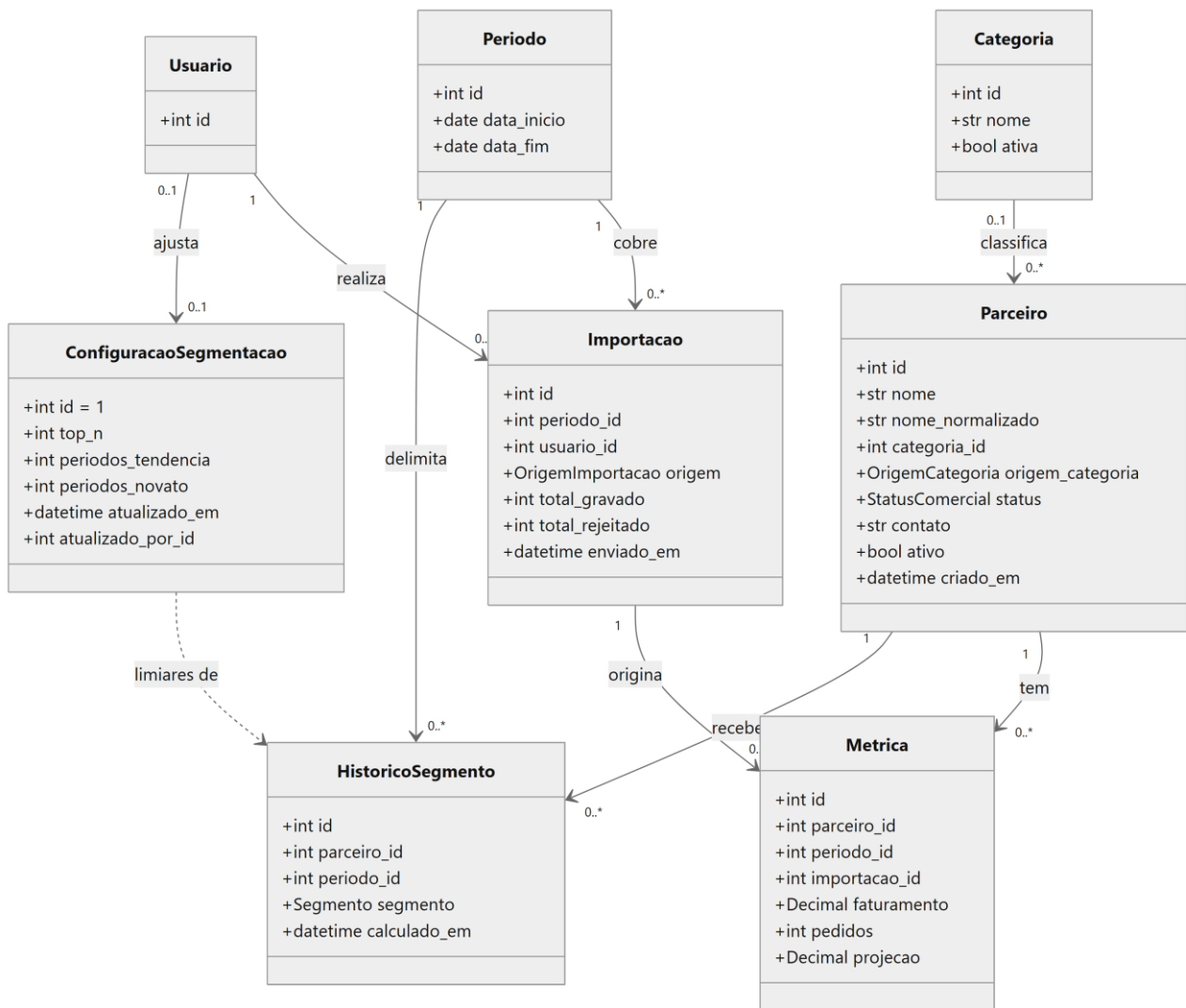


Figura 4 — Entidades de negócio: categoria, parceiro, período, importação, métrica e histórico de segmento.

Métrica não tem atributo de ticket médio: ele é faturamento dividido por pedidos, calculado na consulta. Como coluna, divergiria das parcelas que o originam na primeira correção de dado.

## 2.4 Domínio — núcleo computacional e comunicação

As entidades que registram previsões, execuções do otimizador, planos de campanha e mensagens.

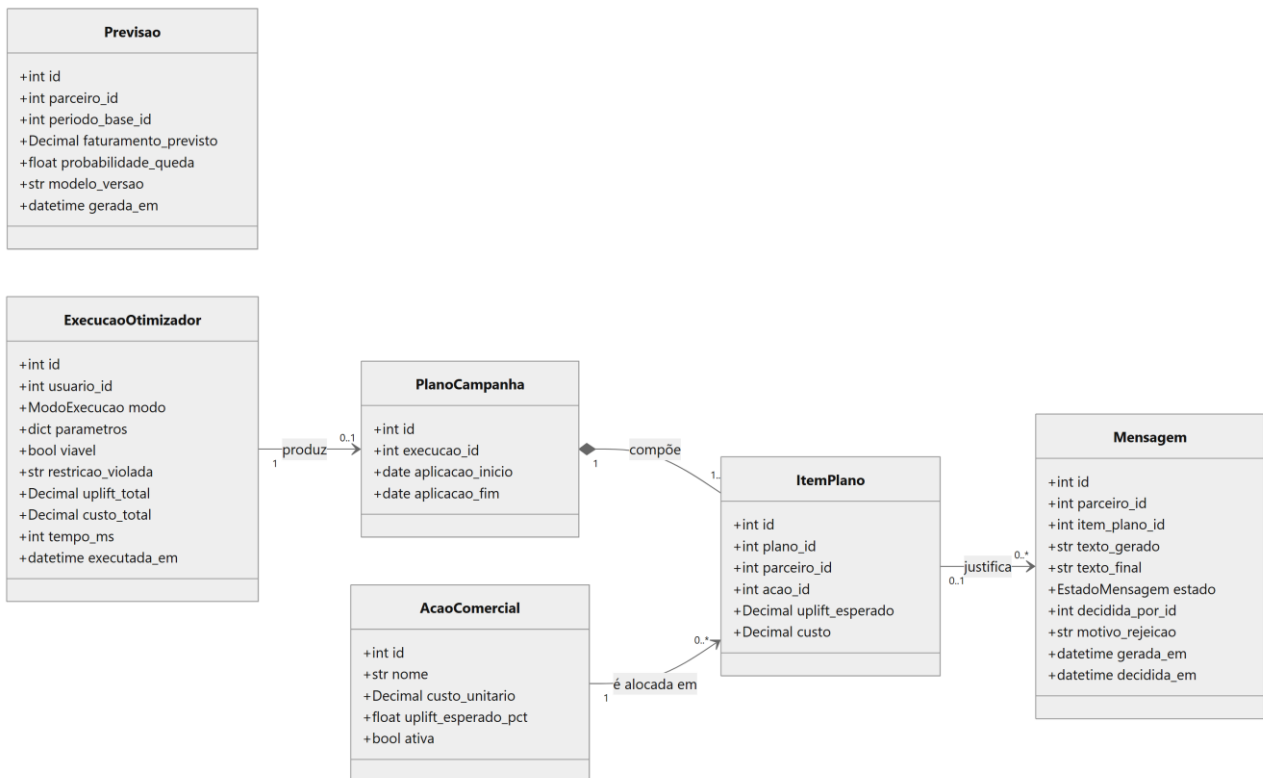


Figura 5 — Entidades do núcleo e da comunicação: previsão, ação, execução, plano, item e mensagem.

A execução do otimizador produz zero ou um plano: ou o plano respeita todas as restrições, ou não existe plano. A execução inviável fica registrada com o motivo, porque explicar por que não deu é mais útil que sumir com a tentativa.

## 2.5 Serviços — acesso e segurança

As entidades acima são anêmicas de propósito: carregam dados e restrições, e a regra de negócio vive nos serviços. A razão é testabilidade — a regra fica exercitável sem instanciar entidade.

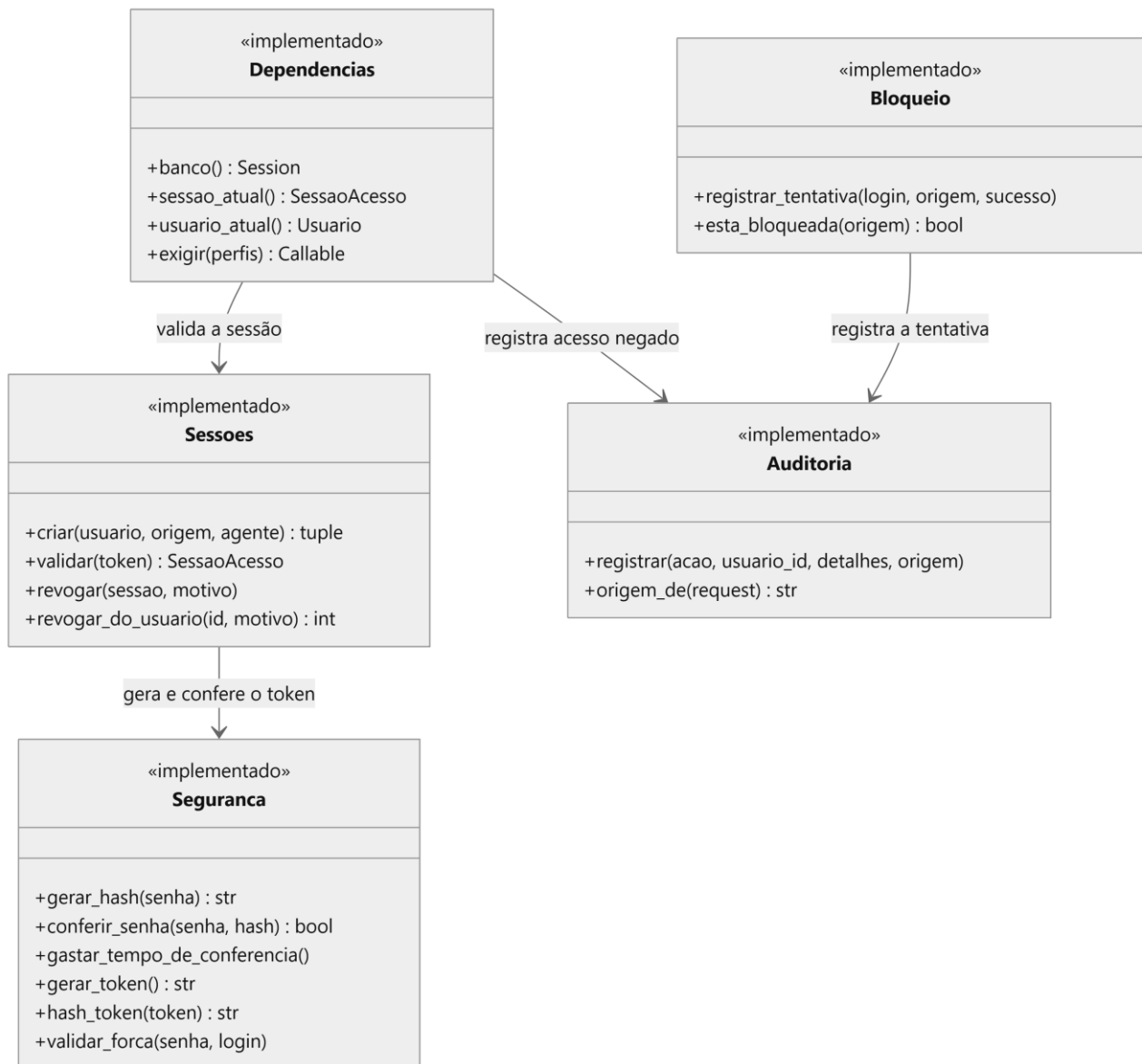


Figura 6 — Serviços de autenticação, sessão, bloqueio por força bruta e auditoria. Todos implementados.

A autorização é dependência declarada na rota, e não uma verificação dentro de cada função: esquecer uma verificação é silencioso, e esquecer a dependência reprova o teste que percorre cada endpoint contra cada perfil.

## 2.6 Serviços — ingestão e análise

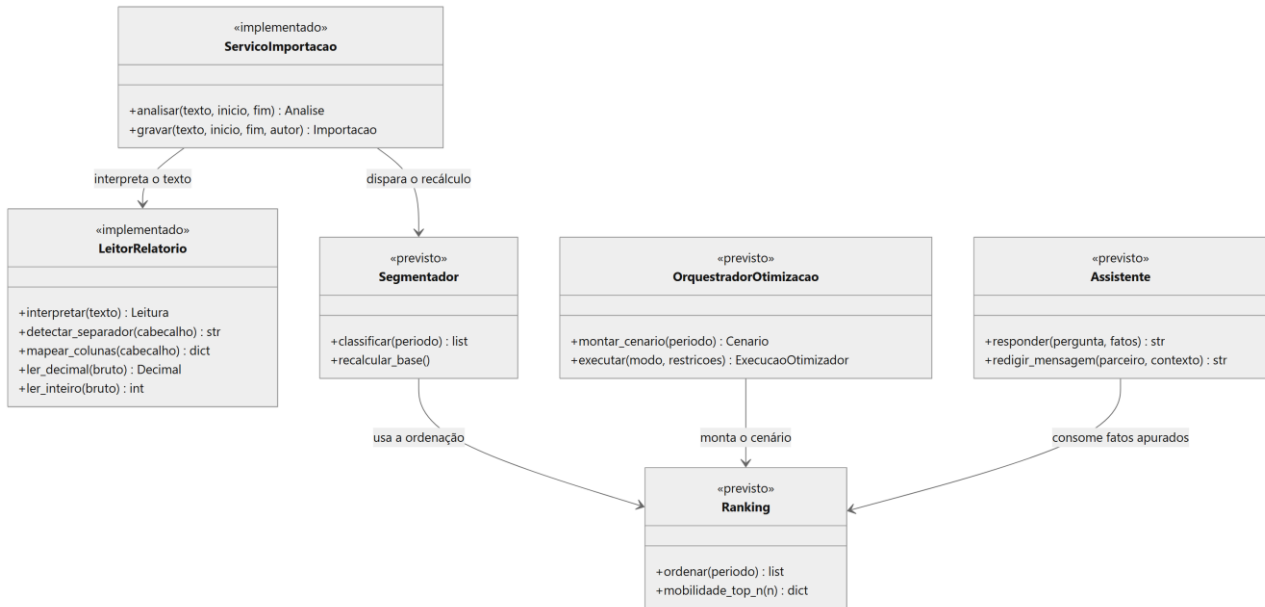


Figura 7 — Serviços de ingestão, segmentação, ranking, otimização e assistente. Os dois primeiros implementados.

O interpretador do relatório não toca no banco: é função pura de texto para resultado. É isso que permite a prévia da importação usar exatamente o mesmo código da gravação, sem risco de a prévia mostrar uma coisa e a gravação fazer outra.

## 2.7 Núcleo computacional — as estruturas

O que entra no otimizador, o que sai, e quem avalia um candidato.

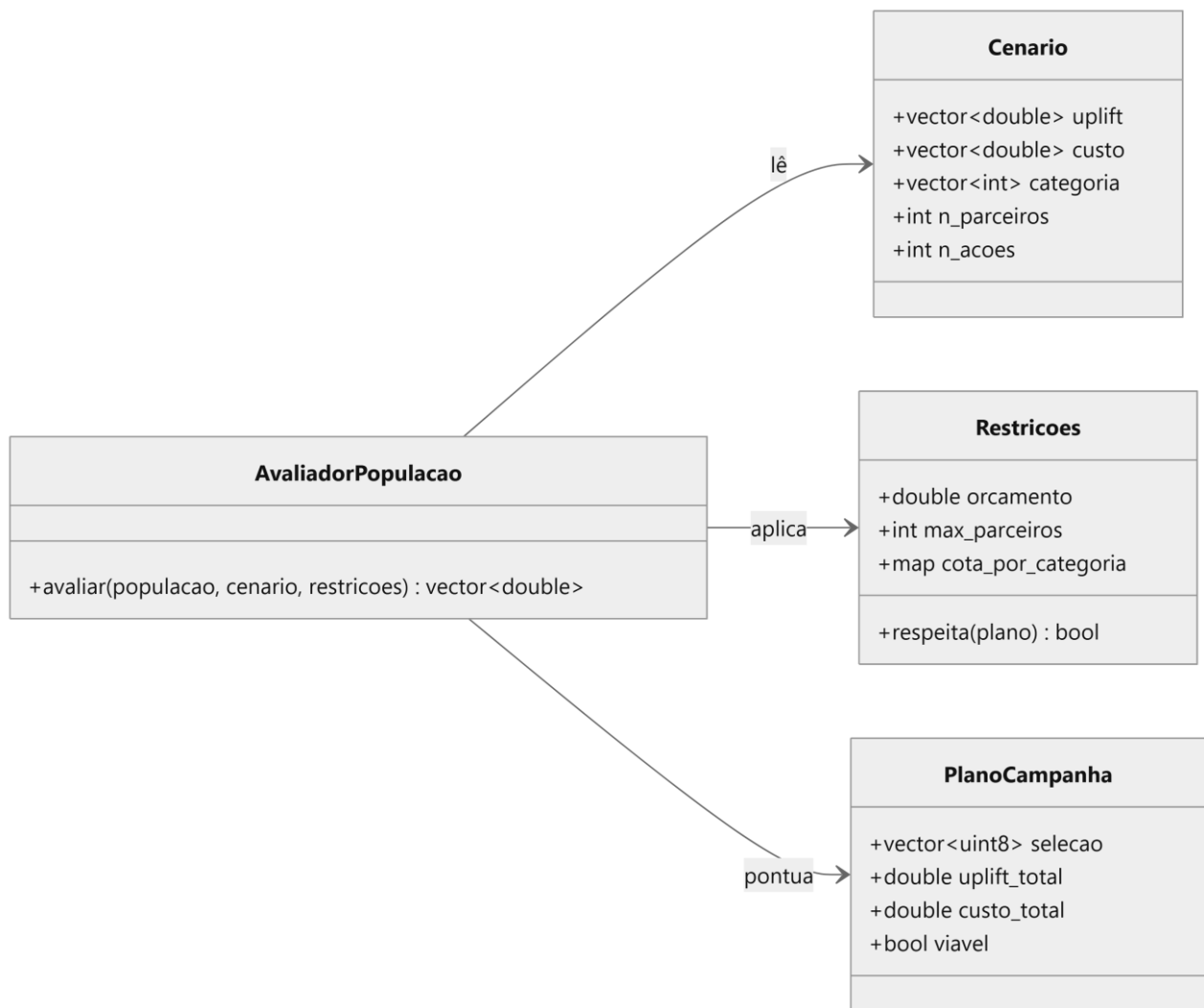


Figura 8 — Estruturas do núcleo em C++: cenário, restrições, plano e avaliador de população.

## 2.8 Núcleo computacional — as três implementações

Três implementações da mesma interface, para que o benchmark compare o que é comparável.

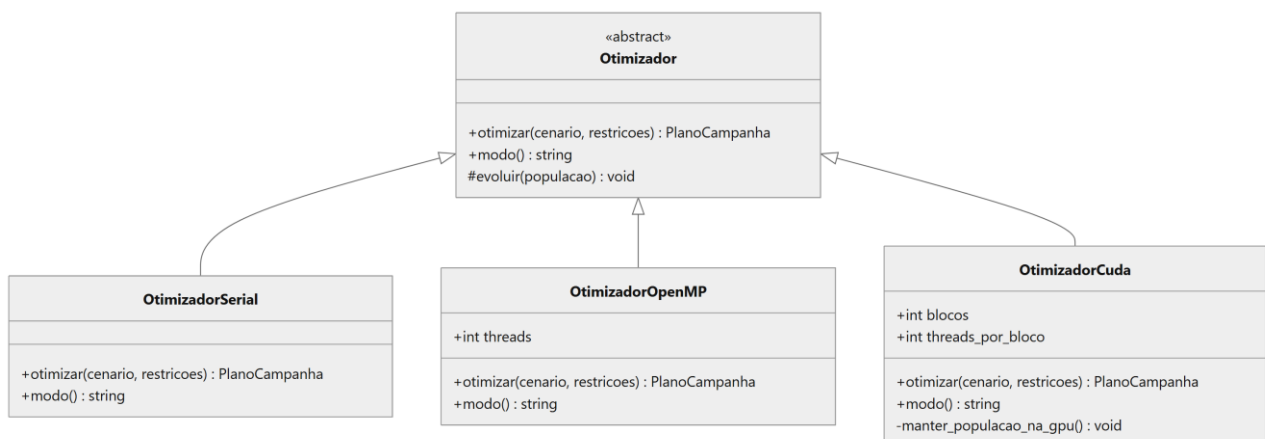


Figura 9 — Hierarquia do otimizador: serial, paralelo em CPU com OpenMP e paralelo em GPU com CUDA.

O método privado de manter a população na GPU não é detalhe de implementação: é o que justifica a classe existir. Sem ele, a GPU ganha apenas 1,1x da CPU paralela; com ele, 11x.

## 2.9 O que já está implementado

Um diagrama que promete código inexistente é pior que diagrama nenhum, porque ninguém reconfere. A separação abaixo pode ser verificada no repositório.

| Camada          | Implementado                                                                           | Previsto                                     |
|-----------------|----------------------------------------------------------------------------------------|----------------------------------------------|
| <b>Domínio</b>  | As 16 entidades, com restrições no banco                                               | —                                            |
| <b>Serviços</b> | Segurança, sessões, bloqueio, auditoria, dependências, leitor de relatório, importação | Segmentador, ranking, otimização, assistente |
| <b>Rotas</b>    | Sessão, usuários, importações, auditoria, saúde                                        | Painel, otimização, mensagens, assistente    |
| <b>Núcleo</b>   | Kernel de avaliação validado em CUDA e OpenMP                                          | Otimizador e as três implementações          |
| <b>Modelo</b>   | —                                                                                      | Previsor em PyTorch                          |

Cobertura de teste da parte implementada: **179 testes automatizados, 96% das linhas.**

### 3. Modelo Entidade-Relacionamento

O modelo conceitual responde "o que o sistema precisa saber", na linguagem do negócio e sem decisão de implementação. São dezesseis entidades.

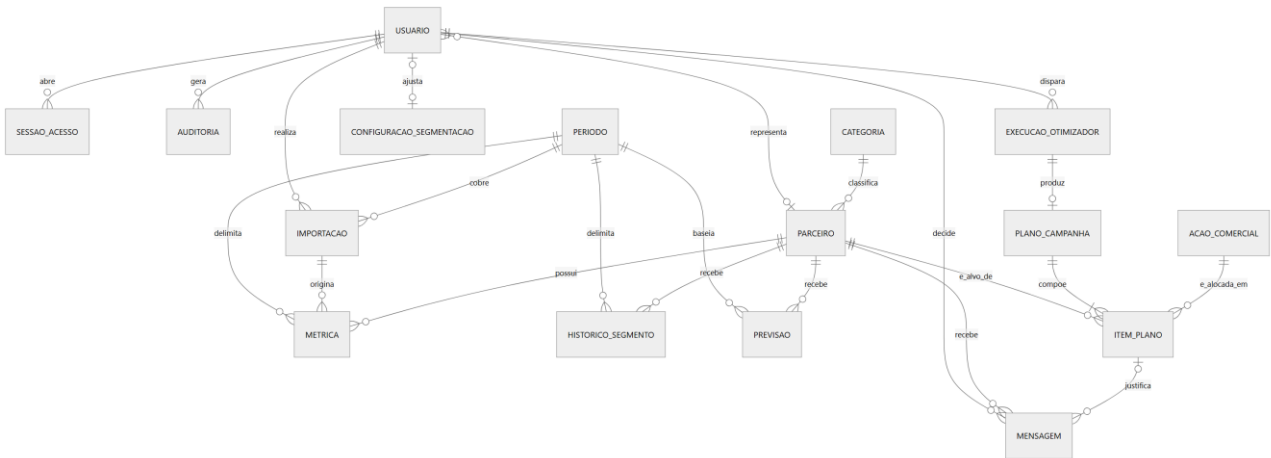


Figura 10 — Modelo conceitual: entidades e relacionamentos, com cardinalidade em notação pé-de-galinha.

A entidade de tentativas de login não aparece no diagrama porque não se relaciona com nenhuma outra, e isso é deliberado: ela registra tentativas inclusive contra logins inexistentes, e ligá-la ao usuário quebraria justamente o caso que ela existe para cobrir.

#### 3.1 Entidades e atributos

Os atributos estão aqui, e não dentro das caixas do desenho: com dezesseis entidades e mais de cem atributos, a figura ficaria ilegível impressa.

| Entidade           | O que representa             | Atributos                                                                 |
|--------------------|------------------------------|---------------------------------------------------------------------------|
| Usuario            | Quem acessa o sistema        | login, nome, senha (só hash), perfil, situação, data de cadastro          |
| SessaoAcesso       | Sessão autenticada em curso  | identificador, início, expiração, revogação e motivo, origem, navegador   |
| TentativaLogin     | Tentativa de autenticação    | login tentado, origem, resultado, momento                                 |
| Auditoria          | Ação sensível praticada      | autor, ação, parâmetros, origem, momento                                  |
| Categoria          | Ramo de atuação              | nome, situação                                                            |
| Parceiro           | Comércio da rede             | nome, categoria, origem da categoria, status comercial, contato, situação |
| Periodo            | Janela de tempo do relatório | data inicial, data final                                                  |
| Importacao         | Uma carga de relatório       | período, autor, origem, total gravado, total rejeitado, momento           |
| Metrica            | Desempenho num período       | faturamento, número de pedidos, projeção                                  |
| HistoricoSegmento  | Classificação num período    | segmento, momento do cálculo                                              |
| Previsao           | Estimativa do modelo         | faturamento previsto, probabilidade de queda, versão do modelo            |
| AcaoComercial      | Tipo de ação da campanha     | nome, custo unitário, uplift esperado, situação                           |
| ExecucaoOtimizador | Uma rodada do otimizador     | modo, parâmetros, viabilidade, restrição violada, uplift, custo, tempo    |
| PlanoCampanha      | Plano de uma execução viável | janela de aplicação                                                       |
| ItemPlano          | Par parceiro–ação escolhido  | uplift esperado, custo                                                    |
| Mensagem           | Comunicação para um parceiro | texto gerado, texto final, estado, autor da decisão, motivo               |

### 3.2 Relacionamentos que carregam regra de negócio

- Métrica é única por parceiro e período — é o que impede uma reimportação de duplicar a série e corromper a segmentação por tendência.
- Execução do otimizador produz zero ou um plano: ou o plano respeita todas as restrições, ou não existe plano.
- Plano de campanha compõe um ou mais itens, no máximo um por parceiro. Item sem plano não tem significado.
- Usuário representa zero ou um parceiro. Só o perfil Parceiro usa esse vínculo, e é ele que restringe o acesso ao próprio desempenho.

## 4. Modelo relacional

A conversão do modelo conceitual em tabelas. Toda tabela tem chave primária inteira e sequencial: a escolha por chave substituta mantém as chaves estrangeiras com uma coluna só, e permite corrigir o nome de um parceiro sem reescrever as métricas dele.

*Notação: o sublinhado marca chave primária e o asterisco marca chave estrangeira.*

```
usuario(id, login, nome, senha_hash, perfil, ativo, parceiro_id*, criado_em)
sessao_acesso(id, token_hash, usuario_id*, criada_em, expira_em, revogada_em,
               motivo_revogacao, origem, agente)
tentativa_login(id, login, origem, sucesso, ocorrido_em)
auditoria(id, usuario_id*, acao, detalhes, origem, ocorrido_em)

categoria(id, nome, ativa)
parceiro(id, nome, categoria_id*, origem_categoria, status, contato, ativo, criado_em)

periodo(id, data_inicio, data_fim)
importacao(id, periodo_id*, usuario_id*, origem, total_gravado, total_rejeitado, enviado_em)
metrica(id, parceiro_id*, periodo_id*, importacao_id*, faturamento, pedidos, projecao)
historico_segmento(id, parceiro_id*, periodo_id*, segmento, calculado_em)

previsao(id, parceiro_id*, periodo_base_id*, faturamento_previsto, probabilidade_queda,
          modelo_versao, gerada_em)
acao_comercial(id, nome, custo_unitario, uplift_esperado_pct, ativa)
execucao_otimizador(id, usuario_id*, modo, parametros, viavel, restricao_violada,
                    uplift_total, custo_total, tempo_ms, executada_em)
plano_campanha(id, execucao_id*, aplicacao_inicio, aplicacao_fim)
item_plano(id, plano_id*, parceiro_id*, acao_id*, uplift_esperado, custo)

mensagem(id, parceiro_id*, item_plano_id*, texto_gerado, texto_final, estado,
          decidida_por_id*, motivo_rejeicao, gerada_em, decidida_em)
```

### 4.1 Detalhamento das tabelas centrais

As demais tabelas seguem o mesmo padrão e estão detalhadas coluna a coluna na documentação do repositório.

#### usuario

| Coluna      | Tipo         | Chave         | Restrição                                 |
|-------------|--------------|---------------|-------------------------------------------|
| id          | serial       | PK            |                                           |
| login       | varchar(60)  |               | único                                     |
| nome        | varchar(120) |               |                                           |
| senha_hash  | varchar(255) |               | somente hash Argon2id                     |
| perfil      | enum         |               | ADMINISTRADOR, GESTOR, ANALISTA, PARCEIRO |
| ativo       | boolean      |               | padrão verdadeiro                         |
| parceiro_id | integer      | FK → parceiro | nulo, exceto no perfil PARCEIRO           |
| criado_em   | timestampz   |               | padrão now()                              |

Uma restrição do banco garante que ter vínculo com parceiro e ser do perfil Parceiro são a mesma condição, nos dois sentidos.

#### metrica — a tabela central do sistema

| Coluna | Tipo   | Chave | Restrição |
|--------|--------|-------|-----------|
| id     | serial | PK    |           |

| Coluna        | Tipo          | Chave           | Restrição                        |
|---------------|---------------|-----------------|----------------------------------|
| parceiro_id   | integer       | FK → parceiro   | único em conjunto com periodo_id |
| periodo_id    | integer       | FK → periodo    |                                  |
| importacao_id | integer       | FK → importacao | de qual carga o registro veio    |
| faturamento   | numeric(12,2) |                 | maior ou igual a zero            |
| pedidos       | integer       |                 | maior ou igual a zero            |
| projecao      | numeric(12,2) |                 | nulo permitido                   |

**Não existe coluna de ticket médio.** Ele é faturamento dividido por pedidos, calculado na consulta. Como coluna, divergiria das parcelas que o originam na primeira correção de dado.

## 4.2 O que o banco garante sozinho

Nove restrições impedem estado inválido independentemente do código da aplicação. É a diferença entre uma regra que vale e uma regra que valeria se ninguém esquecesse de chamá-la.

| Restrição                       | Garante                                                                           |
|---------------------------------|-----------------------------------------------------------------------------------|
| usuario · vínculo com parceiro  | Vínculo se e somente se o perfil for PARCEIRO                                     |
| parceiro · categoria com origem | Categoria sempre acompanhada da procedência                                       |
| periodo · ordem                 | Data final nunca anterior à inicial                                               |
| metrica · faturamento e pedidos | Nenhum dos dois negativo                                                          |
| previsao · probabilidade        | Probabilidade entre zero e um                                                     |
| acao_comercial · custo          | Custo de ação não negativo                                                        |
| execucao_otimizador · motivo    | Execução inviável tem o motivo registrado                                         |
| mensagem · autor da decisão     | Mensagem decidida tem autor e data — nenhuma sai do estado pendente sem um humano |

## 4.3 Índices

Cada índice existe por causa de uma consulta concreta, e não por precaução.

| Índice               | Colunas                 | Serve a                                      |
|----------------------|-------------------------|----------------------------------------------|
| parceiro por nome    | nome                    | Busca no painel e casamento na importação    |
| métrica por período  | periodo_id, faturamento | Ranking do período sem varrer a tabela       |
| segmento por período | periodo_id, segmento    | Filtro por segmento no painel                |
| auditoria por data   | ocorrido_em             | Consulta da trilha por intervalo             |
| sessão por usuário   | usuario_id              | Derrubar as sessões de um usuário de uma vez |
| tentativa por origem | origem, ocorrido_em     | Contagem de falhas na janela do bloqueio     |
| mensagem por estado  | estado                  | Fila de aprovação                            |

## 5. Protótipo das telas principais

Quatro telas navegáveis, construídas em HTML e CSS autorais. O protótipo não é uma imagem estática: é uma página que se abre no navegador e permite percorrer as telas, e está versionada no repositório junto com o restante do projeto.

Protótipo navegável: [github.com/PedroMiranda243/gih-fabrica-de-software/blob/main/docs/prototipo/index.html](https://github.com/PedroMiranda243/gih-fabrica-de-software/blob/main/docs/prototipo/index.html)

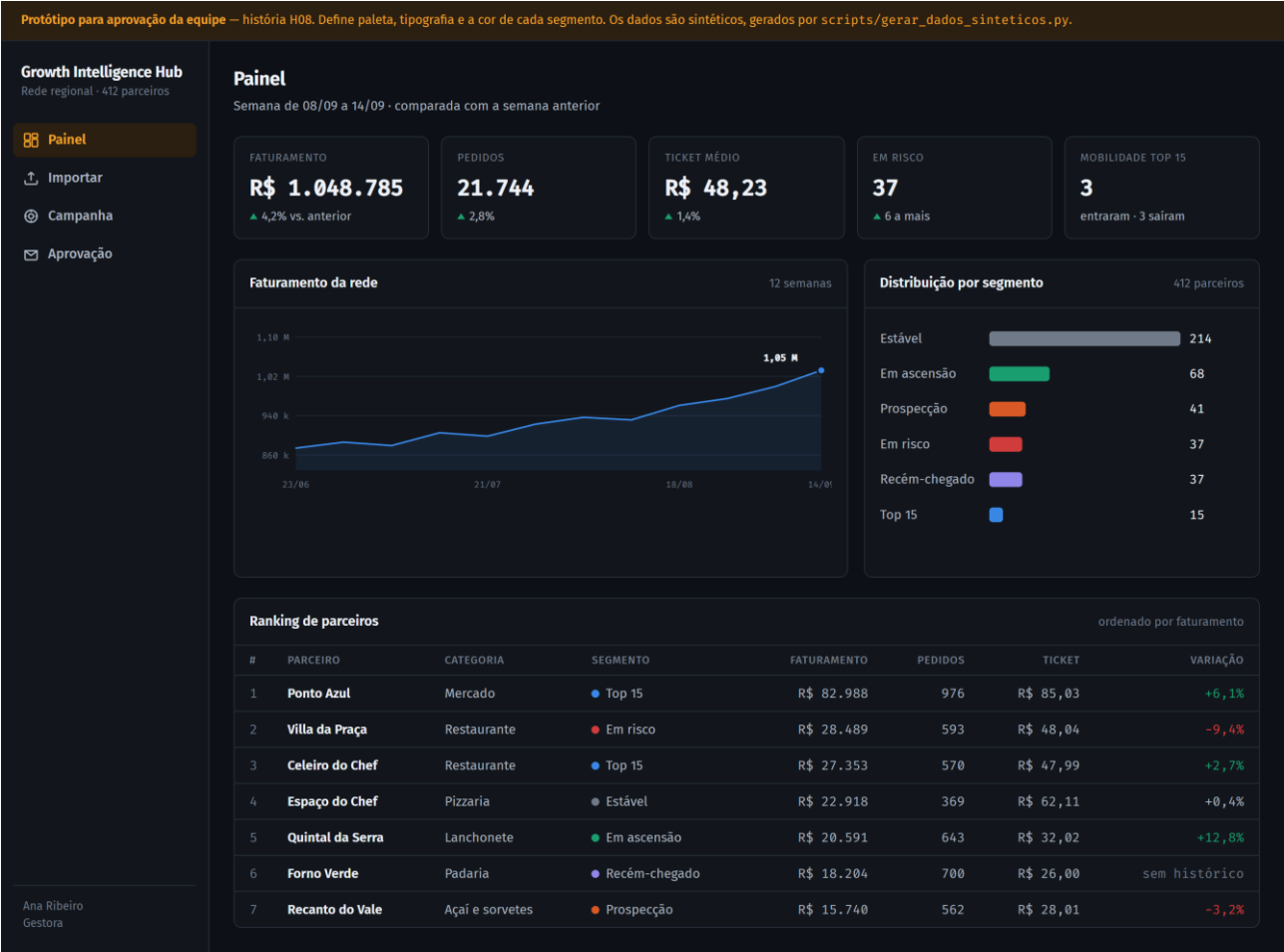
**Todos os dados exibidos são sintéticos**, gerados por script. Nenhum nome de empresa, pessoa ou operação real aparece no protótipo ou em qualquer parte do repositório, que é público.

### 5.1 Decisões visuais que valem explicar

- Cada cor tem um trabalho só. O âmbar significa "responde ao seu clique" — botão, link, foco — e nada mais. Quando uma cor é marca, ação e dado ao mesmo tempo, ela deixa de significar qualquer coisa.
- Cada segmento tem uma cor e só uma, idêntica em tabela, gráfico e indicador.
- A paleta foi validada com ferramenta, não escolhida no olho: o primeiro conjunto reprovou porque vermelho e verde ficavam indistinguíveis para daltônicos, e o verde foi trocado por turquesa.
- Nenhuma informação depende só da cor: o rótulo do segmento aparece sempre em texto ao lado do ponto colorido.

## 5.2 Painel

Responde à pergunta dominante do produto: quem precisa de atenção agora. Indicadores no topo, série histórica e distribuição por segmento, e o ranking de parceiros abaixo.



### 5.3 Importação

O período é campo obrigatório e não tem valor padrão. A prévia mostra o que será gravado — reconhecidos, rejeitados com o motivo de cada um, e parceiros ainda não cadastrados — antes de qualquer gravação.

Protótipo para aprovação da equipe — história H08. Define paleta, tipografia e a cor de cada segmento. Os dados são sintéticos, gerados por scripts/gerar\_dados\_sinteticos.py.

Growth Intelligence Hub

Rede regional · 412 parceiros

Painel

Importar

Campanha

Aprovação

Importar relatório

Cole o texto do relatório ou envie um arquivo CSV

Período

obrigatório

Data inicial \*

Data final \*

08 / 09 / 2026

14 / 09 / 2026

O relatório de origem não traz datas. Sem o período, as métricas ficam sem lugar na linha do tempo e a segmentação por tendência passa a classificar errado sem avisar.

Prévia

412 reconhecidos · 3 rejeitados

| PARCEIRO       | FATURAMENTO   | PEDIDOS | SITUAÇÃO             |
|----------------|---------------|---------|----------------------|
| Ponto Azul     | R\$ 82.988,29 | 976     | Reconhecido          |
| Villa da Praça | R\$ 28.489,35 | 593     | Reconhecido          |
| Forno Dourado  | R\$ 4.102,00  | 158     | Parceiro novo        |
| Parada Central | —             | —       | Faturamento ilegível |

Confirmar importação

Cancelar

Este período já foi importado

em 09/09 por Ana Ribeiro, com 409 registros. Substituir apaga os dados anteriores. O padrão é cancelar.

Ana Ribeiro

Gestora

Figura 12 — Tela de importação: período obrigatório e prévia com reconhecidos, rejeitados e parceiros novos.

Growth Intelligence Hub · Grupo 18 · Sprint 06 · 43

## 5.4 Campanha

Onde o núcleo computacional aparece para o usuário: restrições de orçamento e cota, escolha do modo de execução, comparativo de desempenho entre serial, CPU paralela e GPU, e o plano recomendado.

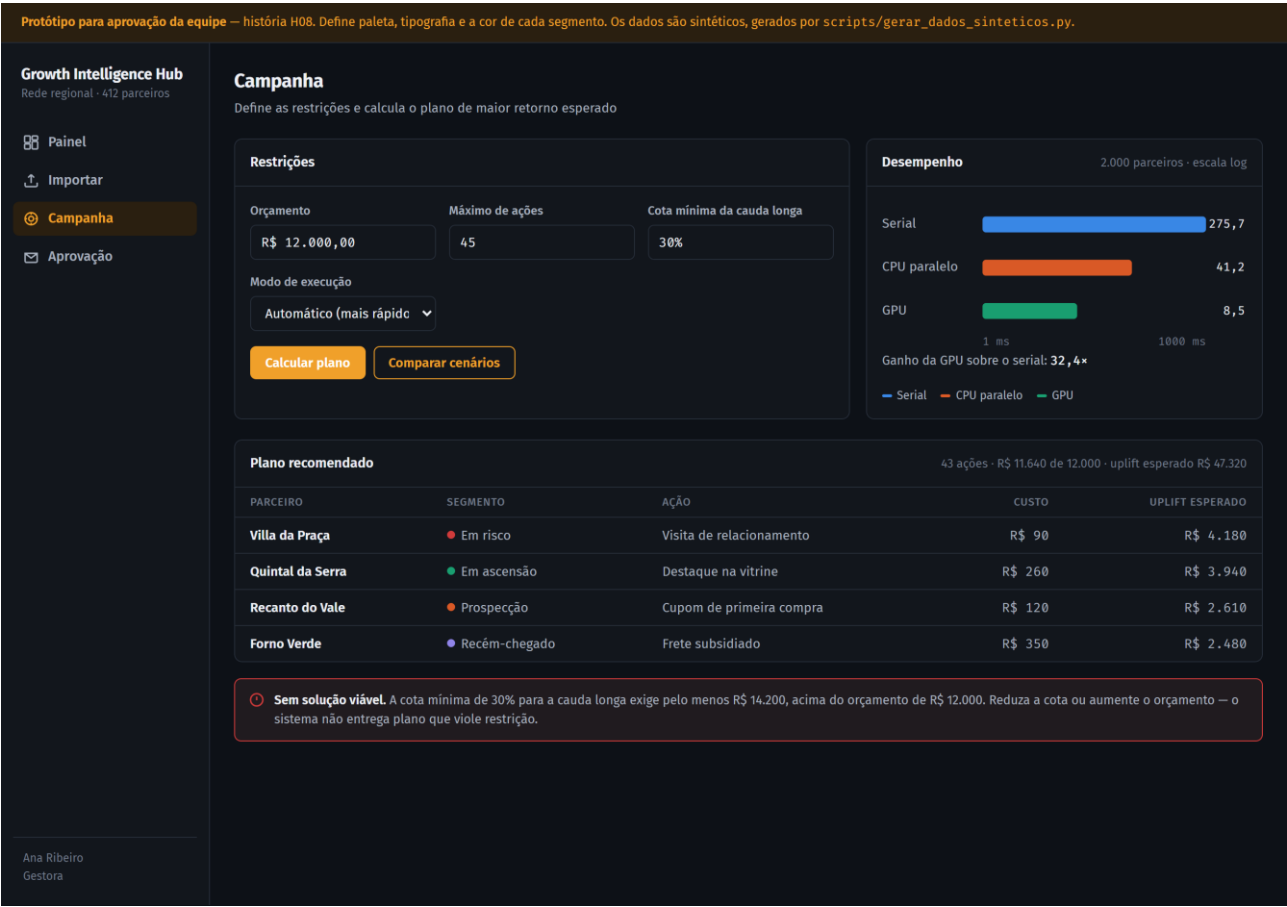


Figura 13 — Tela de campanha: restrições, comparativo de desempenho entre os três modos e plano recomendado.

## 5.5 Aprovação

Nenhuma mensagem chega ao parceiro sem decisão humana. A fila mostra cada mensagem com o destinatário, o segmento e o texto, e o gestor aprova, edita ou rejeita.

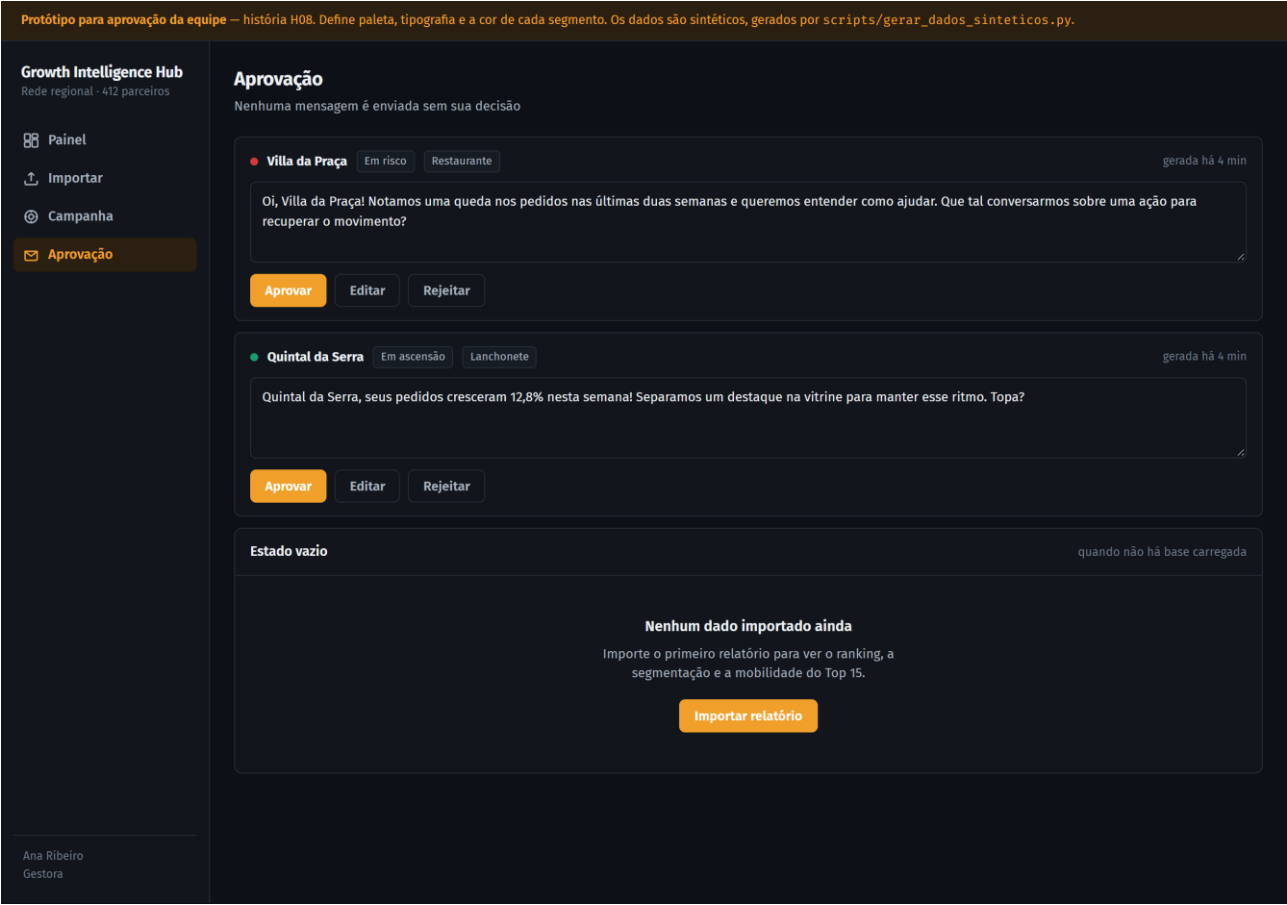


Figura 14 — Tela de aprovação: fila de mensagens pendentes, com a decisão humana como passo obrigatório.

## 6. Banco de dados criado

O esquema não está apenas desenhado: está aplicado e em uso. O ambiente completo sobe com um comando, e o processo de inicialização aplica as migrações antes de servir a API.

### 6.1 Estrutura aplicada

| Objeto                  | Quantidade |
|-------------------------|------------|
| Tabelas de domínio      | 16         |
| Chaves primárias        | 16         |
| Chaves estrangeiras     | 21         |
| Restrições de unicidade | 11         |
| Restrições de validação | 9          |
| Índices                 | 34         |
| Tipos enumerados        | 7          |

Consultado no catálogo do próprio PostgreSQL, e não estimado a partir do código:

```
$ docker compose exec api alembic current
77b3651bd03d (head)

$ docker compose exec postgres psql -U gih -d gih -c "\dt"
      acao_comercial      execucao_otimizador      item_plano      metrica      periodo
      alembic_version      historico_segmento      mensagem      parceiro      plano_campanha
      auditoria      importacao      previsao      categoria      sessao_acesso
      tentativa_login      usuario
(17 rows)
```

São as 16 tabelas de domínio mais a tabela de controle da ferramenta de migração, que registra qual versão do esquema está aplicada.

### 6.2 Esquema versionado, não script solto

O banco é criado por migrações versionadas no repositório. Isso permite que qualquer integrante chegue ao mesmo estado a partir de um clone limpo, e torna a evolução do banco auditável no histórico. O ciclo de reverter e reaplicar foi testado duas vezes, e não uma: a falha conhecida de tipos enumerados só aparece na segunda execução.

A suíte de testes também aplica as mesmas migrações num banco separado, em vez de montar o esquema por outro caminho. Custa alguns segundos por execução e cobre a divergência entre o modelo em código e a migração — defeito que ninguém percebe até a hora de implantar.

## 7. Projeto estruturado no GitHub

Repositório: [github.com/PedroMiranda243/gih-fabrica-de-software](https://github.com/PedroMiranda243/gih-fabrica-de-software)

### 7.1 Organização

| Diretório             | Conteúdo                                                                      |
|-----------------------|-------------------------------------------------------------------------------|
| <code>api/</code>     | API em Python e FastAPI, migrações do banco e suíte de testes                 |
| <code>nucleo/</code>  | Núcleo computacional em C++ e CUDA, com a investigação técnica já concluída   |
| <code>docs/</code>    | Toda a documentação técnica, diagramas, protótipo e o gerador deste documento |
| <code>scripts/</code> | Gerador de dados sintéticos e comando de reinicialização do banco             |
| <code>.github/</code> | Integração contínua e mapa de responsáveis por revisão                        |
| <code>web/</code>     | Interface: login, painel, importação e parceiros (construída na Sprint 03)    |

### 7.2 Evidências de evolução

A disciplina pede que o repositório demonstre a evolução do projeto. O que está lá para ser verificado:

- Histórias de usuário registradas como issues, agrupadas por sprint em milestones, com critérios de aceite.
- Toda alteração entra por Pull Request revisado. A branch principal é protegida e não aceita push direto.
- Integração contínua obrigatória: análise estática, suíte de testes contra um PostgreSQL real, e verificação de que nenhum dado ou segredo foi versionado.
- Mapa de responsáveis por diretório, que convida automaticamente o dono de cada área para revisar.
- Documentação versionada junto com o código, atualizada no mesmo Pull Request que introduz a mudança.

### 7.3 Estado atual do desenvolvimento

| Entrega                            | Situação       | Evidência                                                            |
|------------------------------------|----------------|----------------------------------------------------------------------|
| Banco modelado e migrado           | Concluído      | 16 tabelas, 21 chaves estrangeiras, 9 restrições                     |
| Ambiente sobe com um comando       | Concluído      | Docker Compose com migração automática                               |
| Autenticação, perfis e auditoria   | Concluído      | Testes cobrindo cada endpoint contra cada perfil                     |
| Importação de relatório com prévia | Concluído      | Interpretador tolerante, com testes próprios                         |
| Núcleo em GPU                      | Risco retirado | Ganho medido de até 100x sobre o baseline serial                     |
| Protótipo das telas                | Aprovado       | Aprovado pela equipe em 17/09/2026 — quatro telas, paleta validada   |
| Interface em React                 | Concluído      | Construída na Sprint 03 sobre o protótipo aprovado — ver a Parte III |

*Projeto acadêmico. © 2026 os autores — todos os direitos reservados. Repositório público significa visível para avaliação e portfólio; não constitui licença de uso, cópia ou redistribuição.*

## Parte III — Sprint 03: Estrutura Inicial Funcionando

Esta parte demonstra o sistema em funcionamento. Os seis itens da entrega estão implementados, persistem no banco e são exercitados por teste automatizado — e cada um aparece aqui com saída de execução real, não com descrição.

O sistema já tem interface. Depois que a equipe aprovou o protótipo das telas, a aplicação web foi construída sobre o sistema visual validado: login, painel, importação de relatório e gestão de parceiros, todas consumindo a API real sobre o banco real. As capturas de tela desta parte são da aplicação no ar, e não do protótipo — e cada uma vem acompanhada da execução que a sustenta.

Uma lacuna está declarada em vez de omitida: o cadastro de usuários funciona pela API e por um comando de terminal, mas ainda não tem tela. A seção 4 diz isso com essas palavras.

### Os seis itens, e onde estão

| # | Item                     | Situação    | Evidência neste documento                                       |
|---|--------------------------|-------------|-----------------------------------------------------------------|
| 1 | Banco de dados conectado | Funcionando | Seção 2 — verificação, catálogo do banco e importação pela tela |
| 2 | Login funcional          | Funcionando | Seção 3 — tela de login e transcrição das requisições           |
| 3 | Cadastro de usuários     | Funcionando | Seção 4 — transcrição, persistência e comando de terminal       |
| 4 | Controle de perfis       | Funcionando | Seção 5 — negação por perfil                                    |
| 5 | CRUD principal           | Funcionando | Seção 6 — as quatro operações e a tela de parceiros             |
| 6 | Deploy local             | Funcionando | Seção 7 — um comando sobe banco, API e interface                |

## 1. A estrutura implementada

O sistema está organizado em camadas com responsabilidades separadas, e a separação é o que permite medir o núcleo computacional isoladamente e garantir que a regra de negócio não escape para a interface.

### 1.1 O que existe hoje

| Camada      | O que está implementado                                                                                          | Como se verifica                              |
|-------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Interface   | Login, painel, importação de relatório e gestão de parceiros, sobre o sistema visual aprovado                    | Testes da interface e as capturas desta parte |
| Banco       | 16 tabelas, 21 chaves estrangeiras, 9 restrições de validação, 35 índices                                        | Migração versionada, aplicada na subida       |
| Acesso      | Sessão com estado no servidor, senha em Argon2id, bloqueio por força bruta                                       | 48 verificações contra a API no ar            |
| Usuários    | Cadastro, edição, desativação e atribuição de perfil, pela API e por comando de terminal                         | Testes automatizados e transcrição            |
| Autorização | Quatro perfis, verificados no servidor a cada requisição                                                         | Cada endpoint contra cada perfil              |
| Parceiros   | CRUD completo, com busca sem diferenciar acento, filtros e exclusão condicional                                  | Transcrição das quatro operações e a tela     |
| Ingestão    | Importação por texto ou arquivo CSV, com prévia, período obrigatório, rejeição parcial e substituição de período | Verificação de ponta a ponta e a tela         |
| Painel      | Indicadores do período, ranking com variação de posição e série histórica                                        | Testes automatizados e a tela                 |
| Auditoria   | Trilha de ações sensíveis, com filtro e paginação                                                                | Consulta na própria API                       |
| Núcleo      | Kernel de avaliação validado em CUDA e OpenMP                                                                    | Medição registrada no repositório             |

### 1.2 Números do que foi construído

Medidos nesta entrega, contra o ambiente no ar — nenhum é estimado:

| Medida                              | Valor |
|-------------------------------------|-------|
| Operações da API em funcionamento   | 26    |
| Testes automatizados da API         | 348   |
| Testes automatizados da interface   | 15    |
| Cobertura de linhas da API          | 97%   |
| Verificações de ponta a ponta       | 48    |
| Tabelas no banco                    | 16    |
| Decisões de arquitetura registradas | 9     |

### 1.3 Como a qualidade é verificada

- Toda alteração entra por Pull Request revisado; a branch principal é protegida.
- A integração contínua roda análise estática e a suíte completa contra um PostgreSQL real a cada Pull Request.
- A suíte de testes aplica as mesmas migrações do ambiente real, em um banco separado — e não monta o esquema por outro caminho, para cobrir a divergência entre o modelo em código e a migração.
- Um teste percorre cada endpoint contra cada perfil. Endpoint novo sem permissão declarada reprova a suíte, o que impede a cobertura de envelhecer em silêncio.
- O contraste das cores da interface é medido pintando cada cor e lendo o pixel, nos modos claro e escuro — e não suposto a partir da paleta.

### 1.4 A interface em funcionamento

O painel é a primeira tela depois do login. Resume o período mais recente, mostra a tendência da rede e ordena os parceiros por faturamento, com a posição anterior de cada um. Nenhum número desta tela é calculado no navegador: variação, ticket médio e posição vêm prontos da API, e a interface só formata e desenha.

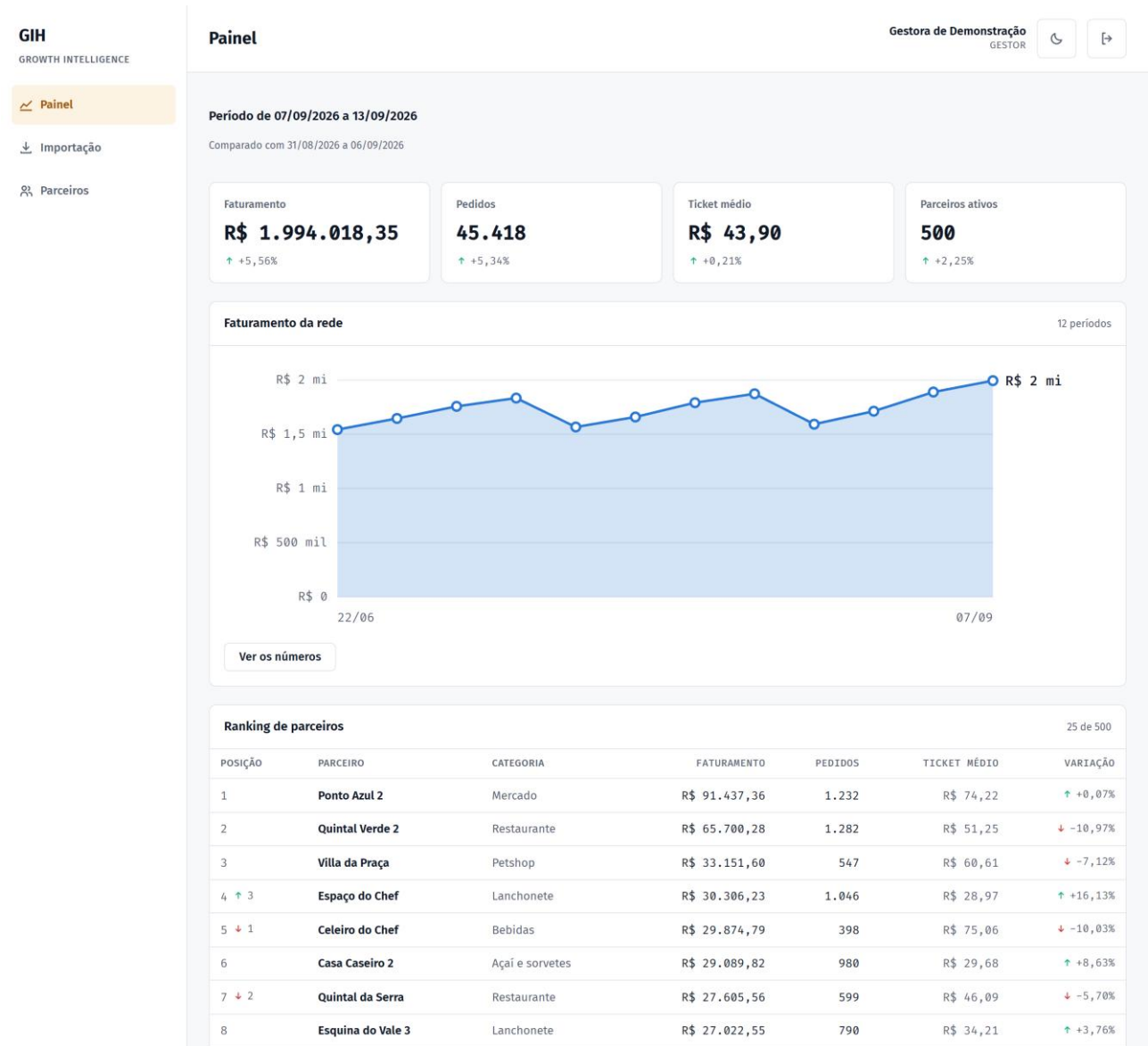


Figura 15 — Painel com os dados do banco: indicadores do período com a variação, série histórica e ranking.

## 2. Banco de dados conectado

A aplicação conversa com o PostgreSQL desde a primeira requisição. A conexão é verificada em três níveis: a própria API reporta o estado do banco, a migração aplicada é identificável, e a trilha de auditoria só tem conteúdo porque houve escrita real.

### 2.1 Verificação executada

A saída abaixo é de uma execução real do comando de verificação, que roda contra a API no ar — HTTP real, cookie real, banco real:

```
[1/6] Banco de dados conectado
      ok   a API responde à verificação de saúde
      ok   a API confirma o banco no ar  banco='ok'
      ok   o banco tem registros gravados pela aplicação  315 eventos na trilha
```

### 2.2 Estrutura aplicada

Consultado no catálogo do próprio PostgreSQL, e não estimado a partir do código:

```
$ docker compose exec api alembic current
a41f9c2be107 (head)

$ docker compose exec postgres psql -U gih -d gih -c "\dt"
      acao_comercial      execucao_otimizador      item_plano      metrica      periodo
      alembic_version      historico_segmento      mensagem      parceiro      plano_campanha
      auditoria      importacao      previsao      categoria      sessao_acesso
      tentativa_login      usuario
(17 rows)
```

São as 16 tabelas de domínio mais a tabela de controle da ferramenta de migração. O esquema é criado por migração versionada no repositório, o que permite a qualquer integrante chegar ao mesmo estado a partir de um clone limpo.

### 2.3 O dado entrando pela interface

A importação é por onde o banco é alimentado, e a tela segue o mesmo fluxo que a API exige: período obrigatório, prévia antes de gravar, e a rejeição de cada linha com o motivo. Na captura, a linha com faturamento "abacaxi" é recusada e dita — e as outras três seguem para gravação.

GIH  
GROWTH INTELLIGENCE

Painel

Importação

Parceiros

Importação de relatório

Gestora de Demonstração  
GESTOR

Relatório do período

Colar o texto ou enviar um arquivo CSV

Início do período

14 / 09 / 2026

Fim do período

20 / 09 / 2026

O relatório não traz datas. Sem o período, as métricas ficam órfãs na linha do tempo e a segmentação por tendência classifica errado sem emitir erro — por isso ele é obrigatório.

Colar o relatório

Parceiro;Faturamento;Pedidos  
Casa da Praça;4210,55;98  
Espaço Azul;7730,10;140  
Empório Bom Prato;abacaxi;38  
Quiosque do Porto;1890,00;52

...ou enviar um arquivo

Escolher arquivo

Nenhum arquivo escolhido

Ver a prévia

Prévia

3 reconhecidas · 1 rejeitadas

1 parceiro(s) novo(s) serão cadastrados: Quiosque do Porto

| LINHA             | CONTEÚDO                     | MOTIVO                               |
|-------------------|------------------------------|--------------------------------------|
| 4                 | Empório Bom Prato;abacaxi;38 | Faturamento não numérico: 'abacaxi'. |
| PARCEIRO          | FATURAMENTO                  | PEDIDOS                              |
| Casa da Praça     | R\$ 4.210,55                 | 98                                   |
| Espaço Azul       | R\$ 7.730,10                 | 140                                  |
| Quiosque do Porto | R\$ 1.890,00                 | 52                                   |

Gravar 3 registros

Figura 16 — Importação com a prévia aberta: três linhas reconhecidas, uma rejeitada com o motivo e um parceiro novo.

### 3. Login funcional

A autenticação é por login e senha, integrada ao banco. A sessão tem estado no servidor: encerrar a sessão a invalida de verdade, e não apenas apaga o cookie do navegador.

#### 3.1 Como funciona

- A senha é armazenada apenas como hash Argon2id, com sal por usuário. Nenhuma senha em texto claro no banco ou em log.
- O identificador da sessão são 256 bits sorteados, guardados no banco como hash — quem ler a tabela não consegue se passar por ninguém.
- O cookie vem com HttpOnly e SameSite, e o identificador é renovado a cada autenticação, o que impede fixação de sessão.
- Cinco tentativas falhas a partir da mesma origem bloqueiam temporariamente novas tentativas.
- Login inexistente e senha errada respondem de forma idêntica — mesma mensagem, mesmo código e mesmo tempo. Responder mais rápido para um login que não existe entregaria a lista de usuários válidos.

#### 3.2 Verificação executada

```
[2/6] Login funcional
ok  credencial válida abre sessão
ok  o cookie de sessão vem protegido  HttpOnly presente
ok  a resposta não devolve senha
ok  a sessão identifica o usuário
ok  encerrar invalida a sessão no servidor
ok  senha errada é recusada
ok  usuário inexistente responde igual a senha errada  indistinguível, como pede o RNF11
```

#### 3.3 A requisição e a resposta

Transcrição de uma autenticação real. A senha aparece mascarada de propósito:

Login – item 2 da entrega

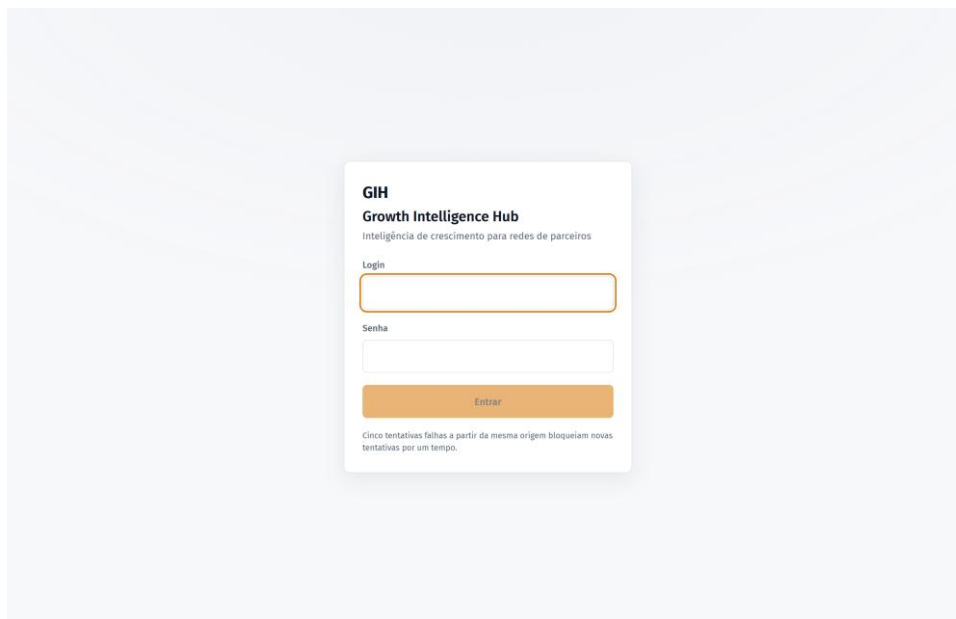
```
=====

$ POST /api/sessao
> {
>   "login": "t94926.analista",
>   "senha": "*****"
> }
201 Created
< {
<   "usuario": {
<     "id": 28,
<     "login": "t94926.analista",
<     "nome": "Analista da Evidência",
<     "perfil": "ANALISTA",
<     "ativo": true,
<     "parceiro_id": null,
<     "criado_em": "2026-09-19T01:08:26.642762Z"
<   },
<   "expira_em": "2026-09-19T09:08:29.040466Z"
< }

=====
```

### 3.4 A tela de login

A tela não distingue login inexistente de senha errada — a API responde igual aos dois, e a interface mostra a mensagem que recebeu em vez de tentar ser prestativa. O contorno em destaque é o foco do teclado, visível em todo controle.



**Figura 17** — Tela de login da aplicação, com o foco no primeiro campo.

## 4. Cadastro de usuários

O cadastro é do Administrador, persiste no banco e registra a criação na trilha de auditoria. Cada usuário recebe exatamente um perfil, entre Administrador, Gestor, Analista e Parceiro.

### 4.1 Verificação executada

```
[3/6] Cadastro de usuários
ok   cria usuário com perfil ADMINISTRADOR e2e25564.administrador
ok   cria usuário com perfil GESTOR e2e25564.gestor
ok   cria usuário com perfil ANALISTA e2e25564.analista
ok   os usuários criados estão persistidos
ok   a listagem não expõe hash de senha
```

### 4.2 A requisição e a resposta

Preparação — um analista, que é quem gerencia parceiros (UC04)

```
=====
$ POST /api/usuarios
> {
>   "login": "t94926.analista",
>   "nome": "Analista da Evidência",
>   "senha": "*****",
>   "perfil": "ANALISTA"
> }
201 Created
< {
<   "id": 28,
<   "login": "t94926.analista",
<   "nome": "Analista da Evidência",
<   "perfil": "ANALISTA",
<   "ativo": true,
<   "parceiro_id": null,
<   "criado_em": "2026-09-19T01:08:26.642762Z"
< }
=====
```

**A resposta não devolve a senha nem o hash dela.** As respostas da API são declaradas campo a campo, e não serializam a entidade inteira — é o que impede um campo sensível de vaziar quando alguém acrescentar uma coluna nova.

### 4.3 Pelo terminal

O cadastro também existe como comando, e por necessidade: a senha do administrador inicial é sorteada e só aparece no log da primeira subida, e importação e parceiros nem são do Administrador. Sem um caminho reproduzível, quem clonasse o projeto pararia na tela de login.

```
$ docker compose exec api python -m app.cli criar-usuario \
  --login seu.login --nome "Seu Nome" --perfil GESTOR
Senha para seu.login:
Repita a senha:
Usuário criado: seu.login (GESTOR)
```

A senha é digitada no terminal, nunca passada como argumento — argumento fica no histórico do shell. O comando aplica as mesmas validações da API e registra a criação na trilha de auditoria. Há também um comando para redefinir a senha, que encerra as sessões abertas daquele usuário.

**O que ainda não existe:** a tela de administração de usuários. O cadastro funciona, persiste e é verificado — pela API e pelo terminal —, mas a interface ainda não o oferece. Está entre os próximos passos.

### 4.4 Não existe apagar usuário

A trilha de auditoria referencia o autor de cada ação, e remover a linha deixaria o histórico apontando para o nada. O que existe é desativar, que resolve o problema real — tirar o acesso — sem destruir o registro. Desativar também encerra as sessões abertas daquele usuário na hora.

Há ainda uma trava: a última conta de Administrador ativa não pode ser desativada nem rebaixada. Sem ela, uma edição distraída deixaria o sistema sem ninguém capaz de gerenciar usuários.

## 5. Controle de perfis

A autorização é verificada no servidor a cada requisição. Esconder um botão não é controle de acesso: mesmo que a interface não ofereça o caminho, o endereço continua acessível por outros meios, e é o servidor que precisa recusar.

### 5.1 A matriz

Os perfis e o que cada um alcança vêm da matriz de permissões definida na Sprint 01, e não de leitura do código:

| Área                | Administrador | Gestor | Analista | Parceiro |
|---------------------|---------------|--------|----------|----------|
| Autenticar          | sim           | sim    | sim      | sim      |
| Gerenciar usuários  | sim           | —      | —        | —        |
| Auditar ações       | sim           | —      | —        | —        |
| Gerenciar parceiros | —             | sim    | sim      | —        |
| Importar relatório  | —             | sim    | sim      | —        |
| Painel e ranking    | só leitura    | sim    | sim      | —        |

### 5.2 Verificação executada

Cada rota protegida foi exercitada com cada perfil, e o resultado confrontado com a matriz:

```
[4/6] Controle de perfis
  · 25 rotas conferidas contra 3 perfis, a partir da matriz de docs/03-casos-de-uso.md
ok sem sessão, todo endpoint protegido responde 401
ok perfil ADMINISTRADOR só acessa o que a matriz permite
ok perfil GESTOR só acessa o que a matriz permite
ok perfil ANALISTA só acessa o que a matriz permite
```

### 5.3 A negação, na prática

Um analista tentando a área de usuários, que é exclusiva do Administrador:

Controle de perfis – item 4 da entrega

=====

O mesmo analista tentando a área de usuários, que é do Administrador:

```
$ GET /api/usuarios
403 Forbidden
< {
<   "detail": "Seu perfil não permite esta operação."
< }
```

=====

## 6. CRUD principal — parceiros

O parceiro é a entidade central do produto: métrica, segmento, previsão, plano de campanha e mensagem pendem dele. As quatro operações estão implementadas, integradas ao banco, e são exercitadas por 41 testes automatizados além da verificação de ponta a ponta.

### 6.1 As operações

| Operação  | Endereço                   | Comportamento                                                                                        |
|-----------|----------------------------|------------------------------------------------------------------------------------------------------|
| Cadastrar | POST /api/parceiros        | Nome único, garantido pelo banco; categoria escolhida vira confirmada                                |
| Consultar | GET /api/parceiros         | Busca parcial por nome sem diferenciar maiúscula nem acento; filtro por categoria, status e situação |
| Consultar | GET /api/parceiros/{id}    | Um parceiro, com a categoria resolvida                                                               |
| Atualizar | PATCH /api/parceiros/{id}  | Nome, categoria, status, contato e situação                                                          |
| Excluir   | DELETE /api/parceiros/{id} | Exclui sem histórico; recusa explicando quando há                                                    |

### 6.2 Verificação executada

```
[5/6] CRUD principal — parceiros
ok  cadastrar  Comércio e2e25564
ok  consultar
ok  consultar pela busca por nome
ok  cadastrar categoria
ok  atualizar
ok  a categoria confirmada por pessoa é gravada como MANUAL  RN05
ok  nome repetido é recusado
ok  excluir
ok  o excluído some da consulta
ok  excluir parceiro com histórico é recusado  409
ok  a recusa diz quantos registros impedem e aponta a desativação
```

## 6.3 Cadastrar, consultar e atualizar

Transcrição das requisições reais, com o corpo enviado e o corpo recebido:

CADASTRAR – item 5 da entrega

```
=====
$ POST /api/categorias
> {
>   "nome": "Categoria t94926"
> }
201 Created
< {
<   "id": 20,
<   "nome": "Categoria t94926",
<   "ativa": true
< }

$ POST /api/parceiros
> {
>   "nome": "Comércio t94926",
>   "status": "PROSPECCAO",
>   "contato": "contato@exemplo.test"
> }
201 Created
< {
<   "id": 549,
<   "nome": "Comércio t94926",
<   "categoria": null,
<   "origem_categoria": null,
<   "status": "PROSPECCAO",
<   "contato": "contato@exemplo.test",
<   "ativo": true,
<   "criado_em": "2026-09-19T01:08:29.087865Z"
< }
=====
```

ATUALIZAR

```
=====
$ PATCH /api/parceiros/549
> {
>   "status": "ATIVO",
>   "categoria_id": 20
> }
200 OK
< {
<   "id": 549,
<   "nome": "Comércio t94926",
<   "categoria": {
<     "id": 20,
<     "nome": "Categoria t94926",
<     "ativa": true
<   },
<   "origem_categoria": "MANUAL",
<   "status": "ATIVO",
<   "contato": "contato@exemplo.test",
<   "ativo": true,
<   "criado_em": "2026-09-19T01:08:29.087865Z"
< }
=====
```

## 6.4 A exclusão, e por que ela tem duas saídas

A entrega pede a operação de excluir. Excluir um parceiro que já tem faturamento importado, porém, falsearia as séries dos períodos já fechados: o total da rede deixaria de bater com a soma das partes, e ninguém perceberia, porque a tela continuaria parecendo correta.

A solução não foi escolher um dos dois lados. Sem histórico, o parceiro é apagado de verdade. Com histórico, a exclusão é recusada dizendo quantos registros de cada tipo impedem, e apontando a desativação — que resolve o problema real, que é tirar o parceiro de circulação.

EXCLUIR — sem histórico, a exclusão acontece

```
=====
$ DELETE /api/parceiros/549
204 No Content
< (sem conteúdo)

$ GET /api/parceiros/549
404 Not Found
< {
<   "detail": "Parceiro não encontrado."
< }
```

=====

E o caminho que protege o histórico:

```
$ DELETE /api/parceiros/549
204 No Content
< (sem conteúdo)
```

```
$ GET /api/parceiros/549
404 Not Found
< {
  < "detail": "Parceiro não encontrado."
  < }
```

=====

EXCLUIR – com histórico, a exclusão é recusada com o motivo

=====

Primeiro uma importação, para o parceiro passar a ter faturamento:

```
$ POST /api/importacoes
> {
  > "periodo_inicio": "2042-10-18",
  > "periodo_fim": "2042-10-24",
  > "texto": "Parceiro;Faturamento;Pedidos\nComércio t94926 hist;12500,40;312\n"
  > }
201 Created
< {
  < "id": 43,
  < "periodo": {
    < "id": 36,
    < "data_inicio": "2042-10-18",
    < "data_fim": "2042-10-24"
    < },
  < "autor": {
    < "id": 28,
    < "nome": "Analista da Evidência"
    < },
  < "origem": "TEXT0",
  < "total_gravado": 1,
  < "total_rejeitado": 0,
  < "enviado_em": "2026-09-19T01:08:29.169613Z",
  < "metricas_vigentes": 0
  < }
```

```
$ DELETE /api/parceiros/551
409 Conflict
< {
  < "detail": {
    < "erro": "O parceiro 'Comércio t94926 hist' tem histórico e não pode ser excluído.",
    < "ajuda": "Excluir apagaría registros que outros períodos já contabilizam. Para tirá-lo
    de circulação sem perder o histórico, desative-o: PATCH neste mesmo endereço com
    {\\"ativo\\": false}.",
    < "vinculos": {
      < "metricas": 1,
      < "segmentos": 0,
      < "previsoes": 0,
      < "itens_de_plano": 0,
      < "mensagens": 0,
      < "usuarios": 0
    }
  }
  < }
```

Apagar um parceiro com faturamento importado falsearia as séries dos períodos já fechados. A recusa diz o que impede e aponta a desativação.

```
$ PATCH /api/parceiros/551
> {
  > "ativo": false
  > }
200 OK
< {
  < "id": 551,
  < "nome": "Comércio t94926 hist",
  < "categoria": null,
```

```
<  "origem_categoria": null,  
<  "status": "ATIVO",  
<  "contato": null,  
<  "ativo": false,  
<  "criado_em": "2026-09-19T01:08:29.169613Z"  
< }
```

=====

### 6.5 Na interface

A consulta pela tela, com a busca "praca" encontrando os parceiros com "Praça" no nome. A busca ignora maiúscula e acento, e usa índice no banco — com 10.000 parceiros, que é a carga prevista, uma função aplicada linha a linha viraria varredura completa. Os filtros ficam no endereço da página, então um recorte pode ser mandado para outra pessoa por link.

GIH

GROWTH INTELLIGENCE

Painel

Importação

Parceiros

Parceiros

Gestora de Demonstração

GESTOR

Buscar por nome

praca

Categoria

Todas as categorias

Status comercial

Todos os status

Situação

Todas as situações

Parceiros

31 encontrados

| PARCEIRO           | CATEGORIA   | STATUS | SITUAÇÃO | CONTATO | CADASTRADO EM |
|--------------------|-------------|--------|----------|---------|---------------|
| Cantina da Praça   | Restaurante | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Casa da Praça      | Lanchonete  | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Casa da Praça 2    | Bebidas     | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Casa da Praça 3    | Gás e água  | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Celeiro da Praça   | Padaria     | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Celeiro da Praça 2 | Restaurante | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Empório da Praça   | Lanchonete  | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Empório da Praça 2 | Pizzaria    | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Empório da Praça 3 | Lanchonete  | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Espaço da Praça    | Farmácia    | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Espaço da Praça 3  | Restaurante | ATIVO  | ● Ativo  | —       | 18/09/2026    |
| Esquina da Praça   | Lanchonete  | ATIVO  | ● Ativo  | —       | 18/09/2026    |

Figura 18 — Tela de parceiros com a busca "praca" encontrando "Praça" — as primeiras doze linhas do resultado.

## 6.6 Documentação interativa

A especificação da API é gerada automaticamente e permite executar qualquer operação pelo navegador. É o contrato que a interface consome: a tela foi escrita contra ela, e não contra a implementação.

|                                                                                      |                                 |                   |
|--------------------------------------------------------------------------------------|---------------------------------|-------------------|
| <b>Growth Intelligence Hub</b> <small>0.2.0</small> <small>OAS 3.1</small>           |                                 |                   |
| <small>/api/openapi.json</small>                                                     |                                 |                   |
| <small>API do painel de inteligência de crescimento para redes de parceiros.</small> |                                 |                   |
| <b>autenticação</b> ^                                                                |                                 |                   |
| POST                                                                                 | /api/sessao                     | Autenticar        |
| DELETE                                                                               | /api/sessao                     | Encerrar          |
| GET                                                                                  | /api/sessao/actual              | Quem Sou          |
| POST                                                                                 | /api/sessao/senha               | Trocar Senha      |
| <b>usuários</b> ^                                                                    |                                 |                   |
| GET                                                                                  | /api/usuarios                   | Listar            |
| POST                                                                                 | /api/usuarios                   | Criar             |
| GET                                                                                  | /api/usuarios/{usuario_id}      | Obter             |
| PATCH                                                                                | /api/usuarios/{usuario_id}      | Editar            |
| <b>parceiros</b> ^                                                                   |                                 |                   |
| GET                                                                                  | /api/parceiros                  | Listar            |
| POST                                                                                 | /api/parceiros                  | Criar             |
| GET                                                                                  | /api/parceiros/{parceiro_id}    | Obter             |
| PATCH                                                                                | /api/parceiros/{parceiro_id}    | Editar            |
| DELETE                                                                               | /api/parceiros/{parceiro_id}    | Excluir           |
| GET                                                                                  | /api/categorias                 | Listar            |
| POST                                                                                 | /api/categorias                 | Criar             |
| <b>importação</b> ^                                                                  |                                 |                   |
| POST                                                                                 | /api/importacoes/previa         | Previa            |
| POST                                                                                 | /api/importacoes                | Importar          |
| GET                                                                                  | /api/importacoes                | Historico         |
| POST                                                                                 | /api/importacoes/arquivo/previa | Previa De Arquivo |
| POST                                                                                 | /api/importacoes/arquivo        | Importar Arquivo  |
| <b>painel</b> ^                                                                      |                                 |                   |
| GET                                                                                  | /api/painel/indicadores         | Indicadores       |
| GET                                                                                  | /api/painel/ranking             | Ranking           |
| GET                                                                                  | /api/painel/series              | Series            |
| <b>auditoria</b> ^                                                                   |                                 |                   |
| GET                                                                                  | /api/auditoria                  | Consultar         |
| GET                                                                                  | /api/auditoria/acoes            | Acoes Possiveis   |
| <b>sistema</b> ^                                                                     |                                 |                   |
| GET                                                                                  | /api/health                     | Health            |

Figura 19 — Documentação interativa em /api/docs: as 26 operações, agrupadas por assunto.

GET

/api/parceiros

Listar

Lista com busca por nome e filtros (RF14, RF24).

**selectinload** na categoria não é detalhe: sem ele, montar a resposta faria uma consulta por linha para buscar o nome da categoria — o N+1 que funciona com 100 parceiros e morre com 10.000, que é a carga do RNF04.

Parameters

Try it out

| Name                                  | Description                                     |
|---------------------------------------|-------------------------------------------------|
| busca                                 | Trecho do nome. Ignora maiúsculas e acentuação. |
| string   (string   null)<br>(query)   | <input type="text" value="busca"/>              |
| categoria_id                          |                                                 |
| integer   (integer   null)<br>(query) | <input type="text" value="categoria_id"/>       |
| status                                | Available values : ATIVO, PROSPECCAO, INATIVO   |
| string   (string   null)<br>(query)   | <input type="text" value="--"/>                 |
| ativo                                 | Filtra por situação.                            |
| boolean   (boolean   null)<br>(query) | <input type="text" value="--"/>                 |
| sem_categoria                         | Só os pendentes de classificação.               |
| boolean   (boolean   null)<br>(query) | <input type="text" value="--"/>                 |
| gih_sessao                            |                                                 |
| string   (string   null)<br>(cookie)  | <input type="text" value="gih_sessao"/>         |

Responses

| Code | Description                                                                                                                                                                                                                                                                                                                                                                                                               | Links    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 200  | Successful Response                                                                                                                                                                                                                                                                                                                                                                                                       | No links |
|      | <div>Media type<br/><input type="text" value="application/json"/></div> <div>Controls Accept header.</div> <div>Example Value   Schema</div> <div><pre>{   "id": 0,   "nome": "string",   "categoria": {     "id": 0,     "nome": "string",     "ativa": true   },   "origem_categoria": "INFERIDA",   "status": "ATIVO",   "contato": "string",   "ativo": true,   "criado_em": "2026-09-19T01:10:44.985Z" }</pre></div> |          |
| 422  | Validation Error                                                                                                                                                                                                                                                                                                                                                                                                          | No links |
|      | <div>Media type<br/><input type="text" value="application/json"/></div> <div>Example Value   Schema</div> <div><pre>{   "detail": [     {       "loc": [         "string",         0       ],       "msg": "string",       "type": "string"     }   ] }</pre></div>                                                                                                                                                       |          |

Figura 20 — Contrato de entrada e saída da listagem de parceiros, com os filtros disponíveis.

## 7. Execução local

O ambiente completo — banco, API e interface — sobe com um comando, a partir de um clone limpo. Não há etapa manual de configuração: o processo de inicialização aplica as migrações e cria o administrador antes de servir a API.

### 7.1 Pré-requisitos

- Docker Desktop — é o único obrigatório
- Python 3.11 e Node.js 22, apenas para rodar a verificação e os testes fora do contêiner
- Placa NVIDIA e CUDA Toolkit são opcionais, e só para o núcleo computacional

### 7.2 Passo a passo

```
git clone https://github.com/PedroMiranda243/gih-fabrica-de-software.git
cd gih-fabrica-de-software
cp .env.example .env
docker compose up
```

O compose sobe o PostgreSQL, espera ele ficar saudável, aplica as migrações, cria o administrador inicial se não houver nenhum, serve a API e sobe a interface. A aplicação fica em <http://localhost:5173>.

A interface e a API dividem o mesmo endereço: o servidor da interface repassa o que é de /api. É o que deixa o cookie de sessão funcionar sem abrir a API a outras origens — a postura de segurança do servidor não precisou mudar para acomodar a tela.

### 7.3 Primeiro acesso

Crie o próprio usuário pelo terminal, com o ambiente no ar. A senha é digitada, duas vezes:

```
docker compose exec api python -m app.cli criar-usuario \
  --login seu.login --nome "Seu Nome"
```

O perfil padrão é Gestor, que é o que usa painel, importação e parceiros. Depois, entre em <http://localhost:5173>.

**Não existe senha padrão.** O repositório é público, e um usuário e senha fixos no código seriam porta aberta em qualquer instalação que esquecesse de trocá-los. O administrador inicial recebe uma senha sorteada, impressa uma única vez no log da primeira subida; perda, ela é redefinida pelo comando `redefinir-senha`.

### 7.4 Verificando que subiu

| O quê                        | Onde                                                                            |
|------------------------------|---------------------------------------------------------------------------------|
| A aplicação                  | <a href="http://localhost:5173">http://localhost:5173</a>                       |
| Estado do sistema e do banco | <a href="http://localhost:8000/api/health">http://localhost:8000/api/health</a> |
| Documentação interativa      | <a href="http://localhost:8000/api/docs">http://localhost:8000/api/docs</a>     |
| Verificação de ponta a ponta | <code>cd api &amp;&amp; GIH_ADMIN_SENHA=... python e2e/verificacao.py</code>    |
| Suíte de testes              | <code>cd api &amp;&amp; pytest</code>                                           |

A verificação sai com código diferente de zero se qualquer checagem falhar — senão não é verificação, é impressão.

## 8. Repositório

Repositório: [github.com/PedroMiranda243/gih-fabrica-de-software](https://github.com/PedroMiranda243/gih-fabrica-de-software)

### 8.1 O que demonstra a evolução

- Histórias de usuário registradas como issues, agrupadas por sprint em milestones, com critérios de aceite verificáveis.
- Toda alteração entra por Pull Request revisado, com o dono de cada diretório convidado automaticamente.
- Integração contínua obrigatória: análise estática, suíte de testes contra um PostgreSQL real, e verificação de que nenhum dado ou segredo foi versionado.
- Documentação versionada junto com o código, atualizada no mesmo Pull Request que introduz a mudança.
- Este documento é gerado a partir da documentação do repositório, e não escrito à parte.

### 8.2 Divisão de responsabilidades

Cada diretório tem um dono. Isso não impede ninguém de contribuir — impede cinco pessoas de editarem o mesmo arquivo na mesma semana.

| Diretório       | Responsável          | Conteúdo                                         |
|-----------------|----------------------|--------------------------------------------------|
| <b>api/</b>     | Dev Backend / Núcleo | API, migrações e testes                          |
| <b>nucleo/</b>  | Dev Backend / Núcleo | Otimizador em C++ e CUDA                         |
| <b>web/</b>     | Dev Frontend         | Interface: login, painel, importação e parceiros |
| <b>docs/</b>    | Product Owner        | Documentação e este gerador                      |
| <b>scripts/</b> | Banco de Dados       | Gerador sintético e reinicialização              |
| <b>.github/</b> | Scrum Master         | Integração contínua                              |

## 9. Dificuldades encontradas

As dificuldades abaixo são reais, com a causa e a correção. Nenhuma delas é de prazo ou de coordenação: são problemas técnicos que custaram tempo e que ficaram registrados no repositório para não voltarem.

### 9.1 A GPU perde para a CPU paralela em escala pequena

A investigação técnica do núcleo mediu o ganho real e encontrou o contrário do esperado: abaixo de cerca de 4.000 planos candidatos, dezesseis threads de CPU batem a GPU inteira. Acima disso, a GPU com transferência a cada geração ganha apenas 1,1 vez do OpenMP — mas o kernel sozinho ganha 11 vezes. A transferência consome 88% do tempo.

**Correção:** manter a população na memória da GPU entre as gerações deixou de ser otimização e virou requisito de projeto do componente. Sem isso, o caminho CUDA não se justificaria.

### 9.2 O benchmark media o relógio, não o trabalho

Na validação do compilador, o ganho do OpenMP em 1.024 planos saiu 7,5 vezes numa execução e 13,9 vezes na seguinte — mesmo cenário, mesmo programa. Cinco repetições de uma operação que dura menos de um milissegundo medem a precisão do relógio, não o desempenho.

**Correção:** o teste passou a calibrar quantas repetições cabem num alvo de tempo, a aquecer antes de medir, e a reportar mediana de várias execuções. Um número que não se reproduz não serve de resultado.

### 9.3 O contêiner morria apontando para um arquivo que existe

O script de inicialização foi gravado com fim de linha do Windows, e o Docker respondeu que o arquivo não existe — apontando exatamente para o arquivo que estava lá. O comparador de versões não mostrava diferença nenhuma, porque ele normaliza a leitura enquanto o Docker copia o arquivo do disco.

**Correção:** o fim de linha passou a ser fixado no repositório por configuração, nos arquivos que o Linux precisa ler.

### 9.4 Os diagramas saíram vazios no PDF

Na entrega anterior, os diagramas foram embutidos em formato vetorial. O documento abria normal na tela, mas ao exportar para PDF o Word desenhava as caixas e as setas e descartava todo o texto dentro delas. Um diagrama de classes sem os nomes das classes não é um diagrama ruim — é uma figura vazia, e teria sido entregue assim.

**Correção:** os diagramas passaram a entrar rasterizados em resolução de impressão, e a conferência da entrega passou a incluir abrir o PDF e olhar as páginas — e não apenas conferir que o arquivo foi gerado.

### 9.5 A própria verificação reprovava por defeito dela

A verificação de ponta a ponta falhou duas vezes por causa de si mesma antes de passar. A varredura de autorização percorria o endereço que encerra a sessão, e tudo depois dele respondia como não autenticado — o sintoma parecia falha de permissão. E o período usado na importação era fixo, colidindo com a execução anterior.

**Correção:** a varredura passou a pular as operações destrutivas, e a marca de cada execução passou a ser sorteada em vez de derivada do relógio. Os dois motivos ficaram comentados no código.

### 9.6 A primeira versão da interface reprovou no contraste

O contraste foi medido, e não suposto: cada cor pintada e o pixel lido, nos dois modos. A primeira versão reprovou em três pontos. O pior foi o texto branco sobre o botão âmbar, a 2,15:1 no modo escuro — o mínimo exigido é 4,5:1. A mesma medição achou o número da variação pintado com a cor de um segmento, a 2,82:1, o que também violava a regra do sistema visual: cor de segmento só como ponto ou barra, nunca como texto.

**Correção:** texto escuro fixo sobre o âmbar (8,27:1 no escuro), texto neutro na variação com a cor só na seta, e a tinta mais clara restrita ao que não é texto. Relendo com a regra na cabeça, a cor de ação também saiu da sigla e do ícone, onde fazia papel de marca — o erro exato do projeto anterior.

### 9.7 A integração contínua rodava num Node sem suporte

A verificação da interface reprovou na primeira execução com um erro que não dizia nada sobre a causa: uma função ausente dentro de uma biblioteca. A integração contínua rodava o Node 20, e a cadeia de teste exige o 22 — na máquina de desenvolvimento, com o 24, tudo passava. O Node 20 saiu de suporte em abril de 2026.

**Correção:** a integração contínua passou para o Node 22, o mesmo da imagem em que a interface roda, em vez de rebaixar as bibliotecas para caber num ambiente sem correção de segurança. A mudança foi conferida num contêiner com o Node 22 antes de publicada.

## 9.8 A verificação sujava o banco em que se demonstra

Para não colidir com execuções anteriores, a verificação de ponta a ponta importava relatórios em períodos no futuro — e nunca os removía. Depois de algumas execuções, o painel, que abre no período mais recente, passou a abrir em 2055, com um parceiro só.

**Correção:** a verificação passou a apagar o que criou ao terminar, e a conferir isso como uma das próprias checagens: o painel precisa voltar a abrir no mesmo período, com os mesmos totais.

## 10. Ajustes no planejamento e na modelagem

A entrega pede que alterações feitas nas sprints anteriores sejam registradas e justificadas. Foram sete, e todas estão versionadas no repositório com a justificativa junto.

| O que mudou                           | De / para                                                     | Por quê                                                                                                                                                                                                                                                  |
|---------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cadência das sprints</b>           | Quinzenal para semanal                                        | Pedido na avaliação da Sprint 01. O atraso passa a aparecer na primeira semana, não na terceira. Duas histórias grandes demais para uma semana foram quebradas; o total de pontos não mudou                                                              |
| <b>Número de tabelas</b>              | 14 para 16                                                    | A implementação da autenticação exigiu guardar a sessão e as tentativas de login. A sessão precisa de estado no servidor para que encerrar tenha efeito imediato                                                                                         |
| <b>Compilação do núcleo</b>           | Em tempo de execução para antecipada                          | O compilador C++ e o toolkit foram instalados, e a implementação em OpenMP exige compilação antecipada de qualquer forma. A decisão anterior ficou registrada como histórico                                                                             |
| <b>Formato do relatório importado</b> | Indefinido para tolerante                                     | A história pedia "o formato definido", e não havia formato definido em lugar nenhum. Foi decidido em equipe: separador livre, ordem de colunas livre, sinônimos aceitos                                                                                  |
| <b>Exclusão de parceiro</b>           | Apenas desativar para exclusão condicional                    | Esta entrega pede a operação de excluir. A exclusão acontece quando não há histórico, e é recusada com explicação quando há                                                                                                                              |
| <b>Aplicação web</b>                  | Sem história no backlog para uma história própria, antecipada | O backlog pressupunha telas em várias histórias, mas nenhuma mandava construir a aplicação — por isso o servidor avançou e a interface não. A lacuna virou história própria, e a interface foi antecipada da Sprint 6 assim que o protótipo foi aprovado |
| <b>Acesso ao sistema</b>              | Senha sorteada no log para comando de terminal                | Com a senha do administrador só no log da primeira subida, quem clonasse o projeto não conseguia entrar. Os comandos de criar usuário e redefinir senha tornaram o primeiro acesso reproduzível                                                          |

Um ponto que estava em aberto na versão anterior deste documento foi fechado: o critério de aceite da história do ambiente dizia que o comando único sobe também a interface, e essa metade não estava cumprida. Agora está — um comando sobe banco, API e interface.

E um ponto segue em aberto, declarado em vez de omitido: a tela de administração de usuários. O cadastro existe e funciona pela API e pelo terminal, mas a interface ainda não o oferece.

## 11. Próximos passos

A ordem abaixo segue o backlog priorizado da Sprint 01, com o ajuste de cadência já aplicado.

| Sprint | Tema                       | O que entra                                                                                |
|--------|----------------------------|--------------------------------------------------------------------------------------------|
| 6      | Ingestão completa e painel | Quase concluída: falta medir o painel com 5.000 parceiros, que é o requisito de desempenho |
| 7      | Segmentação e mobilidade   | Classificação determinística com precedência explícita, ranking e mobilidade do Top N      |
| 8      | Modelo preditivo           | Treino a partir do histórico, previsão de faturamento e probabilidade de queda             |
| 9 a 11 | Núcleo computacional       | Otimizador serial, paralelo em CPU e paralelo em GPU, com o comparativo de desempenho      |
| 12     | Central de comunicação     | Geração de mensagens por segmento, com aprovação humana obrigatória                        |
| 13     | Assistente e fechamento    | Perguntas em linguagem natural sobre dados já apurados, e a entrega final                  |

### 11.1 O que vem primeiro

- Medir o painel com 5.000 parceiros e registrar a mediana com a dispersão — é o que fecha a Sprint 6.
- A tela de administração de usuários, que é a lacuna declarada na seção 4.
- A segmentação determinística e a mobilidade do Top N, lida do ranking que já existe — a Sprint 7.
- A sugestão automática de categoria pelo nome do parceiro, que completa o cadastro.
- O recálculo da segmentação após cada importação, que hoje não acontece.

*Projeto acadêmico. © 2026 os autores — todos os direitos reservados. Repositório público significa visível para avaliação e portfólio; não constitui licença de uso, cópia ou redistribuição.*

## Parte IV — Sprint 04: Primeiro Módulo Completo

O módulo entregue é o de Ingestão e Inteligência de Negócio — os módulos M2 e M3 do escopo, que na prática são um fluxo só: o relatório do período entra, é validado, persiste, cada parceiro é segmentado, e o resultado aparece no painel e na lista de parceiros, onde pode ser recortado e exportado. A autenticação, entregue na Sprint 03, é o que protege esse fluxo; o núcleo de otimização e o modelo preditivo vêm nas próximas sprints.

Cada item desta parte vem com execução real. As transcrições foram geradas por scripts contra a aplicação no ar, e cada troca vem com o resultado que se esperava dela — uma evidência que só mostra respostas não diz se elas estão certas. As capturas são da aplicação no ar, sobre a base de demonstração, e cada erro mostrado foi provocado pela própria tela.

### As seis entregas, e onde estão

| # | Entrega                      | Situação    | Evidência neste documento                                                    |
|---|------------------------------|-------------|------------------------------------------------------------------------------|
| 1 | <b>Módulo funcional</b>      | Funcionando | Seções 1 e 2 — as funcionalidades, a verificação de ponta a ponta e as telas |
| 2 | <b>Persistência</b>          | Funcionando | Seção 3 — os dados depois de desligar e religar a aplicação                  |
| 3 | <b>Validações</b>            | Funcionando | Seção 4 — cada regra provocada, na API e na tela                             |
| 4 | <b>Mensagens de erro</b>     | Funcionando | Seção 5 — o contrato das mensagens e a falha inesperada                      |
| 5 | <b>Navegação entre telas</b> | Funcionando | Seção 6 — o mapa das telas e o percurso                                      |
| 6 | <b>Commits organizados</b>   | Funcionando | Seção 7 — o histórico, gerado do próprio repositório                         |

Quatro lacunas estão declaradas em vez de omitidas, e aparecem com essas palavras nas seções 1 e 11: os limiares da segmentação se configuram pela API e pelo terminal, mas ainda não têm tela; o histórico de importações existe na API e não na interface; a sugestão automática de categoria ainda não foi feita; e a administração de usuários, lacuna já declarada na Sprint 03, continua sem tela.

## 1. O módulo e suas funcionalidades

### 1.1 O fluxo

O gestor recebe da plataforma de delivery um relatório de faturamento por parceiro, em texto, sem data nenhuma. O módulo transforma esse texto em série histórica e em classificação — quem está no topo, quem está subindo, quem está caindo — e deixa o gestor recortar a rede por qualquer um desses critérios.

| Etapa     | O que o sistema faz                                                                                            | Requisito         | Tela           |
|-----------|----------------------------------------------------------------------------------------------------------------|-------------------|----------------|
| Importar  | Recebe o relatório colado ou em arquivo CSV, com o período obrigatório                                         | RF09, RF10        | Importação     |
| Validar   | Mostra a prévia antes de gravar: linhas reconhecidas, rejeitadas com o motivo, parceiros novos                 | RF11              | Importação     |
| Persistir | Grava na mesma transação as métricas e a segmentação — ou nada; período repetido só com substituição explícita | RF12, RF13        | Importação     |
| Segmentar | Classifica cada parceiro em exatamente um segmento, por regra determinística com precedência                   | RF20, RF21        | Painel e lista |
| Analisar  | Indicadores do período, série histórica, distribuição por segmento, ranking e mobilidade do Top N              | RF17 a RF19, RF22 | Painel         |
| Recortar  | Busca sem diferenciar acento, filtros, ordenação por qualquer medida e exportação do recorte                   | RF23 a RF25       | Parceiros      |
| Cadastrar | Cadastro, edição, desativação e exclusão de parceiro, com o desempenho e a série individual                    | RF14, RF19        | Cadastro       |

### 1.2 As regras de negócio que o módulo aplica

Todas estão escritas na especificação de requisitos, e todas são decididas no servidor. A interface exibe e coleta; nenhum cálculo de negócio acontece no navegador.

| Regra | O que diz                                                               | Por que importa                                                             |
|-------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| RN01  | A segmentação segue uma ordem de precedência fixa, e Em Risco vence Top | É o que faz o painel responder "quem está prestes a sair do topo?"          |
| RN02  | A mobilidade do Top N lê o ranking, nunca o segmento gravado            | Lida do segmento, anunciaria a saída de quem continua no topo em queda      |
| RN03  | O período é obrigatório na importação, sem valor padrão                 | Sem as datas, a segmentação por tendência classifica errado sem emitir erro |
| RN04  | O ticket médio é derivado na consulta, nunca armazenado                 | Guardado, divergiria das parcelas na primeira correção de dado              |
| RN05  | Categoria inferida vale como sugestão até alguém confirmar              | Parceiro importado entra sem categoria, em vez de com um palpite            |

### 1.3 O que está fora, e por quê

- Os limiares da segmentação (RF21) mudam sem alteração de código, pela API e por comando de terminal, só pelo Administrador. A tela deles é o próximo passo.
- O histórico de importações (RF13) é gravado e consultável pela API; a interface ainda não o lista.
- A sugestão de categoria pelo nome (RF15) é da Sprint 3 do backlog e está aberta na issue #35.

### 1.4 Números do módulo

Medidos na versão entregue — nenhum é estimado:

| Medida                            | Valor |
|-----------------------------------|-------|
| Testes automatizados da API       | 475   |
| Testes automatizados da interface | 64    |

| Medida                                                                      | Valor   |
|-----------------------------------------------------------------------------|---------|
| Cobertura do núcleo de regras — a integração contínua reprova abaixo de 70% | 99%     |
| Verificações de ponta a ponta contra a aplicação no ar                      | 68      |
| Operações da API                                                            | 31      |
| Tabelas no banco                                                            | 17      |
| Painel com 5.000 parceiros — a consulta mais lenta, mediana                 | 32,3 ms |
| Painel com 5.000 parceiros — a consulta mais lenta, p95                     | 36,4 ms |

O limite do requisito de desempenho é de 2 segundos com 5.000 parceiros. A medição usa um banco separado, repetições calibradas e reporta mediana e p95 — o relatório completo, com o plano de execução de cada consulta, está versionado em docs/medicoes/.

## 2. O módulo funcionando

### 2.1 Verificação de ponta a ponta

A verificação roda contra a aplicação no ar — HTTP real, cookie real, banco real — e sai com código diferente de zero se qualquer checagem falhar. O trecho abaixo é o do módulo; os seis primeiros blocos, da Sprint 03, continuam passando na mesma execução.

```
[ + ] Painel – indicadores, ranking e séries
    ok os indicadores consolidam o período importado 21440.40 em 513 pedidos
    ok o ticket médio é a razão dos totais, e não coluna guardada RN04
    ok o ranking ordena por faturamento, com posições sem buraco 2 parceiros
    ok a série da rede sai em ordem cronológica 15 períodos
    ok a série individual cobre todos os períodos, com lacuna explícita período sem medição
      vem nulo, não sumido

[ + ] Segmentação, distribuição e mobilidade
    ok a importação já deixou o período segmentado sem reprocessamento manual
    ok a distribuição soma os parceiros do período 2 classificados em 1 segmento
    ok a distribuição vem do maior para o menor sem desempate estável, o gráfico parece
      mudar sem nada ter mudado
    ok cada parceiro recebe exatamente um segmento RN01
    ok a mobilidade responde com o Top N configurado Top 15
    ok ninguém que está no Top N aparece como saída RN02 – a mobilidade lê o ranking, não o
      segmento

[ + ] Parceiros – recorte, ordenação e exportação
    ok a lista vem paginada, com o total do recorte 2 de 504
    ok a ordenação por faturamento respeita o sentido pedido 2 com movimento
    ok quem não teve métrica no período vai para o fim não é o melhor nem o pior
    ok ordenar por coluna desconhecida é recusado o valor entra num ORDER BY
    ok o arquivo exportado traz exatamente o recorte da tela 4 linhas
    ok o arquivo vem pronto para a planilha BOM de UTF-8, senão 'Praça' abre como 'PraÃça'

[ + ] Configuração da segmentação
    ok o administrador lê os limiares em vigor Top 15 · tendência 2
    ok o gestor não altera a régua da segmentação quem mexe aqui reescreve o que 'em risco'
      significa para a rede
    ok limiar sem sentido é recusado Top 0 não tem ninguém dentro
    ok alterar o limiar reclassifica o período mais recente configuração que não se reflete
      na tela engana quem a mudou
    ok a alteração fica na trilha, com o valor anterior sem dizer o que era antes, a trilha
      não responde nada
    ok o limiar volta ao que era antes da verificação configuração é global: resíduo aqui
      muda a próxima execução

[ + ] Limpeza – o banco volta ao que era antes da execução
    ok remove do banco o que a execução gravou 3 períodos, 4 importações, 5 métricas, 4
      parceiros, 1 categoria
    ok o painel volta a abrir no mesmo período, com os mesmos totais período de 2026-09-14
      · os usuários da execução ficam, desativados: a trilha de auditoria aponta para eles
```

As 68 verificações passaram.

**Resultado:** As 68 verificações passaram. A última checagem é a da limpeza: a verificação apaga o que gravou e confere que o painel voltou ao mesmo período, com os mesmos totais — ela roda no banco em que se demonstra.

## 2.2 O painel

Resumo antes do detalhe: os indicadores do período no topo, com a variação e a mobilidade do Top N; a série histórica e a distribuição por segmento no meio; o ranking por último. Variação, ticket, posição e segmento vêm prontos da API — a tela formata e desenha.

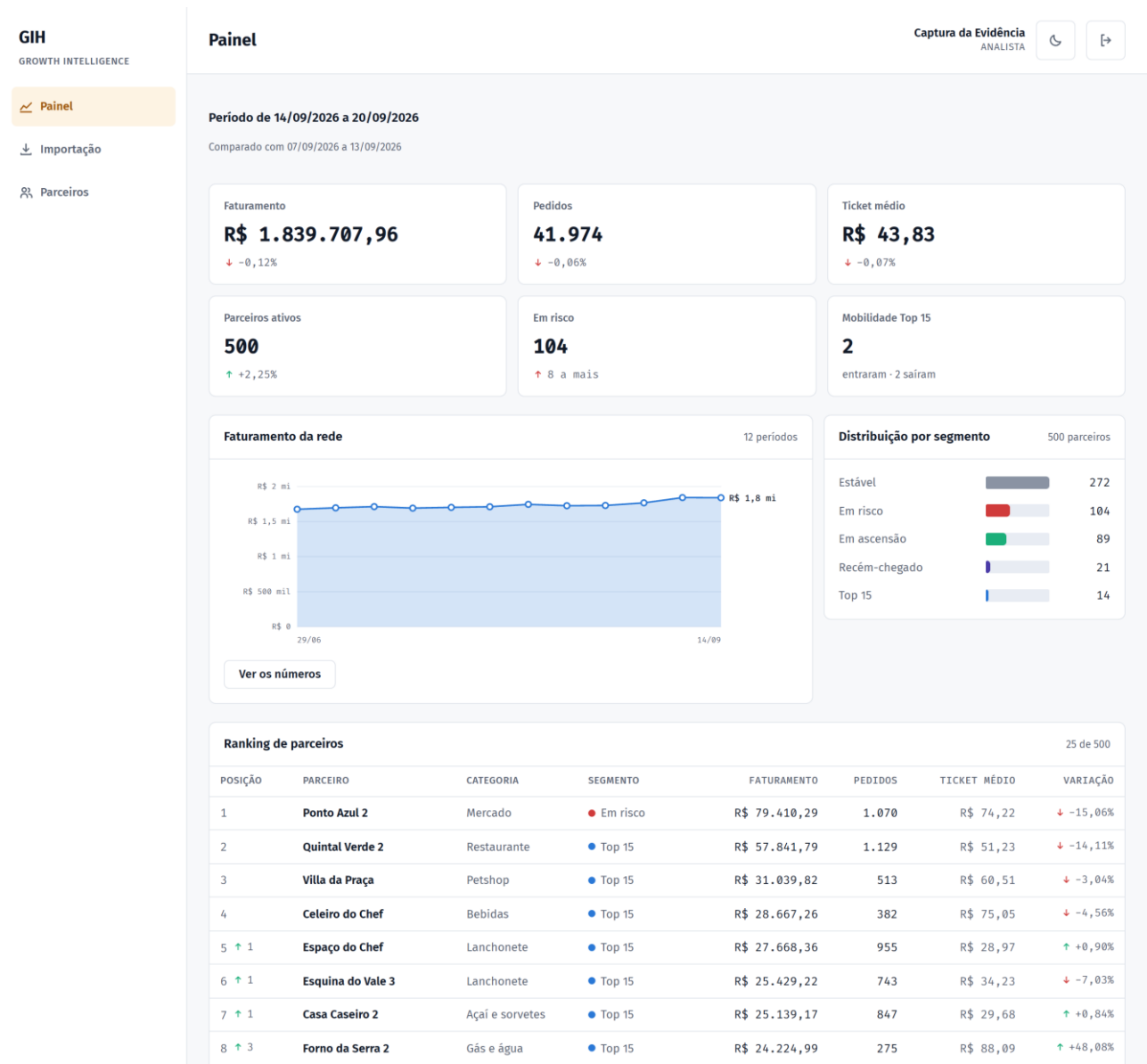


Figura 21 — Painel sobre a base de demonstração: indicadores, série, distribuição por segmento e ranking.

### 2.3 A lista recortada

O recorte vive no endereço da página. Esta captura é de /parceiros?segmento=EM\_RISCO&ordenar\_por=variacao: os parceiros em risco, dos que mais caíram para os que menos caíram. O botão de exportar baixa exatamente este recorte em CSV — é a mesma consulta, em outro formato.

GIH  
GROWTH INTELLIGENCE

Painel

Importação

Parceiros

Parceiros

Captura da Evidência  
ANALISTA

Buscar por nome  
Ignora maiúsculas e acentuação

Categoria  
Todas as categorias

Segmento  
Em risco

Status comercial  
Todos os status

Situação  
Todas as situações

Parceiros

104 encontrados

Exportar CSV

Novo parceiro

| PARCEIRO           | CATEGORIA   | SEGMENTO   | STATUS | SITUAÇÃO | FATURAMENTO   | PEDIDOS | TICKET MÉDIO | VARIAÇÃO ↑ |
|--------------------|-------------|------------|--------|----------|---------------|---------|--------------|------------|
| Forno Real         | Lanchonete  | ● Em risco | Ativo  | ● Ativo  | R\$ 2.869,72  | 79      | R\$ 36,33    | ↓ -53,67%  |
| Parada da Serra 3  | Bebidas     | ● Em risco | Ativo  | ● Ativo  | R\$ 2.053,26  | 25      | R\$ 82,13    | ↓ -39,85%  |
| Esquina Express    | Lanchonete  | ● Em risco | Ativo  | ● Ativo  | R\$ 1.379,33  | 37      | R\$ 37,28    | ↓ -26,16%  |
| Empório Dourado    | Lanchonete  | ● Em risco | Ativo  | ● Ativo  | R\$ 2.590,89  | 76      | R\$ 34,09    | ↓ -25,75%  |
| Ponto Bom Prato    | Lanchonete  | ● Em risco | Ativo  | ● Ativo  | R\$ 11.613,73 | 326     | R\$ 35,62    | ↓ -23,20%  |
| Sabor do Vale 2    | Mercado     | ● Em risco | Ativo  | ● Ativo  | R\$ 3.560,90  | 36      | R\$ 98,91    | ↓ -21,07%  |
| Empório Real 2     | Padaria     | ● Em risco | Ativo  | ● Ativo  | R\$ 75,28     | 3       | R\$ 25,09    | ↓ -20,42%  |
| Cantina do Chef    | Pizzaria    | ● Em risco | Ativo  | ● Ativo  | R\$ 8.065,90  | 114     | R\$ 70,75    | ↓ -20,25%  |
| Quintal da Praça 2 | Restaurante | ● Em risco | Ativo  | ● Ativo  | R\$ 332,01    | 7       | R\$ 47,43    | ↓ -18,57%  |
| Parada Express 2   | Gás e água  | ● Em risco | Ativo  | ● Ativo  | R\$ 3.836,80  | 40      | R\$ 95,92    | ↓ -18,48%  |

Figura 22 — Lista de parceiros filtrada por segmento e ordenada pela variação.

## 2.4 O cadastro do parceiro

O nome na lista abre o cadastro. Além dos dados do parceiro, a tela mostra o desempenho do período — o mesmo da lista — e a série individual, que fecha a metade do RF19 que ainda não tinha tela. A desativação e a exclusão ficam separadas, e as duas pedem confirmação na própria página.

GIH  
GROWTH INTELLIGENCE

Painel

Importação

Parceiros

Parceiros

Captura da Evidência  
ANALISTA

Parceiros > Forno Real

Cadastro

\* obrigatório

Nome \*

Forno Real

Como o parceiro aparece nos relatórios importados.

Categoria

Lanchonete

Status comercial

Ativo

Escolher uma categoria confirma a classificação (RN05).

Prospecção é quem ainda não converteu.

Contato

E-mail ou telefone de quem responde pelo parceiro.

Salvar alterações

Voltar para a lista

Desempenho

14/09/2026 a 20/09/2026

Segmento

Em risco

Faturamento

R\$ 2.869,72

Pedidos

79

Ticket médio

R\$ 36,33

Variação

-53,67%

Faturamento de Forno Real

12 períodos

R\$ 8 mil

R\$ 6 mil

R\$ 4 mil

R\$ 2 mil

R\$ 0

29/06

14/09

R\$ 2,9 mil

Ver os números

Situação

Ativo — entra nas consultas e nas próximas campanhas.

Desativar

Excluir parceiro

Figura 23 — Cadastro de um parceiro em risco: dados, desempenho do período, série histórica e situação.

Growth Intelligence Hub · Grupo 18 · Sprint 06 · 78

## 2.5 A importação

O relatório não traz datas, e por isso o período é informado à parte e é obrigatório. Antes de gravar, a prévia mostra o que vai entrar — e a linha com faturamento "abacaxi" é recusada com o motivo, sem impedir as outras.

GIH  
GROWTH INTELLIGENCE

Painel

Importação

Parceiros

Importação de relatório

Captura da Evidência  
ANALISTA

Relatório do período

Colar o texto ou enviar um arquivo CSV

Início do período

21 / 09 / 2026

Fim do período

27 / 09 / 2026

O relatório não traz datas. Sem o período, as métricas ficam órfãs na linha do tempo e a segmentação por tendência classifica errado sem emitir erro — por isso ele é obrigatório.

Colar o relatório

Parceiro;Faturamento;Pedidos  
Ponto Azul 2;18420,90;402  
Quintal Verde 2;15310,00;351  
Villa da Praça;abacaxi;290

...ou enviar um arquivo

Escolher arquivo

Nenhum arquivo escolhido

Ver a prévia

Prévia

2 reconhecidas · 1 rejeitadas

| LINHA           | CONTEÚDO                   | MOTIVO                               |
|-----------------|----------------------------|--------------------------------------|
| 4               | Villa da Praça;abacaxi;290 | Faturamento não numérico: 'abacaxi'. |
| PARCEIRO        | FATURAMENTO                | PEDIDOS                              |
| Ponto Azul 2    | R\$ 18.420,90              | 402                                  |
| Quintal Verde 2 | R\$ 15.310,00              | 351                                  |

Gravar 2 registros

Figura 24 — Importação com a prévia aberta: duas linhas reconhecidas e uma rejeitada com o motivo.

Gravado, o aviso diz quantos registros entraram e leva ao painel: a importação termina segmentando a base, na mesma transação, e é lá que o resultado aparece.

Importação concluída — 2 registros gravados.

1 linha(s) rejeitada(s) não entraram.

[Ver no painel](#)

Figura 25 — A importação concluída, com o caminho para o painel.

### 3. Persistência

A entrega pede que os dados continuem lá depois de fechar e reabrir o sistema. O teste abaixo faz isso de verdade, e não por simulação: um analista cadastra e altera um parceiro; o comando "docker compose down" remove os três contêineres — não os pausa — e a API para de responder; o "docker compose up -d" cria contêineres novos; e tudo é lido de novo e comparado campo a campo.

A releitura usa o mesmo cookie de antes, sem novo login. A sessão tem estado no servidor, gravada no banco, e por isso também é dado persistido: se ela existisse só na memória do processo, teria morrido com ele.

#### 3.1 Antes de desligar

O estado anotado para comparar depois:

```
parceiro      Comércio t72202 · ATIVO · contato@exemplo.test · Categoria t72202
parceiros na base  501
painel        2026-09-14 a 2026-09-20 · faturamento 1839707.96 · 41974 pedidos
límiaries     Top 15 · tendência em 2 períodos · recém-chegado até 3
```

Contêineres em execução:

```
api          5cd60abfbc28
postgres     58bc554eca60
web          7ace60ef98ba
```

Volume do banco: growth-intelligence-hub\_dados\_postgres, criado em 2026-09-15T20:18:27Z

#### 3.2 Desligar e religar

```
$ docker compose down
```

```
Container growth-intelligence-hub-web-1 Stopping
Container growth-intelligence-hub-web-1 Stopped
Container growth-intelligence-hub-web-1 Removing
Container growth-intelligence-hub-web-1 Removed
Container growth-intelligence-hub-api-1 Stopping
Container growth-intelligence-hub-api-1 Stopped
Container growth-intelligence-hub-api-1 Removing
Container growth-intelligence-hub-api-1 Removed
Container growth-intelligence-hub-postgres-1 Stopping
Container growth-intelligence-hub-postgres-1 Stopped
Container growth-intelligence-hub-postgres-1 Removing
Container growth-intelligence-hub-postgres-1 Removed
Network growth-intelligence-hub_default Removing
Network growth-intelligence-hub_default Removed
```

A API responde? não

Contêineres depois do down:

(nenhum contêiner)

Volume do banco: growth-intelligence-hub\_dados\_postgres, criado em 2026-09-15T20:18:27Z

```
[3/4] Religar – docker compose up -d
```

```
$ docker compose up -d
```

```
Network growth-intelligence-hub_default Creating
Network growth-intelligence-hub_default Created
Container growth-intelligence-hub-postgres-1 Creating
Container growth-intelligence-hub-postgres-1 Created
Container growth-intelligence-hub-api-1 Creating
Container growth-intelligence-hub-api-1 Created
Container growth-intelligence-hub-web-1 Creating
Container growth-intelligence-hub-web-1 Created
Container growth-intelligence-hub-postgres-1 Starting
Container growth-intelligence-hub-postgres-1 Started
Container growth-intelligence-hub-postgres-1 Waiting
Container growth-intelligence-hub-postgres-1 Healthy
Container growth-intelligence-hub-api-1 Starting
Container growth-intelligence-hub-api-1 Started
```

```
Container growth-intelligence-hub-web-1 Starting
Container growth-intelligence-hub-web-1 Started
```

A API voltou a responder em 2 s.

Contêineres novos:

|          |              |
|----------|--------------|
| api      | 32500f4d6700 |
| postgres | 9ed1e87f21fb |
| web      | 66ecf528d041 |

=====

### 3.3 Depois de religar

O estado lido agora:

```
parceiro      Comércio t72202 · ATIVO · contato@exemplo.test · Categoria t72202
parceiros na base  501
painel        2026-09-14 a 2026-09-20 · faturamento 1839707.96 · 41974 pedidos
limiares      Top 15 · tendência em 2 períodos · recém-chegado até 3
```

Conferências:

```
ok    o down removeu os contêineres, e a API parou de responder
ok    o up criou contêineres novos – não é o mesmo processo de antes
ok    o volume do banco é o mesmo, com a mesma data de criação
ok    a sessão aberta antes de desligar continua valendo
ok    o parceiro voltou com todos os campos iguais
ok    a alteração feita antes de desligar continua lá
ok    a base tem o mesmo número de parceiros
ok    o painel mostra os mesmos indicadores
ok    os limiares da segmentação são os mesmos
```

=====

**Resultado:** 9 de 9 conferências passaram.

O volume do banco nunca é apagado: o script monta o comando do Docker em um lugar só, e esse lugar recusa a opção que remove volumes. A transcrição completa, com as requisições do cadastro, está em docs/entrega/evidencias/sprint04/persistencia.txt.

## 4. Validações

Toda validação é do servidor. A interface ajuda — o campo de data exige a data, o campo de nome não deixa digitar além do limite — mas é o servidor que recusa, e a tela mostra a recusa que recebeu. Validar só na tela seria esconder o botão e deixar a porta aberta.

### 4.1 As regras, onde valem e o que dizem

| Regra                         | Onde       | O que o usuário lê                                                           |
|-------------------------------|------------|------------------------------------------------------------------------------|
| Período ausente (RN03)        | Importação | Por que o período é obrigatório, e que não existe valor padrão               |
| Fim antes do início           | Importação | Que o fim não pode ser anterior ao início                                    |
| Data fora do formato          | Importação | Informe uma data válida, no formato AAAA-MM-DD                               |
| Linha com dado inválido       | Importação | Na prévia, cada linha rejeitada com o conteúdo e o motivo                    |
| Nenhuma linha válida          | Importação | Nenhuma linha válida no relatório — nada foi gravado                         |
| Planilha no lugar do CSV      | Importação | Que o arquivo não é texto, e como exportar em CSV                            |
| Período já importado (RF12)   | Importação | Quantos registros existem e quem os trouxe, antes de oferecer a substituição |
| Nome vazio ou curto           | Cadastro   | Obrigatório: informe ao menos 2 caracteres                                   |
| Contato longo                 | Cadastro   | Longo demais: use no máximo 120 caracteres                                   |
| Situação fora da lista        | Cadastro   | Escolha uma destas opções, com as opções                                     |
| Categoria inexistente         | Cadastro   | Que a categoria não existe, no próprio campo                                 |
| Nome em uso                   | Cadastro   | Qual parceiro já usa o nome, com o link para ele                             |
| Exclusão com histórico        | Cadastro   | Por que não pode, e que desativar mantém o histórico                         |
| Filtro ou ordenação inválidos | Lista      | As opções válidas, ou o limite da página                                     |
| Top N zero (RF21)             | Limiares   | Use um valor a partir de 1 — e nada é gravado                                |

## 4.2 Na importação

Transcrição real, com o esperado de cada troca ao pé dela:

RN03 – sem o período, a importação é recusada e diz por quê:

```
$ POST /api/importacoes
> {
>   "texto": "Parceiro;Faturamento;Pedidos\nComércio Alfa;12500,40;312\n"
> }
422 Unprocessable Entity
< {
<   "erro": "Alguns campos precisam de correção.",
<   "campos": [
<     {
<       "campo": "periodo_inicio",
<       "mensagem": "Campo obrigatório.",
<       "ajuda": "Sem o período, as métricas ficam órfãs na linha do tempo e a segmentação por
tendência classifica errado sem emitir erro (RN03). Não existe valor padrão: informe a
data inicial e a data final do relatório."
<     },
<     {
<       "campo": "periodo_fim",
<       "mensagem": "Campo obrigatório.",
<       "ajuda": "Sem o período, as métricas ficam órfãs na linha do tempo e a segmentação por
tendência classifica errado sem emitir erro (RN03). Não existe valor padrão: informe a
data inicial e a data final do relatório."
<     }
<   ]
< }
ok    esperado: 422 com 'periodo_inicio', 'periodo_fim', 'RN03'
```

Período com o fim antes do início:

```
$ POST /api/importacoes
> {
>   "periodo_inicio": "2029-08-13",
>   "periodo_fim": "2029-08-07",
>   "texto": "Parceiro;Faturamento;Pedidos\nComércio Alfa;12500,40;312\n"
> }
422 Unprocessable Entity
< {
<   "erro": "Alguns campos precisam de correção.",
<   "campos": [
<     {
<       "campo": "requisição",
<       "mensagem": "A data final do período não pode ser anterior à inicial."
<     }
<   ]
< }
ok    esperado: 422 com 'anterior'
```

Planilha do Excel enviada no lugar do CSV:

```
$ POST /api/importacoes/arquivo/previa (formulário com arquivo)
> arquivo: relatorio.xlsx, 64 bytes, começando por b'PK\x03\x04'
> periodo_inicio: 2029-08-07
> periodo_fim: 2029-08-13
422 Unprocessable Entity
< {
<   "detail": {
<     "erro": "'relatorio.xlsx' não é um arquivo de texto.",
<     "ajuda": "O relatório precisa ser CSV ou texto. Planilha em .xlsx é arquivo compactado e
não pode ser lida assim: exporte como CSV antes de enviar."
<   }
< }
ok    esperado: 422 com 'exporte como CSV'
```

RF12 – o mesmo período de novo. O padrão é cancelar, e a recusa diz o que seria apagado antes de oferecer a substituição:

```
$ POST /api/importacoes
```

```

> {
>   "periodo_inicio": "2029-08-07",
>   "periodo_fim": "2029-08-13",
>   "texto": "Parceiro;Faturamento;Pedidos\nComércio t88084 hist;12500,40;312\n"
> }
409 Conflict
< {
<   "detail": {
<     "erro": "Este período já foi importado.",
<     "ajuda": "O padrão é cancelar, para não duplicar nem apagar dados sem querer. Para
trocar o conteúdo do período, substitua a importação anterior: as métricas atuais deste
período serão apagadas antes da gravação.",
<     "ja_existe": {
<       "importacao": 28,
<       "autor": "Analista da Evidência",
<       "enviado_em": "2026-09-21T21:19:30.174726+00:00",
<       "metricas_que_serao_apagadas": 1
<     }
<   }
< }
ok    esperado: 409 com 'metricas_que_serao_apagadas'

```

=====

Nome curto:

Contato longo:

Categoria que não existe – erro do campo, e não conflito de nome:

Growth Intelligence Hub · Grupo 18 · Sprint 06 · 86

#### 4.4 Na tela

O mesmo cadastro, pela interface: nome apagado e contato com 121 caracteres. O limite de 120 no campo só impede digitar além dele — aqui ele foi contornado de propósito, para mostrar que o servidor recusa mesmo assim. Cada mensagem aparece embaixo do seu campo, o resumo no topo leva a cada um, e o foco vai para o primeiro campo com erro. Nada foi gravado.

Cadastro

\* obrigatório

Nome \*

Obrigatório: informe ao menos 2 caracteres.

Categoria

Lanchonete

Escolher uma categoria confirma a classificação (RN05).

Status comercial

Ativo

Prospecção é quem ainda não converteu.

Contato

Longo demais: use no máximo 120 caracteres.

Salvar alterações

Voltar para a lista

Desempenho

14/09/2026 a 20/09/2026

|              |              |
|--------------|--------------|
| Segmento     | ● Em risco   |
| Faturamento  | R\$ 2.869,72 |
| Pedidos      | 79           |
| Ticket médio | R\$ 36,33    |
| Variação     | ↓ -53,67%    |

**Figura 26** — Cadastro recusado pelo servidor: as duas mensagens, cada uma no seu campo.

## 5. Mensagens de erro

Toda mensagem é em português e diz o que fazer, não só o que deu errado. A API responde sempre no mesmo formato, e é esse contrato que permite à tela mostrar cada erro no lugar certo sem conhecer as regras:

| Situação                                      | Formato                                          | Na tela                                                                                  |
|-----------------------------------------------|--------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>Campo inválido (422)</b>                   | erro + a lista de campos, cada um com a mensagem | Embaixo de cada campo, e um resumo no topo                                               |
| <b>Recusa de negócio (409)</b>                | erro + ajuda, e o que estiver em jogo            | O aviso com a ajuda e, quando há, a saída ao lado                                        |
| <b>Sem sessão ou sem permissão (401, 403)</b> | Mensagem genérica                                | Sessão encerrada leva ao login, que devolve ao lugar de antes; sem permissão, a mensagem |
| <b>Falha inesperada (500)</b>                 | Mensagem genérica + identificador de correlação  | A mensagem, sem nada do servidor                                                         |

### 5.1 A recusa aponta a saída

Excluir um parceiro com faturamento importado apagaria números de períodos já fechados. A recusa diz isso, e a própria tela oferece o que resolve: desativar, que tira o parceiro das próximas campanhas e mantém o histórico.

**Situação**

**O parceiro 'Forno Real' tem histórico e não pode ser excluído.**

Excluir apagaria registros que outros períodos já contabilizam. Para tirá-lo de circulação sem perder o histórico, desative o parceiro.

**Desativar em vez de excluir**

● Ativo — entra nas consultas e nas próximas campanhas.

**Desativar** **Excluir parceiro**

Figura 27 — Exclusão recusada, com "Desativar em vez de excluir" dentro do próprio aviso.

Nome em uso: a recusa diz qual parceiro já usa o nome e leva ao cadastro dele — o mais provável é que a pessoa quisesse editá-lo, e não criar outro.

**Já existe um parceiro com o nome 'Forno Real'.**  
Corrija o nome, ou abra o cadastro existente e edite-o em vez de criar outro.  
[Abrir o cadastro de Forno Real](#)

**Novo parceiro** \* obrigatório

Nome \*

Forno Real

Como o parceiro aparece nos relatórios importados.

Categoria Sem categoria ▼ Status comercial Ativo ▼

Escolher uma categoria confirma a classificação (RN05). Prospecção é quem ainda não converteu.

Contato

E-mail ou telefone de quem responde pelo parceiro.

**Cadastrar parceiro** **Cancelar**

Figura 28 — Cadastro recusado por nome em uso, com o link para o parceiro existente.

Período já importado: o padrão é cancelar. A substituição só é oferecida depois de dizer quantos registros seriam apagados e quem os trouxe — destruir dado não pode ser um clique distraído.

**Este período já foi importado.**  
O padrão é cancelar, para não duplicar nem apagar dados sem querer. Para trocar o conteúdo do período, substitua a importação anterior: as métricas atuais deste período serão apagadas antes da gravação.

Já existem **2** registros neste período, trazidos por Captura da Evidência. Substituir apaga todos eles.

**Substituir mesmo assim**

Figura 29 — Importação recusada por período repetido, com o que a substituição apagaria.

## 5.2 Acesso

Senha errada e login inexistente respondem exatamente igual. Duas mensagens diferentes entregariam a lista de logins válidos a quem tentasse adivinhar.

Senha errada, e depois um login que não existe. RNF11: as duas respostas são iguais – duas mensagens diferentes entregariam a lista de logins válidos.

```
$ POST /api/sessao
> {
>   "login": "t88084.analista",
>   "senha": "*****"
> }
401 Unauthorized
< {
<   "detail": "Login ou senha inválidos."
< }
ok    esperado: 401 com 'Login ou senha inválidos.'
```

```
$ POST /api/sessao
> {
>   "login": "ninguem.com.este.login",
>   "senha": "*****"
> }
401 Unauthorized
< {
<   "detail": "Login ou senha inválidos."
< }
ok    esperado: 401 com 'Login ou senha inválidos.'
ok    as duas recusas são indistinguíveis
```

### 5.3 A falha inesperada

Para mostrar o que o usuário vê quando algo quebra de verdade, o banco foi parado por alguns segundos. A resposta traz só uma mensagem genérica e um identificador; o detalhe técnico fica no log do servidor, com o mesmo identificador — é o que liga a reclamação de um usuário ao evento, sem mostrar a ele nada do servidor.

Uma tela pedindo a lista de parceiros:

```
$ GET /api/parceiros?tamanho=1
500 Internal Server Error
< {
<   "erro": "Não foi possível concluir a operação.",
<   "correlacao": "3ed06f836c97"
< }
ok   esperado: 500 com 'Não foi possível concluir a operação.', 'correlacao'
ok   a resposta não mostra nada do servidor

$ docker compose logs api (só o que tem 3ed06f836c97)
[3ed06f836c97] GET /api/parceiros
sqlalchemy.exc.OperationalError: (psycopg.OperationalError) [Errno -2] Name or service not
known
ok   o mesmo identificador está no log, com a exceção que ficou fora da resposta
```

Com o banco de volta, a mesma tela funciona sem reiniciar a API: a conexão é testada antes de cada uso, e a que morreu com o banco é descartada.

**Resultado da transcrição de validações e erros:** 27 de 27 respostas como esperado.

## 6. Navegação entre telas

Seis telas, cada uma com endereço próprio. O botão voltar desfaz o último passo, um recorte filtrado pode ser mandado por link, e um cadastro abre direto — tudo isso depende de a tela estar no endereço, e não num estado escondido da página.

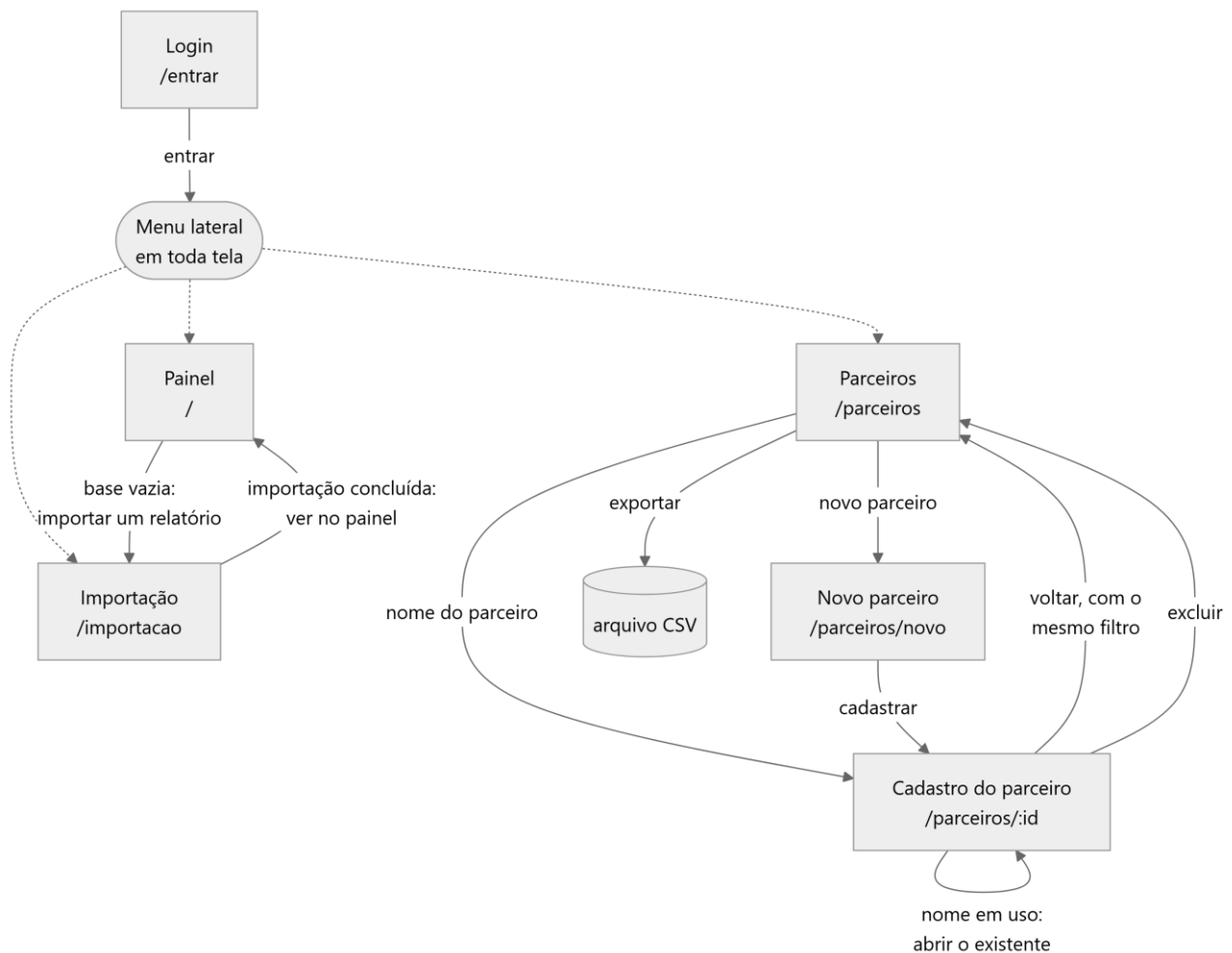


Figura 30 — Mapa das telas. As setas cheias são os caminhos que a própria tela oferece; o menu lateral está em todas.

### 6.1 O percurso da demonstração

| # | De onde, para onde                   | Como                                                                  |
|---|--------------------------------------|-----------------------------------------------------------------------|
| 1 | Login → Painel                       | Entrar; o login devolve à tela pedida antes, se houver                |
| 2 | Painel → Importação                  | Pelo menu, ou pela saída do painel vazio                              |
| 3 | Importação → Painel                  | "Ver no painel", no aviso de importação concluída                     |
| 4 | Painel → Parceiros                   | Pelo menu; o recorte vai para o endereço                              |
| 5 | Parceiros → Cadastro                 | O nome do parceiro na lista                                           |
| 6 | Cadastro → Parceiros                 | "Parceiros" na trilha, ou "Voltar para a lista" — com o mesmo recorte |
| 7 | Parceiros → Novo parceiro → Cadastro | "Novo parceiro"; cadastrado, a tela passa ao cadastro criado          |
| 8 | Cadastro → Cadastro existente        | O link da recusa por nome em uso                                      |

As capturas da seção 2 seguem esse percurso, na ordem. Três comportamentos valem para todas as telas: sessão encerrada leva ao login, que devolve ao lugar de antes; o recorte da lista vive no endereço, e por isso voltar do cadastro devolve o mesmo recorte; e toda tela vazia oferece a saída — nunca uma tela em branco sem explicação.

## 7. Repositório e commits

Repositório: [github.com/PedroMiranda243/gih-fabrica-de-software](https://github.com/PedroMiranda243/gih-fabrica-de-software)

O registro abaixo é gerado do próprio histórico do git, lido da branch principal do repositório remoto — nenhum número foi escrito à mão.

### Totais

```
-----
commits de trabalho          103
Pull Requests incorporados   27
commits direto na main       22   (até 15/09, antes de a main ser protegida)
```

### Por tipo (Conventional Commits)

```
-----
docs      31   30%
feat      27   26%
test      21   20%
fix       12   12%
ci         6    6%
perf       2    2%
spike      2    2%
chore      1    1%
fora       1   (fora da convenção:)
2026-09-15 Revert "docs: corta a central de comunicação e torna o assistente...
```

### 7.1 Como os commits são organizados

- Um assunto por commit, no padrão Conventional Commits — o tipo diz se é funcionalidade, correção, teste, documentação ou integração contínua.
- Nas correções desta entrega, o teste vai em commit próprio e é conferido reprovando no código antigo — senão não se sabe se ele testa o defeito.
- Cada Pull Request trata de um assunto só, e passa pela integração contínua antes de entrar.
- Os commits feitos direto na branch principal, antes de ela ser protegida, aparecem no registro como tais — e não foram reescritos.

## 7.2 Autoria e coautoria

O registro mostra a autoria como ela é. Todos os commits têm o mesmo integrante como autor, e os demais aparecem como coautores, pela área de responsabilidade de cada um — interface, banco e testes, documentação, integração contínua. Os Pull Requests também foram abertos e incorporados pelo mesmo integrante: a revisão cruzada entre os integrantes ainda não acontece no GitHub, e está nos próximos passos como ponto a corrigir.

Autoria e coautoria

```
-----
Cada commit tem um autor e, por área, o colega responsável como coautor
(trailer Co-authored-by, que o GitHub credita ao coautor).
autor      Pedro Miranda      103
coautor    João Pedro Nunes de França  39
coautor    Marcio Maycom      22
coautor    Ingryd Vitoria de Araújo Barbosa  12
coautor    Thiago José Falcão de Freitas  9
sem coautor                                26
```

## 7.3 Os Pull Requests desta entrega

Sprint 04 – 20/09 a 26/09: 16

- #61 20/09 Segmenta cada parceiro por regra determinística (H33)
- #52 20/09 Mede o tempo de resposta do painel com base grande (H40)
- #62 20/09 Mobilidade do Top N e o painel com paridade com o protótipo (H35)
- #63 20/09 Limiares da segmentação configuráveis sem alterar código (H34)
- #64 20/09 Filtrar, ordenar e exportar a lista de parceiros (H36, H38)
- #65 20/09 Verificação de ponta a ponta do núcleo de regras, e piso de cobertura (H69)
- #66 21/09 Sazonalidade do gerador sem viés de direção nem de calendário (#60)
- #67 21/09 As recusas do cadastro de parceiro dizem o que aconteceu (UC04)
- #69 21/09 Tela de cadastro de parceiro – reaberta: a #68 não chegou à main
- #70 21/09 Os contêineres voltam sozinhos quando o Docker volta
- #71 21/09 Modelagem atualizada e mapa de navegação entre as telas
- #72 21/09 A recusa do período já importado fala com quem usa a tela
- #73 21/09 As evidências da Sprint 03 ganham pasta própria, e a Parte III fica como foi...
- #74 21/09 Geradores das evidências da Sprint 04 – persistência, validações, capturas e...
- #75 21/09 A importação concluída leva ao painel
- #76 21/09 Nenhuma mensagem de validação sai em inglês, e o 500 ganha teste

## 8. Execução e roteiro de demonstração

O ambiente sobe com um comando, como na Sprint 03, e agora volta sozinho quando o Docker é reiniciado. A base de demonstração é gerada e já sai segmentada:

```
docker compose up -d
python scripts/resetar_banco.py --parceiros 500      # base sintética, já segmentada
docker compose exec api python -m app.cli criar-usuario --login seu.login --nome "Seu Nome"
```

### 8.1 Roteiro para a orientação

| # | Passo                                                                                                   |
|---|---------------------------------------------------------------------------------------------------------|
| 1 | Entrar e mostrar o painel: indicadores, mobilidade do Top N, série, distribuição por segmento e ranking |
| 2 | Ir a Parceiros, filtrar por "Em risco" e ordenar pela variação; exportar o recorte em CSV               |
| 3 | Abrir um parceiro pelo nome: desempenho, série individual; tentar excluir e mostrar a recusa            |
| 4 | Voltar pela trilha e mostrar que o recorte continua                                                     |
| 5 | Novo parceiro com o nome de um existente: a recusa e o link para ele                                    |
| 6 | Importação: prévia com uma linha ruim, gravar, e "Ver no painel"                                        |
| 7 | Importar o mesmo período de novo: a recusa com o que seria apagado                                      |
| 8 | Fechar a aplicação com "docker compose down", subir de novo e mostrar que tudo continua                 |

Antes da orientação, o roteiro é ensaiado na máquina que vai ser usada. O Docker Desktop precisa estar aberto antes, e a verificação de ponta a ponta é a forma mais rápida de conferir que tudo subiu.

## 9. Dificuldades encontradas

Todas reais, com a causa e a correção — e várias achadas pelas próprias ferramentas de verificação, que é para isso que elas existem.

### 9.1 A massa de demonstração subia inteira na mesma semana

Com o gerador de dados sintéticos, 43% da rede saía "em ascensão" e só 4% "em risco" — longe dos perfis de trajetória que o gerador deveria produzir. A causa era a sazonalidade, que empurrava todos os parceiros na mesma direção nas mesmas semanas. A correção sugerida na issue, trocar a forma da onda, acertava com 12 períodos e voltava a enviar com 13.

**Correção:** cada parceiro passou a ter a própria fase na onda sazonal, sorteada a partir da semente. Medido antes e depois, com 12 e com 13 períodos: as contagens de duas altas e de duas quedas seguidas ficaram equilibradas nos dois casos.

### 9.2 A exportação pendurava sem erro

O CSV da lista é gerado aos poucos, enquanto é baixado. A primeira versão lia do banco pela conexão da requisição — que o servidor já tinha devolvido quando o download começava. Os testes travavam sem mensagem nenhuma.

**Correção:** a exportação abre a própria conexão, que vive enquanto o arquivo é gerado, e fecha no fim.

### 9.3 A medição de desempenho mentia

A primeira medição do painel com 5.000 parceiros acusou varredura completa de tabela onde havia índice. O banco tinha acabado de ser populado, e as estatísticas do planejador ainda eram as de antes da carga; e a varredura, onde aparecia, era a escolha certa para aquele tamanho de tabela.

**Correção:** a medição atualiza as estatísticas antes de medir, e só reprova a varredura quando, forçado a usar índice, o banco não tem nenhum para usar. Medição surpreendente geralmente é medição quebrada.

### 9.4 Um script de verificação quebrado, com a integração contínua verde

A lista de parceiros passou a responder em páginas, e o script de transcrição, que não roda na integração contínua, continuou lendo a resposta como lista. A integração contínua seguiu verde, e o defeito só apareceu quando a transcrição foi executada de novo.

**Correção:** o script passou a ler a página. A lição ficou registrada: verde na integração contínua cobre o que ela executa, e os scripts de evidência rodam antes de cada entrega.

### 9.5 Categoria inexistente respondia "nome duplicado"

Os testes escritos para a tela de cadastro acharam um defeito real: cadastrar um parceiro com uma categoria que não existe violava a chave estrangeira, e a API tratava toda violação de integridade como nome repetido.

**Correção:** a categoria é conferida antes de gravar, e o erro vai para o próprio campo; o nome repetido é reconhecido pela restrição exata que o banco reporta, e não por qualquer violação.

### 9.6 Todo aviso de sucesso saía com a barra vermelha de erro

A cor verde do aviso de sucesso estava na folha de estilo da importação, carregada antes da folha base. Com a mesma especificidade, a regra do aviso de erro, que vinha depois, apagava a cor. Medido no navegador: a barra saía em vermelho em todas as telas.

**Correção:** a variante mudou-se para a folha base, colada à regra que ela modifica.

### 9.7 Mensagens de validação em inglês

A transcrição de validações desta entrega achou três tipos de erro que o tradutor não conhecia — situação fora da lista, filtro de situação inválido e número com casas decimais — e que chegavam ao usuário como "Input should be...".

**Correção:** os três ganharam mensagem própria, e qualquer tipo que ainda não estiver na tabela cai numa frase genérica em português, e não na original. Um teste varre várias entradas ruins de uma vez atrás das palavras da frase original.

### 9.8 Um Pull Request incorporado num ramo morto

A tela de parceiro foi aberta sobre o ramo de outro Pull Request, ainda não incorporado. O primeiro entrou antes, e o segundo foi incorporado ao ramo dele — que já não ia para lugar nenhum. O trabalho não chegou à branch principal, e a integração contínua não rodou, porque só roda para Pull Requests contra ela.

**Correção:** o trabalho foi reaberto contra a branch principal. Desde então, um Pull Request que depende de outro só é aberto depois que o primeiro entra.

## 10. Ajustes no planejamento, na arquitetura e na modelagem

A modelagem e a especificação são documentos vivos: a Parte II deste documento já mostra o modelo atual, e esta seção registra o que mudou e por quê. As evidências de execução, ao contrário, são retrato — a Parte III está exatamente como foi entregue em 19/09.

| O que mudou                                  | De / para                                    | Por quê                                                                                                                                                      |
|----------------------------------------------|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tabela <code>configuracao_segmentacao</code> | 16 para 17 tabelas                           | O RF21 pede limiares configuráveis sem alterar código. Uma linha só — o banco cobra isso —, com o autor e a data da última alteração                         |
| Segmentação na importação                    | Recálculo à parte para a mesma transação     | Era um próximo passo da Sprint 03. Ou o período entra com métrica e classificação, ou não entra: período sem segmento deixaria o painel incompleto sem aviso |
| Resposta da lista de parceiros               | Lista para página                            | Com 5.000 parceiros, devolver tudo a cada filtro não se sustenta. A página traz o total e o período de referência, e a interface passou a navegar por ela    |
| Regras de cálculo                            | Dentro das rotas para módulos próprios       | Ranking, variação e ticket passaram a um lugar só, usado pelo painel, pela lista e pela exportação — com cobertura mínima cobrada a cada Pull Request        |
| Tela de cadastro de parceiro                 | Só na API para tela própria                  | A entrega pede validações nos formulários, e a interface tinha dois. O cadastro já existia na API; ganhou tela, desempenho e série individual                |
| Mensagens de recusa                          | Linguagem de API para linguagem de usuário   | Duas recusas orientavam com termos técnicos da API, e apareciam assim na tela. Hoje dizem a ação, e a tela oferece o botão                                   |
| Gerador de dados sintéticos                  | Sazonalidade comum para fase por parceiro    | A massa de demonstração saía enviesada — seção 9.1                                                                                                           |
| Ambiente                                     | Contêineres parados para reinício automático | O Docker Desktop desta equipe cai quando a máquina fica ociosa; os contêineres agora voltam sozinhos com ele                                                 |

### 10.1 Planejamento

A equipe segue à frente do calendário interno. A Sprint 7 interna — segmentação e mobilidade do Top N — estava prevista para 19/10 a 23/10 e foi concluída em 21/09. A folga não muda o plano: vira margem para o núcleo em GPU, onde o risco técnico do projeto se concentra. A tabela de equivalência entre as sprints da disciplina e as da equipe foi atualizada no cronograma: a Sprint 03 corresponde à interna 6, e esta à interna 7.

### 10.2 Um ponto em aberto

A regra de segmentação precisa de um limiar para "recém-chegado", e ele não estava na especificação. O valor em uso, três períodos, foi derivado da própria regra e está configurável — mas espera o aval do Product Owner, registrado na issue #59. Declarado aqui em vez de apresentado como decidido.

## 11. Próximos passos

| Sprint | Tema                    | O que entra                                                                           |
|--------|-------------------------|---------------------------------------------------------------------------------------|
| 8      | Modelo preditivo        | Treino a partir do histórico, previsão de faturamento e probabilidade de queda        |
| 9 a 11 | Núcleo computacional    | Otimizador serial, paralelo em CPU e paralelo em GPU, com o comparativo de desempenho |
| 12     | Central de comunicação  | Geração de mensagens por segmento, com aprovação humana obrigatória                   |
| 13     | Assistente e fechamento | Perguntas em linguagem natural sobre dados já apurados, e a entrega final             |

### 11.1 O que fecha o módulo

- A tela dos limiares da segmentação, para o Administrador — hoje só pela API e pelo terminal.
- O histórico de importações na interface, com autor, data e período de cada uma.
- A sugestão de categoria pelo nome do parceiro, na issue #35.
- O aval do Product Owner sobre o limiar de recém-chegado, na issue #59.

### 11.2 O que a equipe corrige no processo

- A tela de administração de usuários, lacuna declarada desde a Sprint 03.
- Revisão cruzada nos Pull Requests: cada um revisado pelo dono da área antes de entrar, e não só pelo autor.
- Os scripts de evidência rodados antes de cada entrega, e não só a integração contínua.

*Projeto acadêmico. © 2026 os autores — todos os direitos reservados. Repositório público significa visível para avaliação e portfólio; não constitui licença de uso, cópia ou redistribuição.*

## Parte V — Sprint 05: Segundo Módulo Funcionando

O segundo módulo é o de Previsão — a parte preditiva do módulo M4 do escopo, anunciada como próximo passo na Parte IV. A partir do histórico que o primeiro módulo grava, o sistema treina uma rede neural, mede o erro dela contra contas simples no mesmo conjunto de teste, põe em uso a versão só se ela for melhor, e mostra, no cadastro de cada parceiro, o faturamento previsto para o próximo período e a chance de ele entrar em risco. É a pergunta "quem vai cair?" do problema do produto, respondida com número medido.

Cada item desta parte vem com execução real: transcrições geradas por scripts contra a aplicação no ar, capturas da interface sobre a base de demonstração, e registros de testes, de bugs e de commits gerados de fonte — a suíte rodada, as issues do GitHub, o próprio histórico do git.

### As cinco entregas, e onde estão

| # | Entrega                                        | Situação            | Evidência neste documento                                       |
|---|------------------------------------------------|---------------------|-----------------------------------------------------------------|
| 1 | <b>Segundo módulo completo, com fluxo real</b> | Funcionando         | Seções 1 e 2 — o fluxo, o modelo medido e as telas              |
| 2 | <b>Integração com o banco</b>                  | Funcionando         | Seção 3 — armazenar, consultar, atualizar, e depois de religar  |
| 3 | <b>Regras de negócio atualizadas</b>           | Registradas         | Seção 4 — a regra nova e as que mudaram, com o porquê           |
| 4 | <b>Testes das funcionalidades</b>              | Todos passando      | Seção 5 — as três suítes e os quatro tipos que o enunciado pede |
| 5 | <b>Correção dos bugs</b>                       | 12 de 12 corrigidos | Seção 6 — o registro, gerado das issues                         |

## 1. O segundo módulo: Previsão

### 1.1 O fluxo

Um gestor abre a tela Modelo, vê a versão em uso e as métricas dela, e dispara um treino. O treino roda fora da requisição — a tela acompanha o andamento e anuncia o resultado —, e as previsões passam a aparecer no cadastro de cada parceiro. Uma importação nova deixa a previsão "com dados até" um período antigo, e um novo treino a atualiza.

| Etapa     | O que o sistema faz                                                                                | Requisito     | Tela     |
|-----------|----------------------------------------------------------------------------------------------------|---------------|----------|
| Consultar | Mostra a versão em uso, com data, período-base, volume e métricas, e se dá para treinar agora      | UC07, RF27    | Modelo   |
| Treinar   | Grava o treino em andamento e o executa em segundo plano; um por vez, travado pelo banco           | UC07, H45     | Modelo   |
| Separar   | Monta a janela dos 4 períodos de cada parceiro e separa treino, validação e teste pelo tempo       | H41, RN09     | —        |
| Medir     | Treina a rede e mede o erro dela e o de três referências no mesmo conjunto de teste                | H42, H43, H46 | Modelo   |
| Decidir   | Põe a versão em uso só se ela superar as referências nas duas saídas; senão fica a anterior        | UC07-A1       | Modelo   |
| Prever    | Grava a previsão do próximo período e o risco de cada parceiro, com a versão que a produziu        | RF27, UC07-A2 | —        |
| Mostrar   | Exibe previsão e risco no cadastro do parceiro, marcados como estimativa, ou o motivo de não haver | RF28, H44     | Parceiro |

### 1.2 O modelo, e contra o que ele se mede

Uma rede neural pequena, escrita em PyTorch, com duas saídas: o faturamento do próximo período e a chance de o parceiro estar Em Risco nele. Ela olha os quatro últimos períodos de cada parceiro — faturamento e pedidos, ticket médio, tendência, posição no ranking do período, categoria e tempo de casa — e aprende com o histórico inteiro da rede. O último período da base testa, o penúltimo valida, e os anteriores treinam: embaralhar os períodos vazaria o futuro para dentro do treino e daria uma métrica boa e falsa.

Uma rede que não bate uma conta de cabeça não agrega nada, e por isso ela é medida contra três referências: repetir o último período, a média móvel dos últimos quatro, e — para o risco — a taxa observada no treino, separada por "caiu no último período". A tabela abaixo é a medição publicada no repositório: três redes geradas, cinco sementes de treino em cada uma, mediana dos quinze treinos.

| Medida (mediana de 15 treinos)               | Rede  | Repetir o último | Média móvel | Taxa observada |
|----------------------------------------------|-------|------------------|-------------|----------------|
| Erro no faturamento (MAPE), 500 parceiros    | 9,8%  | 11,5%            | 10,2%       | —              |
| Erro no faturamento (MAPE), 5.000 parceiros  | 9,9%  | 12,2%            | 10,6%       | —              |
| Erro no risco (Brier), 500 parceiros         | 0,105 | —                | —           | 0,122          |
| Erro no risco (Brier), 5.000 parceiros       | 0,096 | —                | —           | 0,122          |
| Erro de calibração do risco, 500 parceiros   | 0,042 | —                | —           | 0,003          |
| Erro de calibração do risco, 5.000 parceiros | 0,010 | —                | —           | 0,001          |

- A rede superou as referências nas duas saídas em 15 de 15 treinos com 500 parceiros e em 15 de 15 com 5.000.
- A vantagem no faturamento é pequena: mediana de 0,6 ponto percentual sobre a melhor referência de cada treino, com 500 parceiros, e de 0,6 ponto percentual com 5.000. Está registrada como é.

- O risco da rede separa melhor quem cai de quem não cai — Brier menor —, mas é menos calibrado que a taxa observada, que é calibrada por construção: é a própria frequência, em dois grupos. A rede ganha no Brier porque distingue os parceiros dentro de cada grupo.
- Tempo de treino: mediana de 1,0 s com 500 parceiros e 7,5 s com 5.000, numa thread — medido, e é o que permite treinar em segundo plano no próprio processo da API.

Se a rede não superasse as referências, o módulo funcionaria do mesmo jeito: a versão não entraria em uso, e as previsões saíam da referência, identificadas assim na tela. Conclusão negativa bem medida é resultado válido — a arquitetura registrou isso antes de a rede existir.

### 1.3 Números do módulo

| Medida                                                | Valor                               |
|-------------------------------------------------------|-------------------------------------|
| Testes da API                                         | 557, todos passando                 |
| Testes do modelo preditivo                            | 31, todos passando                  |
| Testes da interface                                   | 110, todos passando                 |
| Verificação de ponta a ponta contra a aplicação no ar | 81 verificações passaram            |
| Operações da API                                      | 37 — eram 31 na Sprint 04           |
| Tabelas no banco                                      | 18 — a do treino do modelo é a nova |
| Imagem da API, com o PyTorch da roda CPU              | 326 MB → 1,6 GB                     |

## 2. O módulo funcionando

### 2.1 A tela do modelo

Para Administrador e Gestor, pelo menu — que vem do servidor, a partir das permissões das próprias rotas. À esquerda, os fatos da versão em uso; à direita, o erro da rede ao lado do erro das referências, um gráfico por métrica. A rede na cor dos gráficos, as referências em cinza: o gráfico existe para responder uma pergunta só — a rede erra menos que a conta simples? A barra começa no zero, e o número vem escrito na ponta.



Figura 31 — A versão em uso, com a comparação contra as referências no mesmo conjunto de teste.

## 2.2 Treinar

O treino pede confirmação, que diz o que vai acontecer: roda em segundo plano, e a versão nova só entra em uso se errar menos que as referências nas duas saídas.



Figura 32 — A confirmação antes do treino.

A requisição volta na hora, com o treino gravado em andamento. A tela consulta o estado a cada dois segundos; quem sair e voltar reencontra o andamento, porque ele vem do servidor e não da memória da página. A barra corre sem porcentagem, porque não há como sabê-la — a parada antecipada decide quando o treino acaba.

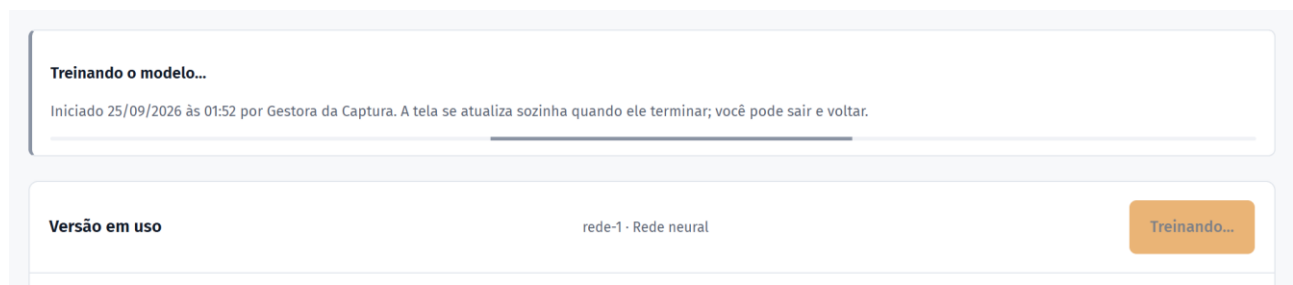


Figura 33 — O treino em andamento, com o botão desligado até ele terminar.

Ao terminar, o resultado é anunciado — a versão nova entrou em uso, ou ficou a anterior, com o motivo escrito pela API. Um leitor de tela ouve o mesmo anúncio.



Figura 34 — Treino concluído: a versão nova entrou em uso, com as métricas dela.

| Histórico de treinos |                    |           |              |             |                |                       | 2 treinos |
|----------------------|--------------------|-----------|--------------|-------------|----------------|-----------------------|-----------|
| INICIADO             | POR                | SITUAÇÃO  | ERRO DA REDE | MÉDIA MÓVEL | ÚLTIMO PERÍODO | RESULTADO             |           |
| 25/09/2026 às 01:52  | Gestora da Captura | Concluído | 9,2%         | 10,0%       | 11,5%          | rede-18 entrou em uso |           |
| 24/09/2026 às 21:28  | terminal           | Concluído | 9,2%         | 10,0%       | 11,5%          | rede-1 entrou em uso  |           |

Figura 35 — O histórico de treinos: quando, por quem, o erro da rede e das referências, e o que cada um deixou.

## 2.3 A previsão no cadastro do parceiro

Abaixo do desempenho medido, o bloco "Próximo período", com a etiqueta Estimativa: o faturamento previsto, a chance de estar em risco, a data dos dados de onde a previsão parte e a versão que a produziu. Número previsto ao lado de número medido, sem marca, seria lido como medição.



Figura 36 — O desempenho do período mais recente e, embaixo, a previsão do próximo, marcada como estimativa.

Na série do parceiro, o trecho até o próximo período é tracejado, com marcador vazado, e a legenda diz qual é qual. Cor diferente sozinha não bastaria: quem não a distingue leria a previsão como medição.



**Figura 37** — A série histórica com a estimativa do próximo período, tracejada.

## 2.4 Verificação de ponta a ponta

A verificação roda contra a aplicação no ar e ganhou a seção do modelo — que roda antes de qualquer importação da própria verificação, porque ela grava semanas no futuro e o treino parte do período mais recente. Os blocos das sprints anteriores continuam passando na mesma execução.

- [ + ] Modelo preditivo – treino, versão em uso e previsão
  - ok o analista não treina o modelo treinar muda as previsões de toda a equipe (UC07)
  - ok a tela do modelo diz se dá para treinar e por quê 12 períodos na base; mínimo 8 (RN09)
  - ok o treino é aceito e roda fora da requisição HTTP 202
  - ok um segundo treino, com o primeiro rodando, é recusado um treino por vez, travado pelo banco (ADR-010)
  - ok o treino termina e registra data, volume e métricas (RF27) 500 parceiros · 12 períodos · 0,82 s
  - ok a versão em uso é a que venceu as referências, ou a anterior com o motivo (UC07-A1) rede 9,2% contra 10,0% da melhor referência; em uso: rede-14
  - ok o cadastro do parceiro mostra a previsão, com base e versão (RF28) Ponto Azul 2: R\$ 84818,54 previstos, risco de 24%
  
- [ + ] Limpeza – o banco volta ao que era antes da execução
  - ok remove do banco o que a execução gravou 3 períodos, 4 importações, 5 métricas, 4 parceiros, 1 categoria, 2 treinos
  - ok o painel volta a abrir no mesmo período, com os mesmos totais período de 2026-09-14
  - ok a versão do modelo em uso volta a ser a de antes rede-1
    - os usuários da execução ficam, desativados: a trilha de auditoria aponta para eles

**Resultado:** As 81 verificações passaram. O treino da verificação sai na limpeza, e a versão em uso volta a ser a de antes — conferido.

### 3. Integração com o banco

O módulo grava, consulta e atualiza no mesmo banco do primeiro, e a transcrição abaixo mostra as três coisas contra a aplicação no ar. Ela termina lendo as linhas do banco por SQL — só leitura —, para mostrar o que ficou gravado de fato, e não o que a API diz que gravou.

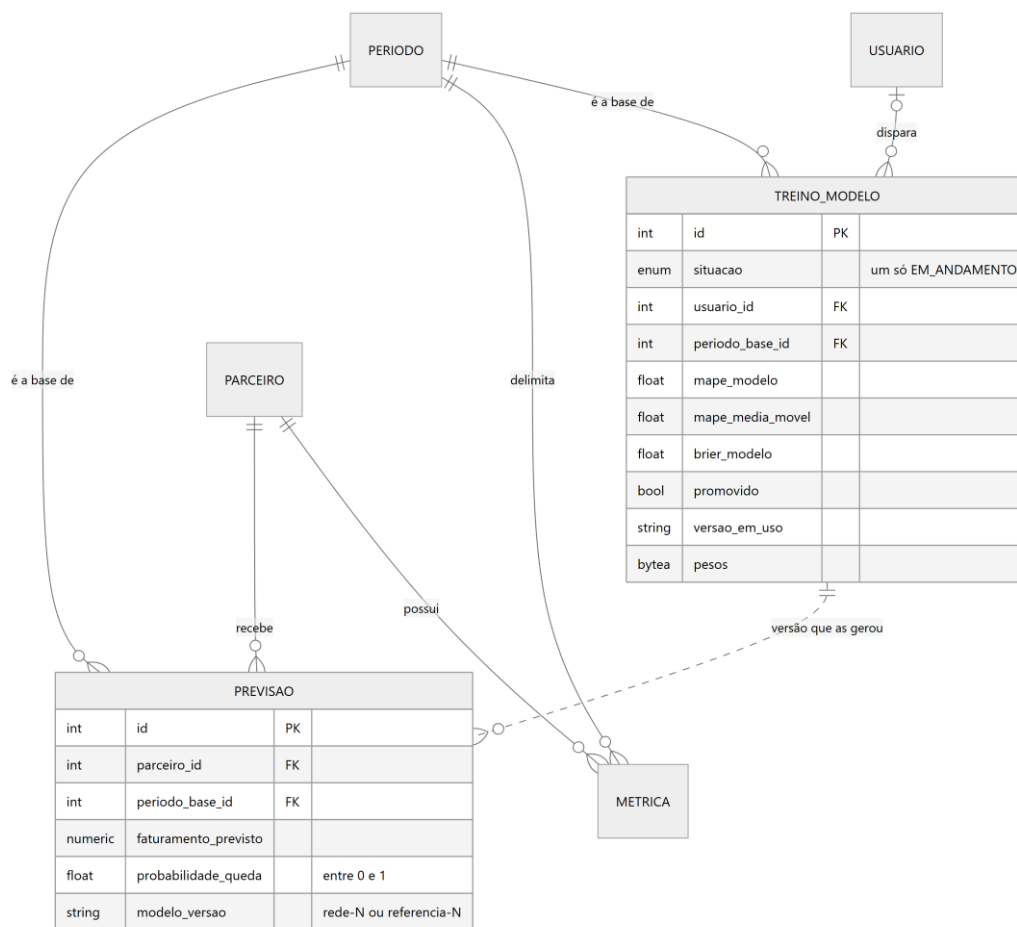


Figura 38 — O recorte do modelo de dados que o módulo usa. A ligação tracejada não é chave estrangeira: vem da versão, pelo nome.

### 3.1 Armazenar — o treino gravado

O gestor dispara o treino. A resposta já traz o registro gravado em andamento, com o autor e o período-base; o volume e as métricas chegam na mesma linha quando o treino termina.

```
$ POST /api/modelo/treinos
202 Accepted
< {
  < "id": 19,
  < "situacao": "EM_ANDAMENTO",
  < "autor": "Gestor da Evidência",
  < "iniciado_em": "2026-09-25T05:05:04.416327Z",
  < "concluido_em": null,
  < "periodo_base": {
  < "id": 12,
  < "data_inicio": "2026-09-14",
  < "data_fim": "2026-09-20"
  < },
  < "volume": {
  < "parceiros": null,
  < "periodos": null,
  < "amostras_treino": null,
  < "amostras_validacao": null,
  < "amostras_teste": null
  < },
  < "metricas": {
  < "mape_modelo": null,
  < "mape_ultimo": null,
  < "mape_media_movel": null,
  < "brier_modelo": null,
  < "brier_referencia": null,
  < "calibracao_modelo": null,
  < "calibracao_referencia": null
  < },
  < "curva": [],
  < "segundos": null,
  < "promovido": null,
  < "versao": "rede-19",
  < "versao_em_uso": null,
  < "motivo": null
  < }
ok o treino é aceito e gravado em andamento (202)

... 2 consulta(s) a GET /api/modelo/treinos/19 até a situação mudar (1.0 s)
```

### 3.2 Consultar — a previsão no cadastro

O analista abre o cadastro de Ponto Azul 2, o maior do período mais recente:

```
$ GET /api/parceiros/246/previsao
200 OK
< {
  < "disponivel": true,
  < "faturamento_previsto": "84818.54",
  < "probabilidade_queda": 0.23553469215055362,
  < "periodo_base": {
    < "id": 12,
    < "data_inicio": "2026-09-14",
    < "data_fim": "2026-09-20"
  < },
  < "modelo_versao": "rede-19",
  < "origem": "MODELO",
  < "gerada_em": "2026-09-25T05:05:04.428378Z",
  < "desatualizada": false,
  < "motivo": null,
  < "ajuda": null
  < }
ok a previsão do parceiro sai da versão em uso, com a base
```

### 3.3 Atualizar — um novo treino muda a versão em uso

Um segundo treino grava uma versão nova, e o mesmo cadastro, lido de novo, passa a mostrar a previsão dela:

O cadastro de Ponto Azul 2, de novo:

```
$ GET /api/parceiros/246/previsao
200 OK
< {
  < "disponivel": true,
  < "faturamento_previsto": "84818.54",
  < "probabilidade_queda": 0.23553469215055362,
  < "periodo_base": {
    < "id": 12,
    < "data_inicio": "2026-09-14",
    < "data_fim": "2026-09-20"
  < },
  < "modelo_versao": "rede-20",
  < "origem": "MODELO",
  < "gerada_em": "2026-09-25T05:05:05.537493Z",
  < "desatualizada": false,
  < "motivo": null,
  < "ajuda": null
  < }
ok a previsão do parceiro passou a sair da versão nova
```

### 3.4 No banco

Os dois treinos, com os pesos da rede na própria linha — poucos KB, o que dispensa arquivo em volume e faz a versão em uso sobreviver a reinício —, e as previsões das duas versões sobre o mesmo período: elas coexistem, e é isso que permite comparar versões sobre o mesmo histórico.

```
$ SQL, só leitura, no banco da API
> SELECT id, situacao, parceiros, amostras_teste,
> round(mape_modelo::numeric, 4), round(brier_modelo::numeric, 4),
> promovido, versao_em_uso, octet_length(pesos)
> FROM treino_modelo WHERE id IN (:a, :b) ORDER BY id
< id | situacao | parceiros | teste | mape | brier | promovido | em_uso | bytes
< ---+-----+-----+-----+-----+-----+-----+-----+-----
< 19 | CONCLUIDO | 500 | 469 | 0.0922 | 0.1051 | True | rede-19 | 11781
< 20 | CONCLUIDO | 500 | 469 | 0.0922 | 0.1051 | True | rede-20 | 11781
ok os dois treinos estão gravados, com os pesos na própria linha (ADR-010)

$ SQL, só leitura, no banco da API
> SELECT modelo-versao, periodo_base_id, count(*)
> FROM previsao WHERE modelo-versao IN (:a, :b)
> GROUP BY modelo-versao, periodo_base_id ORDER BY modelo-versao
< modelo-versao | periodo_base_id | previsoes
< -----+-----+-----
< rede-19 | 12 | 469
< rede-20 | 12 | 469
ok as previsões das versões coexistem no banco sobre o mesmo período (UC07-A2)
```

**Resultado da transcrição:** 10 de 10 conferências passaram.

### 3.5 Depois de desligar e religar

O teste de persistência da Sprint 04 — "docker compose down", que remove os contêineres, e "up", que cria outros — passou a conferir também o modelo: a versão em uso, com as mesmas métricas, e a previsão de um parceiro, lidas antes e depois. Se algo do modelo morasse na memória do processo, teria morrido com ele.

O estado anotado para comparar depois:

|                   |                                                                   |
|-------------------|-------------------------------------------------------------------|
| parceiro          | Comércio t57729 · ATIVO · contato@exemplo.test · Categoria t57729 |
| parceiros na base | 501                                                               |
| painel            | 2026-09-14 a 2026-09-20 · faturamento 1839707.96 · 41974 pedidos  |
| limiares          | Top 15 · tendência em 2 períodos · recém-chegado até 3            |
| modelo em uso     | rede-1 (treino 1) · MAPE 0.0922 · Brier 0.1051                    |
| previsão          | R\$ 84818.54 · risco 0.2355 · rede-1                              |

Contêineres em execução:

|          |              |
|----------|--------------|
| api      | e8fe5bba5702 |
| postgres | ae4fb994acba |
| web      | 2f8bcff54ca4 |

Volume do banco: growth-intelligence-hub\_dados\_postgres, criado em 2026-09-15T20:18:27Z

O estado lido agora:

|                   |                                                                   |
|-------------------|-------------------------------------------------------------------|
| parceiro          | Comércio t57729 · ATIVO · contato@exemplo.test · Categoria t57729 |
| parceiros na base | 501                                                               |
| painel            | 2026-09-14 a 2026-09-20 · faturamento 1839707.96 · 41974 pedidos  |
| limiares          | Top 15 · tendência em 2 períodos · recém-chegado até 3            |
| modelo em uso     | rede-1 (treino 1) · MAPE 0.0922 · Brier 0.1051                    |
| previsão          | R\$ 84818.54 · risco 0.2355 · rede-1                              |

Conferências:

- ok o down removeu os contêineres, e a API parou de responder
- ok o up criou contêineres novos — não é o mesmo processo de antes
- ok o volume do banco é o mesmo, com a mesma data de criação
- ok a sessão aberta antes de desligar continua valendo
- ok o parceiro voltou com todos os campos iguais
- ok a alteração feita antes de desligar continua lá
- ok a base tem o mesmo número de parceiros
- ok o painel mostra os mesmos indicadores
- ok os limiares da segmentação são os mesmos

ok a versão do modelo em uso é a mesma, com as mesmas métricas  
ok a previsão do parceiro é a mesma, da mesma versão

**Resultado:** 11 de 11 conferências passaram.

## 4. Regras de negócio implementadas e atualizadas

### 4.1 A regra nova: RN09

O caso de uso de treino e a história do risco falavam em "probabilidade de queda" e em "mínimo de períodos", e nenhum dos dois estava definido na especificação. A regra do projeto manda perguntar antes de escolher: a pergunta foi registrada na issue #85, decidida, e virou a RN09.

| # | A regra                                                                                                               | Por quê                                                                                                                               |
|---|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Queda prevista é estar Em Risco no período seguinte, pelo critério da RN01 — quedas seguidas até o limiar configurado | O rótulo do treino sai da mesma função que classifica o segmento: modelo e segmentação nunca discordam, e mudar o limiar muda os dois |
| 2 | O treino exige 8 períodos na base; abaixo disso, é recusado dizendo quantos faltam                                    | Quatro de janela, dois para treinar, um para validar e um para testar — sem isso não há como separar no tempo                         |
| 3 | Parceiro com menos de 4 períodos, ou fora do período mais recente, não recebe previsão — e a tela diz por quê         | A janela é de 4 períodos, e a previsão parte do período mais recente                                                                  |
| 4 | Versão que não supera a referência não entra em uso; sem versão anterior, vale a referência                           | Substituir por um modelo pior não é conteúdo técnico; resultado negativo bem medido é                                                 |
| 5 | Previsão é estimativa e aparece como tal, com o período-base e a versão                                               | Critério de aceite da história H44                                                                                                    |

### 4.2 O que mudou desde a Sprint 04

| Regra ou documento                          | O que mudou                                                                                                      | Por quê                                                                                              |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>RN05 — categoria sugerida</b>            | Passou a dizer como a categoria é inferida: palavra inteira, exatamente uma categoria, só se existe e está ativa | A sugestão pelo nome foi implementada (RF15); regra aprovada na issue #35, e escrita antes do código |
| <b>"Pendente de classificação"</b>          | É quem não tem categoria confirmada — em branco ou só sugerida                                                   | Com a sugestão automática, "sem categoria" deixaria de fora quem tem só palpite                      |
| <b>UC03 — histórico de importações</b>      | O Administrador passa a ler o histórico                                                                          | O caso de uso contradizia o RF13, que dá o histórico a ele; prevaleceu o requisito                   |
| <b>UC07 — treino do modelo</b>              | O mínimo do fluxo de exceção passou a ser o da RN09, e a nota diz como o caso aparece na interface               | O mínimo não estava definido                                                                         |
| <b>Modo de execução, no modelo de dados</b> | SERIAL, CPU_PARALELO e GPU — os valores que o banco tem                                                          | O documento dizia outros, desde a Sprint 2 — issue #106                                              |

### 4.3 Onde as regras são decididas

- No servidor, sempre: a tela recebe "pode treinar" e o porquê, o motivo de não haver previsão, e o que o treino deixou — ela não compara erro com erro para concluir nada.
- O modelo preditivo é um pacote à parte, que prevê e não decide: recebe números e devolve números. Quem diz o que é queda e se uma versão entra em uso é a API.
- A autorização também é regra: treinar é de Administrador e Gestor; a previsão no cadastro é de Gestor e Analista. A matriz de permissões é testada rota por rota, perfil por perfil.

## 5. Testes realizados e resultados

O registro abaixo foi gerado rodando as três suítes — a da API contra um banco PostgreSQL de verdade, separado do de trabalho —, e não escrito à mão. Os defeitos de comportamento corrigidos nesta entrega vieram com teste — o da tela que seguia oferecendo treino reprova no código antigo —; os de documento e de estilo foram conferidos no próprio documento e nas capturas.

| Suíte                     | Testes | Passaram | Falharam | Tempo |
|---------------------------|--------|----------|----------|-------|
| API (pytest, Postgres)    | 557    | 557      | 0        | 174 s |
| Modelo preditivo (pytest) | 31     | 31       | 0        | 6 s   |
| Interface (Vitest)        | 110    | 110      | 0        | 33 s  |

**Resultado:** 698 testes, 698 passaram, 0 falharam.

### 5.1 Os quatro tipos que o enunciado pede

Fluxos principais, operações com o banco, validações e situações de erro — com exemplos nomeados, conferidos contra a execução: se um exemplo deixasse de existir, o registro recusaria em vez de publicar um teste que não rodou.

#### Fluxos principais

```
ok o treino pela tela grava data, volume e métricas (RF27, UC07)
  api · test_previsao.py · test_treinar_registra_data_volume_e_metricas
ok a previsão chega ao cadastro do parceiro com base e versão (RF28)
  api · test_previsao.py · test_previsao_no_cadastro_e_estimativa_com_base_e-versao
ok a rede supera as referências nas duas saídas (H42, H43)
  modelo · test_treino.py · test_na_rede_sintetica_a_rede_supera_as_referencias
ok a importação grava as métricas do período (módulo da Sprint 04)
  api · test_importacao.py · test_importa_e_grava_as_metricas
ok a tela dispara o treino, acompanha e anuncia o resultado
  interface · Modelo.test.jsx · treinar pede confirmação, acompanha o andamento e
    anuncia o resultado
```

#### Operações com o banco

```
ok os pesos gravados no banco refazem as previsões gravadas
  api · test_previsao.py · test_os_pesos_gravados_refazem_as_previsoes
ok versões coexistem no banco sobre o mesmo período (UC07-A2)
  api · test_previsao.py · test-versao_que_nao_supera_nao_entra_e_as-versoes_coexistem
ok o índice único parcial recusa dois treinos em andamento
  api · test_previsao.py · test_o_banco_nao_aceita_dois_treinos_em_andamento
ok ler o histórico não faz uma consulta por parceiro
  api · test_previsao.py · test_ler_o_historico_nao_faz_uma_consulta_por_parceiro[60]
ok substituir um período troca as métricas sem duplicar
  api · test_importacao.py · test_substituir_troca_as_metricas_sem_duplicar
```

#### Validações

```
ok histórico curto é recusado dizendo quantos períodos faltam (UC07-E1, RN09)
  api · test_previsao.py · test_historico_curto_recusa_dizendo_quantos_faltam
ok o rótulo de risco é o segmento Em Risco, ponto a ponto (RN09)
  api · test_previsao.py · test_o_rotulo_de_risco_e_o_segmento_em_risco
ok nenhuma amostra de treino enxerga o período de teste (H41)
  modelo · test_variaveis.py ·
    test_nenhuma_amostra_anterior_enxerga_o_periodo_seguinte[teste]
ok importação sem período é recusada (RN03)
  api · test_importacao.py · test_sem_periodo_a_importacao_e_recusada
ok erro de validação sem tradução não vaza a frase em inglês
  api · test_erros.py · test_tipo_ainda_sem_traducao_nao_vaza_a_frase_em_ingles
```

#### Situações de erro

```
ok falha no treino vira FALHOU, com motivo, e libera a trava
  api · test_previsao.py · test_falha_no_treino_vira_falhou_com_motivo_e_libera_a_trava
ok a API que reinicia no meio do treino o marca como falho na subida
  api · test_previsao.py · test_a_subida_da_api_libera_o_treino_interrompido
ok um segundo treino, com outro rodando, é recusado com 409
  api · test_previsao.py · test_com_um_treino_rodando_o_segundo_e_recusado
ok pesos adulterados não executam código ao serem lidos
  modelo · test_treino.py · test_pesos_sao_lidos_sem_executar_codigo
ok falha inesperada responde genérico, com identificador de correlação
  api · test_erros.py · test_falha_inesperada_responde_generico_com_correlacao
```

ok     a recusa da API aparece na tela com a ajuda  
interface · Modelo.test.jsx · a recusa do servidor aparece com a ajuda

## 5.2 As situações de erro, na aplicação no ar

Os testes automáticos cobrem os casos; a transcrição abaixo mostra os mesmos erros contra a aplicação no ar, com o esperado de cada troca ao pé dela. O período-base décadas à frente é o da própria transcrição: ela importa um relatório numa semana sorteada no futuro, para não tocar na base de demonstração, e desfaz tudo no fim.

O analista lê a previsão no cadastro (RF28), mas não troca o modelo que toda a equipe usa (UC07):

```
$ POST /api/modelo/treinos
403 Forbidden
< {
<   "detail": "Seu perfil não permite esta operação."
< }
ok      esperado: 403 com 'Seu perfil não permite esta operação.'
```

Um treino por vez, travado pelo banco (ADR-010). O segundo pedido chega com o primeiro ainda rodando:

```
$ POST /api/modelo/treinos
202 Accepted
< {
<   "id": 17,
<   "situacao": "EM_ANDAMENTO",
<   "autor": "Gestor da Evidência",
<   "iniciado_em": "2026-09-25T04:51:20.744343Z",
<   "concluido_em": null,
<   "periodo_base": {
<     "id": 61,
<     "data_inicio": "2061-02-12",
<     "data_fim": "2061-02-18"
<   },
<   "volume": {
<     "parceiros": null,
<     "periodos": null,
<     "amostras_treino": null,
<     "amostras_validacao": null,
<     "amostras_teste": null
<   },
<   "metricas": {
<     "mape_modelo": null,
<     "mape_ultimo": null,
<     "mape_media_movel": null,
<     "brier_modelo": null,
<     "brier_referencia": null,
<     "calibracao_modelo": null,
<     "calibracao_referencia": null
<   },
<   "curva": [],
<   "segundos": null,
<   "promovido": null,
<   "versao": "rede-17",
<   "versao_em_uso": null,
<   "motivo": null
< }

$ POST /api/modelo/treinos
409 Conflict
< {
<   "detail": {
<     "erro": "Já existe um treino em andamento.",
<     "ajuda": "Acompanhe o treino atual; quando ele terminar, você pode disparar outro.",
<     "em_andamento": 17
<   }
< }
ok      esperado: 409 com 'Já existe um treino em andamento.', 'Acompanhe o treino atual'
```

Um treino que não existe:

```
$ GET /api/modelo/treinos/999999
404 Not Found
< {
<   "detail": "Treino não encontrado."
```

```
< }  
ok    esperado: 404 com 'Treino não encontrado.'
```

Um parceiro recém-chegado, com menos períodos que a janela da RN09: o cadastro não inventa previsão, e diz por quê.

```
$ GET /api/parceiros/585/previsao  
200 OK  
< {  
<  "disponivel": false,  
<  "faturamento_previsto": null,  
<  "probabilidade_queda": null,  
<  "periodo_base": {  
<    "id": 61,  
<    "data_inicio": "2061-02-12",  
<    "data_fim": "2061-02-18"  
<  },  
<  "modelo_versao": "rede-17",  
<  "origem": null,  
<  "gerada_em": null,  
<  "desatualizada": false,  
<  "motivo": "Com 1 período de histórico, ainda não há previsão: são necessários 4.",  
<  "ajuda": "A previsão aparece no primeiro treino depois que o parceiro completar a janela."  
< }  
ok    esperado: 200 com '"disponivel":false', 'ainda não há previsão', 'são necessários 4'
```

**Resultado da transcrição de validações e erros, com as seções das Sprints 03 e 04:** 31 de 31 respostas como esperado.

### 5.3 Os mesmos erros, na tela

Um treino pedido enquanto outro roda — disparado em outra aba — é recusado, e a tela passa a acompanhar o treino que já roda, com o botão desligado. Esta captura achou um defeito: na primeira versão, a tela recusava e seguia oferecendo "Treinar agora" (issue #108, seção 6).



Figura 39 — Treino recusado porque outro já roda: a recusa com a ajuda, e o botão acompanhando o treino em curso.

O Analista lê a previsão no cadastro, mas não treina — e a tela do modelo, aberta pelo endereço, recusa:

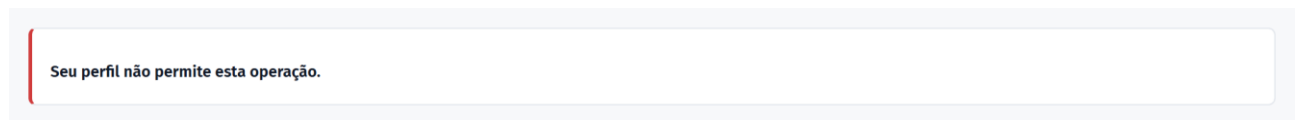


Figura 40 — O Analista na tela do modelo: a rota recusa, e o menu dele nem oferece a tela.

Parceiro com histórico curto demais: o cadastro não inventa previsão, e diz por quê.



Figura 41 — Sem previsão para um parceiro recém-chegado, com o motivo da RN09.

## 6. Bugs identificados e correções

A partir desta sprint, todo defeito vira issue com o rótulo "fix", com o que apareceu, a causa, a correção e como foi encontrado, e é fechado pelo Pull Request que o corrige. O registro abaixo é gerado dessas issues. Os nove defeitos achados entre a entrega da Sprint 04 e o início desta foram registrados assim depois de corrigidos — as issues dizem isso —, para que o registro começasse completo.

| Issue | Defeito                                                                          | Situação  | Correção |
|-------|----------------------------------------------------------------------------------|-----------|----------|
| #97   | O menu do Administrador oferecia Parceiros, que a rota lhe recusa                | Corrigido | #79, #80 |
| #98   | O UC03 negava ao Administrador o histórico que o RF13 lhe dá                     | Corrigido | #79, #83 |
| #99   | Recusas do cadastro de usuário fora do contrato {erro, ajuda}                    | Corrigido | #79      |
| #100  | Abaixo de 860 px, o menu ficava solto no meio de uma faixa vazia                 | Corrigido | #81      |
| #101  | A regra de estilo do histórico de importações não fazia efeito                   | Corrigido | #80      |
| #102  | A Parte IV da entrega lia o mapa vivo e mudaria uma parte já entregue            | Corrigido | #83      |
| #103  | A verificação no ar treinava o modelo a partir de uma semana de teste            | Corrigido | #95      |
| #104  | Mensagens do modelo com "período(s)"                                             | Corrigido | #95      |
| #105  | Probabilidade de 0,998 aparecia como "100%" de chance                            | Corrigido | #96      |
| #106  | O docs/08 lista o modo de execução com valores que o banco não tem               | Corrigido | #107     |
| #108  | Depois da recusa por treino em andamento, a tela seguia oferecendo Treinar agora | Corrigido | #109     |
| #110  | O renderizador de diagramas não achava nenhum bloco com os .md em CRLF           | Corrigido | #111     |

**Resultado:** 12 defeitos registrados — 12 corrigidos, 0 pendentes.

### 6.1 Como foram encontrados

Quase nenhum pelo usuário final: a maior parte veio das ferramentas de verificação, que é para isso que elas existem.

- Capturas de tela em duas larguras e dois temas acharam o menu solto a 768 px, a regra de estilo sem efeito e a probabilidade exibida como certeza.
- As próprias evidências desta entrega acharam a tela que seguia oferecendo treino depois da recusa, e a verificação no ar que treinava a partir de uma semana de teste.
- Revisar a especificação contra o código achou o caso de uso que contradizia o requisito, e o modelo de dados com valores que o banco não tem.

### 6.2 Dois defeitos em detalhe

#### #105 — Probabilidade de 0,998 aparecia como "100%" de chance

Apareceu: "Chance de estar em risco: 100%" — uma estimativa apresentada como certeza. No outro extremo, 172 de 469 parceiros saíam com "0%".

Causa: Arredondamento para inteiro.

Correção: comoProbabilidade: "mais de 99%" e "menos de 1%" nos extremos (#96).

#### #108 — Depois da recusa por treino em andamento, a tela seguia oferecendo Treinar agora

Apareceu: Com um treino já rodando (disparado em outra aba), pedir outro mostra a recusa "Já existe um treino em andamento." — e a tela continua com "Treinar agora" ativo e sem andamento, como se nada estivesse rodando.

Causa: A tela só relia o estado do modelo quando o próprio pedido dava certo. A recusa não disparava a releitura, e o treino em andamento que o servidor conhecia não chegava à tela.

Correção: Depois de qualquer falha no pedido, reler o estado: a tela encontra o treino em andamento e passa a acompanhá-lo, e o aviso de recusa sai quando ele termina.



## 7. Repositório e commits

Repositório: [github.com/PedroMiranda243/gih-fabrica-de-software](https://github.com/PedroMiranda243/gih-fabrica-de-software)

O registro é gerado do histórico do git, lido da branch principal do repositório remoto. A janela da Sprint 04 fecha no dia em que o PDF dela foi gerado, 21/09: o trabalho de 22/09 em diante é o desta entrega.

### Totais

```
-----
commits de trabalho          163
Pull Requests incorporados   42
commits direto na main       22   (até 15/09, antes de a main ser protegida)
```

### Por tipo (Conventional Commits)

```
-----
docs      56   34%
feat      41   25%
test      31   19%
fix       18   11%
ci         8    5%
perf       2    1%
refactor   2    1%
spike      2    1%
build      1    1%
chore      1    1%
fora       1   (fora da convenção:)
2026-09-15 Revert "docs: corta a central de comunicação e torna o assistente...
```

### Sprint 05 – 22/09 a 03/10: 13

```
#79 24/09 A sessão diz as telas do perfil; histórico para o Administrador; recusas de...
#80 24/09 O menu segue o perfil, e o histórico de importações na tela
#81 24/09 A tela dos limiares da segmentação (RF21)
#82 24/09 A tela de usuários (UC02)
#83 24/09 O mapa de navegação com as telas de administração; a Parte IV guarda o mapa...
#84 24/09 A categoria sugerida pelo nome do parceiro (H27, RF15)
#93 24/09 docs: RN09 e ADR-010 – queda prevista, histórico mínimo e onde o treino roda
#94 24/09 feat(modelo): o pacote do modelo preditivo – variáveis, referências, rede e...
#95 24/09 feat: o módulo de previsão pela API – treino, versão em uso e previsão no...
#96 24/09 feat(web): a tela do modelo e a previsão no cadastro do parceiro
#107 25/09 Registros de bugs e de testes gerados de fonte; docs/08 e docs/05 corrigidos
#109 25/09 Geradores das evidências da Sprint 05 – integração com o banco...
#111 25/09 Entrega da Sprint 05 – Parte V, evidências e a Parte II como foi entregue
```

### 7.1 Autoria e coautoria

Como na Sprint 04, o registro mostra a autoria como ela é: todos os commits têm o mesmo integrante como autor, e os demais aparecem como coautores, pela área de responsabilidade de cada um. Os Pull Requests também foram abertos e incorporados pelo mesmo integrante — a revisão cruzada no GitHub, posta como ponto a corrigir na Sprint 04, ainda não acontece, e continua nos próximos passos.

### Autoria e coautoria

```
-----
Cada commit tem um autor e, por área, o colega responsável como coautor
(trailer Co-authored-by, que o GitHub credita ao coautor).
```

```
autor      Pedro Miranda          163
coautor    João Pedro Nunes de França  50
coautor    Marcio Maycom             48
coautor    Ingrid Vitoria de Araújo Barbosa  23
coautor    Thiago José Falcão de Freitas  11
sem coautor                                36
```

## 8. Execução e roteiro de demonstração

O ambiente sobe com um comando, e a base de demonstração é gerada já segmentada e com o modelo treinado:

```
docker compose up -d
python scripts/resetar_banco.py --parceiros 500      # segmenta e treina o modelo
docker compose exec api python -m app.cli treinar-modelo  # o treino, pelo terminal
```

### 8.1 Roteiro para a orientação

| # | Passo                                                                                                                                                     |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Entrar como Gestor e abrir Modelo: a versão em uso, os fatos e a comparação com as referências                                                            |
| 2 | Treinar agora: a confirmação, o andamento e o resultado anunciado; o histórico ganha uma linha                                                            |
| 3 | Abrir um parceiro grande: o desempenho medido, a previsão marcada como estimativa, e a série tracejada                                                    |
| 4 | Abrir um parceiro recém-chegado: sem previsão, e o motivo                                                                                                 |
| 5 | Importar um período novo e voltar a Modelo: o aviso de que há período mais novo que as previsões                                                          |
| 6 | Treinar de novo e abrir um parceiro que está no relatório importado: a previsão parte do período novo — quem não está nele fica sem, e a tela diz por quê |
| 7 | Abrir Modelo em duas abas e treinar nas duas: a recusa, e a segunda aba acompanhando o treino da primeira                                                 |
| 8 | Fechar a aplicação com "docker compose down", subir de novo e mostrar a mesma versão em uso e a mesma previsão                                            |

O roteiro da Sprint 04 continua valendo e foi ensaiado de novo com o módulo novo no ar. O Docker Desktop precisa estar aberto antes, e a verificação de ponta a ponta é a forma mais rápida de conferir que tudo subiu.

## 9. Dificuldades encontradas

### 9.1 Saber se a rede agrega, e não só se ela treina

Uma rede sempre treina e sempre devolve um número. A pergunta que importa é se ela erra menos que uma conta de cabeça — e, com a massa sintética, a vantagem sobre a média móvel saiu pequena. Uma medição de um treino só não diria se ela se sustenta.

**O que foi feito:** a medição usa três redes geradas e cinco sementes de treino em cada uma, com mediana e faixa, nas bases de 500 e de 5.000 parceiros, e está publicada no repositório. A vantagem se sustentou em todos os treinos; o relatório diz que ela é pequena, e que o risco da rede é menos calibrado que a referência.

### 9.2 Treinar sem segurar a requisição, e sem dois treinos ao mesmo tempo

A arquitetura proíbe cálculo pesado dentro da requisição. Tirar o treino dela abre duas portas: dois pedidos ao mesmo tempo gravando versões cruzadas, e um treino interrompido por reinício que ficaria "em andamento" para sempre.

**O que foi feito:** o treino roda em segundo plano, com o estado gravado no banco; um índice único parcial deixa haver só um em andamento — trava do banco, e não da memória do processo —; e a subida da API marca como falho o treino que um reinício interrompeu. Cada caso tem teste.

### 9.3 O PyTorch pesa

A imagem da API foi de 326 MB para 1,6 GB, mesmo com a roda CPU do PyTorch, a menor que existe — a roda padrão traria as bibliotecas do CUDA junto. E importar o PyTorch na subida da API custaria segundos e memória a um processo que, na maior parte do tempo, só serve telas.

**O que foi feito:** a roda CPU, com o tamanho medido e registrado; e o pacote do modelo só carrega o PyTorch quando se treina ou prevê — um teste confere que importar o pacote não o carrega.

### 9.4 Duas execuções da suíte no mesmo banco de teste

Durante o desenvolvimento, a suíte da API foi rodada duas vezes ao mesmo tempo. As duas usam o mesmo banco de teste e esvaziam as tabelas entre um teste e outro — uma apagava os dados da outra, e o resultado não valia nada.

**O que foi feito:** a execução foi descartada e refeita sozinha, e o gerador do registro de testes diz, no cabeçalho, que não pode rodar em paralelo.

### 9.5 Evidência que parte de dado de teste

A verificação de ponta a ponta grava semanas no futuro para não colidir com a base de demonstração. A seção do modelo rodou depois de uma dessas gravações, e o treino partiu de uma semana de teste com dois parceiros: o maior parceiro do período não tinha previsão.

**O que foi feito:** a seção do modelo roda antes de qualquer gravação da verificação (issue #103). A lição vale para as próximas: quando o resultado depende do "período mais recente", a ordem das seções é parte do teste.

## 10. Ajustes no planejamento, na arquitetura e na modelagem

| O que mudou                       | De / para                              | Por quê                                                                                                                              |
|-----------------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| História H45 — retreino pela tela | Sprint 9 interna para a 8              | Sem disparar o treino, o módulo não teria fluxo de uso: o usuário só leria números prontos. A Sprint 9 fica com o otimizador         |
| Pacote do modelo preditivo        | Nenhum para modelo/, próprio           | Prevê e não decide: sem banco, sem FastAPI, testável por linha de comando — o mesmo desenho do núcleo em C++ (decisão ADR-010)       |
| Onde o treino roda                | Requisição para segundo plano          | A arquitetura proíbe cálculo pesado na requisição; fila de tarefas traria um serviço a mais para um treino de segundos (ADR-010)     |
| Onde ficam os pesos               | Nenhum lugar para a linha do treino    | Poucos KB: a versão em uso sobrevive a reinício sem volume de arquivo (ADR-010)                                                      |
| Tabela treino_modelo              | 17 para 18 tabelas                     | O RF27 pede data, volume e métricas de cada treino; a versão em uso fica na própria linha, sem uma segunda fonte que possa discordar |
| Imagem da API                     | Contexto único para contexto adicional | O modelo mora fora da API; levar a raiz inteira para o build traria a interface e a documentação junto                               |
| Integração contínua               | Três para quatro trabalhos             | O pacote do modelo tem suíte própria, e o piso de cobertura do núcleo passou a cobrar a previsão                                     |
| Regras de negócio                 | RN01–RN08 para RN01–RN09               | Seção 4                                                                                                                              |
| Figuras da Parte II               | Diagramas vivos para figuras entregues | Renderizada de novo, a Parte II mudaria uma entrega já feita; a modelagem nova aparece na seção 3 desta parte                        |

### 10.1 As lacunas da Parte IV, fechadas

A Parte IV declarou quatro lacunas em vez de omiti-las. As quatro foram fechadas antes do módulo novo: a tela dos limiares da segmentação, o histórico de importações na interface, a sugestão de categoria pelo nome do parceiro — com a regra escrita antes do código — e a tela de administração de usuários. O menu passou a vir do servidor, com as telas que cada perfil abre.

### 10.2 Planejamento

A equipe segue à frente do calendário interno: a Sprint 8 — modelo preditivo — estava prevista para 26/10 a 30/10. A tabela de equivalência do cronograma foi atualizada: esta entrega corresponde à Sprint 8 interna mais a H45 da 9. A folga continua sendo margem para o núcleo em GPU.

## 11. Próximos passos

| Sprint | Tema                    | O que entra                                                                                                                               |
|--------|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 9 a 11 | Núcleo computacional    | Otimizador serial, paralelo em CPU e paralelo em GPU, alimentado pela previsão e pelo risco deste módulo, com o comparativo de desempenho |
| 12     | Central de comunicação  | Geração de mensagens por segmento, com aprovação humana obrigatória                                                                       |
| 13     | Assistente e fechamento | Perguntas em linguagem natural sobre dados já apurados, e a entrega final                                                                 |

### 11.1 No módulo de previsão

- A coluna de risco na lista de parceiros, com ordenação por ela — a pergunta "em quem investir?" feita à rede inteira de uma vez.
- O risco calibrado de novo à medida que a base crescer: com poucas semanas, a rede separa melhor, mas erra mais a proporção que a taxa observada.
- O aval do Product Owner sobre o limiar de recém-chegado, na issue #59.

### 11.2 No processo

- Revisão cruzada nos Pull Requests: cada um revisado pelo dono da área antes de entrar, e não só pelo autor — ponto posto na Sprint 04 e ainda aberto.
- Todo defeito como issue com o rótulo "fix", desde o primeiro minuto — o registro desta entrega só começou completo porque os anteriores foram registrados depois.

*Projeto acadêmico. © 2026 os autores — todos os direitos reservados. Repositório público significa visível para avaliação e portfólio; não constitui licença de uso, cópia ou redistribuição.*

## Parte VI — Sprint 06: Aprimoramento do Sistema

Esta entrega junta o terceiro módulo aos dois primeiros e cuida do que a Pré-Banca apontou. O terceiro módulo é a Campanha: com a previsão e o risco de cada parceiro, que o segundo módulo calcula, e a posição no ranking, que o primeiro calcula, o sistema decide em quem investir a verba de uma campanha — que ação dar a cada parceiro, dentro do orçamento, do número de ações que a equipe consegue executar e das cotas. É a pergunta "em quem investir?" do problema do produto, e é onde está o núcleo computacional: o mesmo otimizador em Python, em C++, em paralelo na CPU e na GPU.

Cada item vem com execução real: transcrições e capturas geradas por scripts contra a aplicação no ar, com a GPU, e registros de testes, de bugs, de commits e de revisões gerados de fonte. Gerar essas evidências achou três defeitos da própria sprint, corrigidos antes desta entrega — seção 7.

### As cinco entregas, e onde estão

| # | Entrega                                 | Situação                               | Evidência neste documento                                                |
|---|-----------------------------------------|----------------------------------------|--------------------------------------------------------------------------|
| 1 | <b>Correções apontadas na Pré-Banca</b> | Feitas, com duas pendências declaradas | Seção 1 — observação por observação                                      |
| 2 | <b>Terceiro módulo implementado</b>     | Funcionando                            | Seções 2 e 3 — o problema, as quatro versões medidas, as telas e o banco |
| 3 | <b>Integração entre os módulos</b>      | Funcionando                            | Seção 4 — pelo dado e pela tela, conferida contra a aplicação no ar      |
| 4 | <b>Melhorias na interface</b>           | Feitas e medidas                       | Seção 5 — antes e depois, e a medição no navegador                       |
| 5 | <b>Ajustes de navegação</b>             | Feitos                                 | Seção 6 — o menu, o título das abas, a página que não existe e a trilha  |

## 1. Correções apontadas na Pré-Banca

As devolutivas das Sprints 01 a 04, observação por observação: o que foi feito e onde está a evidência. Duas continuam em aberto, e estão escritas como estão, com a medida que falta.

| Sprint | Observação                                                                   | O que foi feito                                                                                                                                                                                                                         | Evidência                                    | Situação                |
|--------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|-------------------------|
| 01     | Reduzir o escopo do MVP                                                      | O escopo foi priorizado, e não cortado: histórias Must, Should e Could, e o assistente como válvula de escape. O código fechou antes do calendário, e a válvula não precisou ser usada                                                  | docs/04; docs/05, risco R2                   | Resolvido de outro modo |
| 01     | Definir os papéis da equipe                                                  | Product Owner, Scrum Master, Frontend, Banco de Dados e Backend/Núcleo, com o dono de cada diretório no CODEOWNERS, que convida o dono a revisar o PR                                                                                   | docs/06; .github/CODEOWNERS                  | Resolvido               |
| 01     | Sprints semanais                                                             | Treze sprints semanais desde 16/09; as histórias de 13 pontos foram quebradas                                                                                                                                                           | docs/05, seção 2                             | Resolvido               |
| 02     | Documentação acompanhando o código                                           | Arquitetura, modelo de dados e classes reescritos como construídos; nesta entrega, cada PR atualizou o documento que mudou — o mapa de navegação, o fluxo entre os módulos                                                              | docs/07 versão 2.1; docs/09                  | Resolvido               |
| 02     | Controlar o escopo; pouca margem para atraso                                 | A construção terminou antes do calendário, e o congelamento foi antecipado de 30/11 para 25/10, para sobrar tempo aos vídeos                                                                                                            | docs/05, riscos R2 e R7                      | Resolvido               |
| 02     | Evidência objetiva de modelo, paralelismo e GPU                              | Medições geradas por script; o mesmo plano nos quatro modos; o benchmark na tela; e, nesta entrega, os testes do otimizador rodando na placa (NVIDIA GeForce RTX 4060), sem pular nenhum                                                | Seções 2, 3 e 7; docs/medicoes               | Resolvido               |
| 03     | Tela de administração de usuários                                            | Feita antes da Sprint 05: criar a conta, mudar o nome e o perfil, desativar e reativar, pela interface                                                                                                                                  | Parte V, seção 10.1                          | Resolvido               |
| 03     | Levar à interface o rigor do backend                                         | O axe em todo teste da interface; as 20 telas medidas no navegador, nos dois temas; e dois defeitos de interface achados pelas evidências e corrigidos (#190, #192)                                                                     | Seções 5 e 7; CLAUDE.md                      | Resolvido               |
| 03     | Ampliar as validações negativas                                              | A transcrição de validações ganhou uma seção por entrega — importação, cadastro, limiares, acesso, modelo e, agora, a campanha                                                                                                          | Seção 7.3 — 39 de 39 respostas como esperado | Resolvido               |
| 03     | Cronograma sincronizado com o antecipado                                     | O cronograma registra o calendário real e a correspondência entre as duas numerações, atualizada a cada entrega                                                                                                                         | docs/05, seção 1                             | Resolvido               |
| 04     | Fechar as lacunas: limiares, histórico de importações, sugestão de categoria | As três foram feitas antes do segundo módulo                                                                                                                                                                                            | Parte V, seção 10.1                          | Resolvido               |
| 04     | Distribuir a autoria dos commits                                             | Cada commit leva como coautor o dono da área, que o GitHub credita. Mas o autor dos 339 commits continua sendo o mesmo integrante                                                                                                       | Seção 8                                      | Em aberto               |
| 04     | Revisão cruzada real dos Pull Requests                                       | Cada PR desta entrega foi aberto convidando o dono de cada área a revisar, e pedindo a aprovação no próprio PR — mas a aprovação foi dada fora da plataforma. Dos 38 PRs desde a entrega anterior, 0 têm aprovação registrada no GitHub | Seção 8 — registro de revisões               | Em aberto               |

**Resultado:** 11 de 13 observações resolvidas; 2 em aberto.

### 1.1 As duas em aberto, e o que falta

- Revisão cruzada. A proteção da branch principal já exige uma aprovação antes do merge, e ela vinha sendo contornada pela permissão de administrador. O que falta é a aprovação no GitHub — na aba "Files changed", "Review changes" e "Approve" —, que é o que o registro mostra. Ele é gerado de novo a cada entrega.

- **Autoria.** Os integrantes passam a commitar com a própria conta o que fizerem até a entrega final: a decisão do limiar de recém-chegado (issue #59, do Product Owner), o registro da validação do README numa máquina limpa (H72, do Scrum Master) e os ajustes pedidos nas revisões.

## 2. O terceiro módulo: Campanha e otimizador

### 2.1 O problema

O gestor tem uma verba e um catálogo de ações comerciais — cupom, destaque na vitrine, campanha de categoria, visita de relacionamento, frete subsidiado —, cada uma com custo e efeito estimado. O sistema escolhe, para cada parceiro, uma ação ou nenhuma, de modo que o ganho esperado da campanha seja o maior possível sem passar de nenhuma restrição. Com centenas de parceiros e cinco ações, as combinações são mais que os átomos do universo: é um problema de otimização combinatória, e é por isso que ele mora no núcleo computacional, e não numa planilha.

| Regra              | O que diz                                                                                                                                           | Onde aparece                                                          |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>RN10</b>        | O ganho de uma ação sai da previsão do parceiro: faturamento previsto $\times$ crescimento, mais previsto $\times$ risco de queda $\times$ retenção | O plano e o cadastro do parceiro, marcados como estimativa            |
| <b>RN11</b>        | Recebe ação quem está ativo e tem previsão da versão em uso; a cauda longa é quem está fora do Top N do ranking                                     | A tela da campanha diz quantos entram e quantos ficam fora, e por quê |
| <b>RN07, RF31</b>  | Campanha que não cabe não vira plano: é recusada antes da busca, dizendo a restrição e quanto falta                                                 | A execução inviável, registrada sem plano                             |
| <b>RF32, RNF06</b> | O cálculo roda no modo escolhido ou no primeiro disponível — GPU, CPU paralela, serial —, e sem GPU o sistema funciona igual                        | O seletor de modo, com o porquê do que falta                          |
| <b>RF33 a RF35</b> | Cada execução fica gravada, com autor, parâmetros, modo, tempo e plano; dois planos se comparam lado a lado                                         | Execuções e comparação                                                |

### 2.2 O mesmo otimizador, em quatro versões

Um algoritmo genético, com os planos gulosos na população inicial, escrito quatro vezes: em Python, que é a referência; em C++; em C++ com OpenMP, nos núcleos do processador; e em CUDA, na placa de vídeo, com a população morando na GPU de uma geração para a outra. Com a mesma semente, as quatro dão o mesmo plano — é o que permite comparar o tempo de uma com o da outra, e é conferido em teste, no benchmark e na tela.

- Contra o ótimo: em recortes pequenos, onde dá para enumerar todos os planos, o genético chegou ao ótimo em 75 de 75.
- Contra a escolha gulosa, que é o que uma planilha faria: empatou em 15 de 27 buscas com 2.000 parceiros e ficou acima nas demais — nunca abaixo, porque os planos gulosos estão na população inicial.

### 2.3 Os números, medidos

Lidos da medição publicada no repositório, no contêiner da API e na placa da equipe. O cenário de referência é o dos requisitos: 2.000 parceiros e 5 ações.

| Medida                                                         | Meta         | Medido                           |
|----------------------------------------------------------------|--------------|----------------------------------|
| <b>RNF01 — a campanha calculada, de ponta a ponta, na GPU</b>  | até 5 s      | 248 ms (223 ms a 287 ms)         |
| <b>RNF02 — ganho da GPU sobre o Python</b>                     | no mínimo 5x | 26,29 s contra 248 ms: 106,0x    |
| <b>RNF02 — o plano da GPU contra o do Python</b>               | dentro de 2% | 0,0% de diferença: o mesmo plano |
| <b>OpenMP, 8 threads, sobre o C++ serial — 2.000 parceiros</b> | —            | 5,9x                             |
| <b>O laço na GPU sobre o OpenMP — 2.000 e 10.000 parceiros</b> | —            | 7,4x e 9,3x                      |
| <b>A busca inteira na GPU sobre o OpenMP — 2.000 e 10.000</b>  | —            | 0,3x e 1,1x                      |

A última linha é a mais honesta da tabela. Antes de qualquer conta, a GPU paga o custo fixo de iniciar o driver e criar o contexto da placa — 188 ms (170 ms a 227 ms) com 2.000 parceiros. O laço na GPU é o mais rápido dos modos, mas, com o contexto, a busca inteira perde do OpenMP até perto de 10.000 parceiros. O resultado está medido e explicado na própria tela do benchmark, e não escondido: é uma conclusão técnica válida sobre quando a GPU compensa.

### 2.4 Números do módulo

| Medida                                                | Valor                          |
|-------------------------------------------------------|--------------------------------|
| Testes da API                                         | 1213, 1208 passando, 5 pulados |
| Testes do otimizador, na placa                        | 277, todos passando            |
| Testes do modelo preditivo                            | 31, todos passando             |
| Testes da interface                                   | 248, todos passando            |
| Verificação de ponta a ponta contra a aplicação no ar | 119 verificações passaram      |
| Operações da API                                      | 64 — eram 37 na Sprint 05      |
| Tabelas no banco                                      | 20 — eram 18 na Sprint 05      |

## 3. O módulo funcionando

### 3.1 Com o que a campanha vai trabalhar

Ao abrir a tela, o gestor vê de onde vêm as previsões, quantos parceiros podem receber ação e quantos ficam fora, e por quê. As restrições são o orçamento, o máximo de ações, a cota mínima da cauda longa e as cotas por categoria. O modo automático diz qual roda nesta instalação — aqui, a GPU.

Restrições

Previsões da rede-1, com dados até 14/09/2026 a 20/09/2026

469 parceiros podem receber ação: ativos e com previsão (RN11). Ficam fora 31 com histórico curto.

Orçamento \*

ex.: 12.000,00

Em reais, com vírgula nos centavos.

Máximo de ações \*

Quantas a equipe consegue executar no período.

Cota mínima da cauda longa (%)

ex.: 30

Das ações, quantas vão para quem está fora do Top 15.

Início da aplicação \*

21/09/2026

Fim da aplicação \*

27/09/2026

Cotas por categoria

Em % do máximo de ações. Só conta a categoria confirmada: 0 elegíveis sem categoria confirmada recebem ação, mas não entram em cota (RN11).

Adicionar cota de categoria

Modo de execução

Automático (GPU)

Sem escolha, roda na GPU, se houver, e senão no CPU paralelo. Nesta instalação: GPU.

Calcular plano

Figura 42 — As restrições da campanha, com os elegíveis e os excluídos (RN11) e o modo de execução (RF32).

| Catálogo de ações        |            |             |          |          | Ganho = previsto × crescimento + previsto × risco × retenção (RN10) |  | Nova ação |
|--------------------------|------------|-------------|----------|----------|---------------------------------------------------------------------|--|-----------|
| AÇÃO                     | CUSTO      | CRESCIMENTO | RETENÇÃO | SITUAÇÃO |                                                                     |  |           |
| Cupom de primeira compra | R\$ 120,00 | 8,0%        | 3,0%     | Ativa    | Editar                                                              |  |           |
| Destaque na vitrine      | R\$ 260,00 | 14,0%       | 5,0%     | Ativa    | Editar                                                              |  |           |
| Campanha de categoria    | R\$ 480,00 | 22,0%       | 8,0%     | Ativa    | Editar                                                              |  |           |
| Visita de relacionamento | R\$ 90,00  | 6,0%        | 30,0%    | Ativa    | Editar                                                              |  |           |
| Frete subsidiado         | R\$ 350,00 | 18,0%       | 12,0%    | Ativa    | Editar                                                              |  |           |

Figura 43 — O catálogo de ações, que só o gestor edita, com a fórmula do ganho (RN10).

### 3.2 Calcular

O cálculo pede confirmação e roda fora da requisição: a tela acompanha o andamento e pode ser fechada. A captura abaixo é do modo serial, em Python, que demora o bastante para o andamento aparecer.

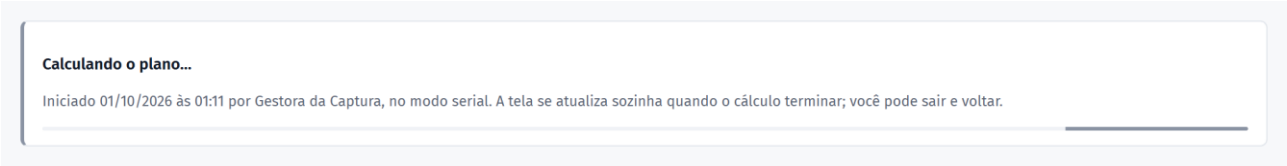


Figura 44 — O cálculo em andamento, no modo serial.

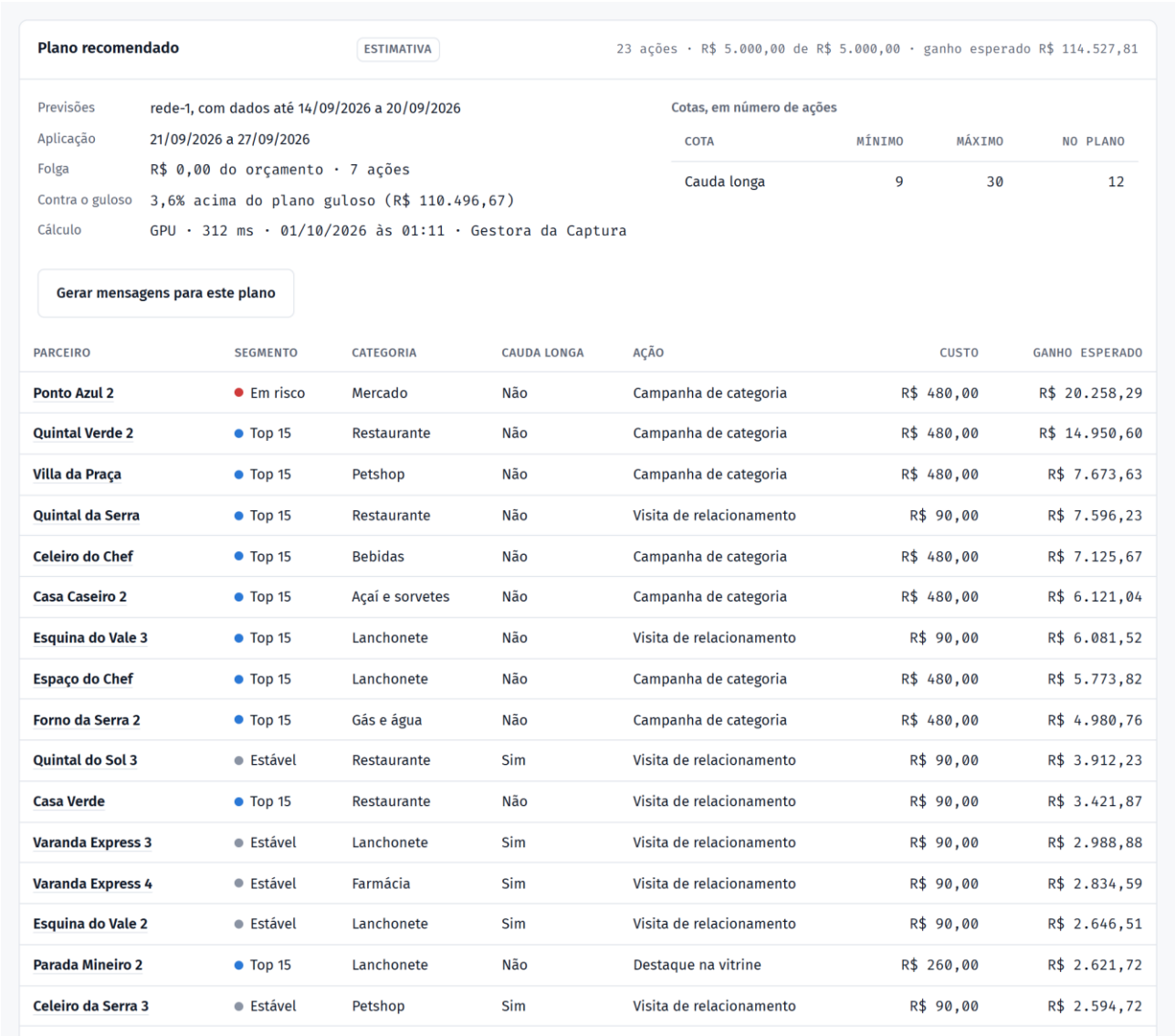


Figura 45 — O plano calculado na GPU: as ações, o custo, o ganho esperado, as cotas e a comparação com o plano guloso.

O mesmo pedido, nos dois modos, pela tela:

O mesmo pedido, calculado pela tela em dois modos – orçamento de R\$ 5.000,00, até 30 ações, 30% delas na cauda longa:  
Serial (Python) execução #120 · 23 ações · custo R\$ 5000.00 · ganho R\$ 114527.81 · 7116 ms  
Automático (GPU) execução #121 · 23 ações · custo R\$ 5000.00 · ganho R\$ 114527.81 · 312 ms  
o mesmo plano, parceiro a parceiro e ação a ação: sim

### 3.3 A campanha que não cabe

Com R\$ 100,00 e 30% das ações na cauda longa, nem a ação mais barata para cada parceiro que as cotas exigem cabe no orçamento. A execução é registrada sem plano, dizendo qual restrição falhou e quanto falta — e não com um plano que desrespeite o orçamento.

Plano recomendado01/10/2026 às 01:11

Sem solução viável. As cotas mínimas exigem pelo menos R\$ 810,00, acima do orçamento de R\$ 100,00: faltam R\$ 710,00.

Mesmo com a ação mais barata para cada parceiro que as cotas exigem, o orçamento não paga. Aumente o orçamento ou reduza as cotas mínimas.

O sistema não entrega plano que viole restrição (RN07).

Cotas, em número de ações

| COTA        | MÍNIMO | MÁXIMO | NO PLANO |
|-------------|--------|--------|----------|
| Cauda longa | 9      | 30     | —        |

Figura 46 — Campanha inviável: sem plano, com a restrição e o que falta (RN07).

O analista consulta a campanha, mas não decide onde vai a verba: no lugar do botão, o porquê.

RestriçõesPrevisões da rede-1, com dados até 14/09/2026 a 20/09/2026

469 parceiros podem receber ação: ativos e com previsão (RN11). Ficam fora 31 com histórico curto.

Orçamento \*

ex.: 12.000,00

Em reais, com vírgula nos centavos.

Máximo de ações \*

Quantas a equipe consegue executar no período.

Cota mínima da cauda longa (%)

ex.: 30

Das ações, quantas vão para quem está fora do Top 15.

Início da aplicação \*

21/09/2026

Fim da aplicação \*

27/09/2026

Cotas por categoria

Em % do máximo de ações. Só conta a categoria confirmada: 0 elegíveis sem categoria confirmada recebem ação, mas não entram em cota (RN11).

Adicionar cota de categoria

Modo de execução

Automático (GPU)

Sem escolha, roda na GPU, se houver, e senão no CPU paralelo. Nesta instalação: GPU.

Só o gestor calcula o plano; o analista consulta.

Figura 47 — A campanha aberta pelo analista.

### 3.4 O histórico e a comparação

| Histórico                                                   |                     |                    |                                                      |        |        | 4 no total                                         |
|-------------------------------------------------------------|---------------------|--------------------|------------------------------------------------------|--------|--------|----------------------------------------------------|
| Marque dois planos calculados para compará-los lado a lado. |                     |                    |                                                      |        |        | <button>Comparar os dois</button>                  |
|                                                             | INICIADA EM         | POR                | PARÂMETROS                                           | MODO   | TEMPO  | RESULTADO                                          |
| <input type="checkbox"/>                                    | 01/10/2026 às 01:11 | Gestora da Captura | R\$ 8.000,00 · até 30 ações · cauda longa $\geq$ 30% | GPU    | 255 ms | 30 ações · ganho de R\$ 132.995,81                 |
|                                                             | 01/10/2026 às 01:11 | Gestora da Captura | R\$ 100,00 · até 30 ações · cauda longa $\geq$ 30%   | GPU    | 18 ms  | Sem solução viável<br>restrição: orçamento         |
| <input type="checkbox"/>                                    | 01/10/2026 às 01:11 | Gestora da Captura | R\$ 5.000,00 · até 30 ações · cauda longa $\geq$ 30% | GPU    | 312 ms | 23 ações · ganho de R\$ 114.527,81                 |
| <input type="checkbox"/>                                    | 01/10/2026 às 01:11 | Gestora da Captura | R\$ 5.000,00 · até 30 ações · cauda longa $\geq$ 30% | Serial | 7,1 s  | 23 ações · ganho de R\$ 114.527,81                 |
| 1-4 de 4                                                    |                     |                    |                                                      |        |        | <button>Anterior</button> <button>Próxima</button> |

Figura 48 — O histórico de execuções: quem, quando, com que parâmetros, em que modo, em quanto tempo e com que resultado (RF34).

| O que foi pedido             |                                                    |                                                    | 1 parâmetro mudou |
|------------------------------|----------------------------------------------------|----------------------------------------------------|-------------------|
| PARÂMETRO                    | PLANO A<br>01/10/2026 às 01:11, Gestora da Captura | PLANO B<br>01/10/2026 às 01:11, Gestora da Captura |                   |
| Orçamento <span>mudou</span> | R\$ 5.000,00                                       | R\$ 8.000,00                                       |                   |
| Máximo de ações              | 30                                                 | 30                                                 |                   |
| Cauda longa                  | ao menos 30% das ações                             | ao menos 30% das ações                             |                   |
| Cotas por categoria          | —                                                  | —                                                  |                   |
| Início da aplicação          | 21/09/2026                                         | 21/09/2026                                         |                   |
| Fim da aplicação             | 27/09/2026                                         | 27/09/2026                                         |                   |
| Modo pedido                  | Automático                                         | Automático                                         |                   |

| O que saiu     |                                                    |                                                    |  | A diferença é do plano B para o plano A |
|----------------|----------------------------------------------------|----------------------------------------------------|--|-----------------------------------------|
| MEDIDA         | PLANO A<br>01/10/2026 às 01:11, Gestora da Captura | PLANO B<br>01/10/2026 às 01:11, Gestora da Captura |  | DIFERENÇA                               |
| Ganho esperado | R\$ 114.527,81                                     | R\$ 132.995,81                                     |  | +R\$ 18.468,00 (+16,13%)                |
| Custo          | R\$ 5.000,00                                       | R\$ 7.810,00                                       |  | +R\$ 2.810,00 (+56,20%)                 |
| Ações          | 23                                                 | 30                                                 |  | +7                                      |

| Parceiro a parceiro |            |                                                   |                                                   |               | 3 mudaram de ação · 1 só no A · 8 só no B · 19 iguais |
|---------------------|------------|---------------------------------------------------|---------------------------------------------------|---------------|-------------------------------------------------------|
| PARCEIRO            | SEGMENTO   | PLANO A                                           | PLANO B                                           | O QUE MUDOU   |                                                       |
| Parada Mineiro 2    | ● Top 15   | Destaque na vitrine<br>ganho de R\$ 2.621,72      | Visita de relacionamento<br>ganho de R\$ 1.586,48 | mudou de ação |                                                       |
| Ponto Central 3     | ● Em risco | Visita de relacionamento<br>ganho de R\$ 1.980,44 | Frete subsidiado<br>ganho de R\$ 2.665,42         | mudou de ação |                                                       |
| Casa Verde          | ● Top 15   | Visita de relacionamento<br>ganho de R\$ 3.421,87 | Campanha de categoria<br>ganho de R\$ 3.982,25    | mudou de ação |                                                       |
| Sabor Express 2     | ● Em risco | Cupom de primeira compra<br>ganho de R\$ 604,44   | sem ação                                          | só no plano A |                                                       |
| Parada Central      | ● Top 15   | sem ação                                          | Campanha de categoria<br>ganho de R\$ 4.098,91    | só no plano B |                                                       |
| Cantina Popular 3   | ● Em risco | sem ação                                          | Campanha de categoria<br>ganho de R\$ 3.488,07    | só no plano B |                                                       |
| Espaço do Vale 2    | ● Top 15   | sem ação                                          | Campanha de categoria<br>ganho de R\$ 3.253,63    | só no plano B |                                                       |
| Parada Bom Prato    | ● Em risco | sem ação                                          | Campanha de categoria<br>ganho de R\$ 3.193,81    | só no plano B |                                                       |

Figura 49 — Dois planos lado a lado: só o orçamento mudou, e a comparação diz o que mudou parceiro a parceiro (RF35).

### 3.5 O benchmark

O mesmo problema sintético, com a mesma semente, em cada modo desta instalação, rodado pela tela. O plano é o mesmo nos quatro, e a tela explica quando a GPU não ganha.



Figura 50 — O benchmark no cenário de referência: tempo, dispersão e ganho de cada modo, e o mesmo plano nos quatro.



Figura 51 — A escalabilidade, em escalas logarítmicas: o tempo de cada modo pelo número de parceiros.

### 3.6 No banco

A execução, o plano e os itens ficam gravados, e é deles que as telas leem. O recorte do modelo de dados que o módulo usa:

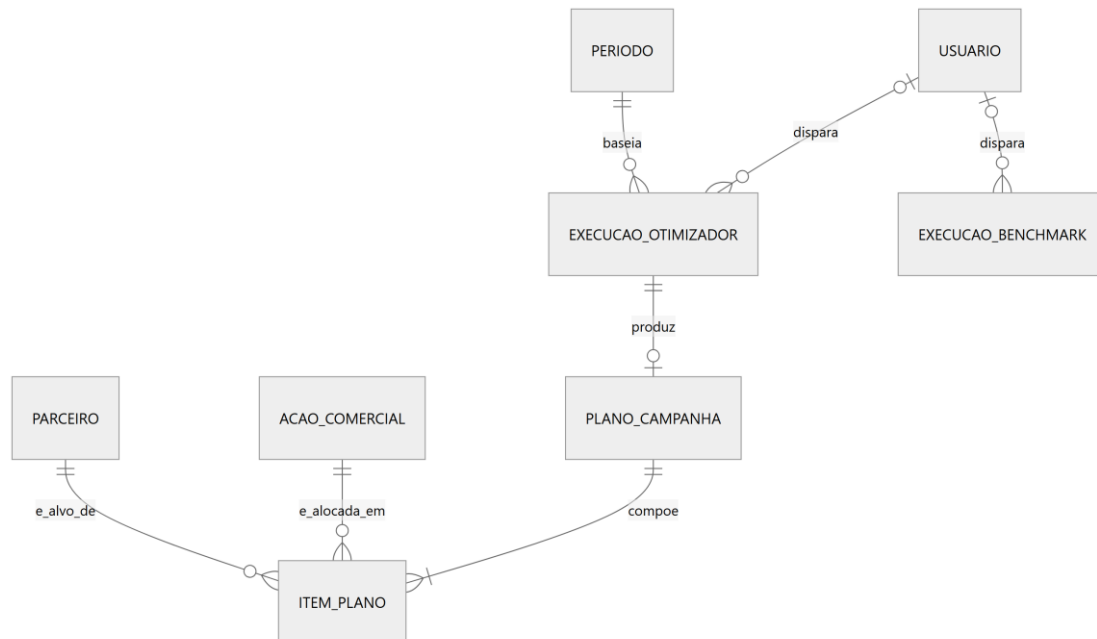


Figura 52 — O recorte do modelo de dados da campanha e do benchmark.

A transcrição da campanha calcula dois planos pela API e termina lendo as linhas do banco por SQL — só leitura —, para mostrar o que ficou gravado de fato:

```
$ SQL, só leitura, no banco da API
> SELECT id, situacao, modo, viavel, custo_total, uplift_total,
> modelo-versao, periodo_base_id
> FROM execucao_otimizador WHERE id IN (:a, :b) ORDER BY id
< id | situacao | modo | viavel | custo | ganho | versao | periodo
< ---+-----+-----+-----+-----+-----+-----+-----
< 87 | CONCLUIDA | GPU | True | 5000.00 | 114527.81 | rede-1 | 12
< 88 | CONCLUIDA | GPU | True | 7810.00 | 132995.81 | rede-1 | 12
ok as duas execuções estão gravadas, com o resultado na própria linha

$ SQL, só leitura, no banco da API
> SELECT p.execucao_id, p.aplicacao_inicio, p.aplicacao_fim,
> count(i.id), sum(i.custo), sum(i.uplift_esperado)
> FROM plano_campanha p JOIN item_plano i ON i.plano_id = p.id
> WHERE p.execucao_id IN (:a, :b)
> GROUP BY p.id ORDER BY p.execucao_id
< execucao | inicio | fim | itens | custo | ganho
< ---+-----+-----+-----+-----+-----
< 87 | 2026-10-05 | 2026-10-11 | 23 | 5000.00 | 114527.81
< 88 | 2026-10-05 | 2026-10-11 | 30 | 7810.00 | 132995.81
ok o plano e os itens de cada execução estão gravados, com o custo total dela
```

**Resultado da transcrição:** 21 de 21 conferências passaram.

### 3.7 Depois de desligar e religar

O teste de persistência passou a incluir a campanha: um plano calculado antes do "docker compose down" volta igual, item a item, depois do "up" — e o cadastro de um parceiro do plano mostra a mesma ação.

```
ok o down removeu os contêineres, e a API parou de responder
ok o up criou contêineres novos — não é o mesmo processo de antes
ok o volume do banco é o mesmo, com a mesma data de criação
ok a sessão aberta antes de desligar continua valendo
ok o parceiro voltou com todos os campos iguais
```

ok a alteração feita antes de desligar continua lá  
ok a base tem o mesmo número de parceiros  
ok o painel mostra os mesmos indicadores  
ok os limiares da segmentação são os mesmos  
ok a versão do modelo em uso é a mesma, com as mesmas métricas  
ok a previsão do parceiro é a mesma, da mesma versão  
ok o plano de campanha voltou igual, item a item – parceiro, ação, custo e ganho  
ok o cadastro do parceiro do plano mostra a mesma ação (H81)

**Resultado:** 13 de 13 conferências passaram.

## 4. A integração entre os módulos

Os três módulos não se chamam: cada um grava no banco o que produz, e o seguinte lê dali. A importação grava as métricas; a segmentação e o ranking saem delas; o treino grava a previsão de cada parceiro; e a campanha lê a previsão da versão em uso e a posição no ranking. Nenhum módulo recalcula o que o anterior gravou — por isso o risco da lista, o do cadastro e o que entra no ganho da campanha são o mesmo número.

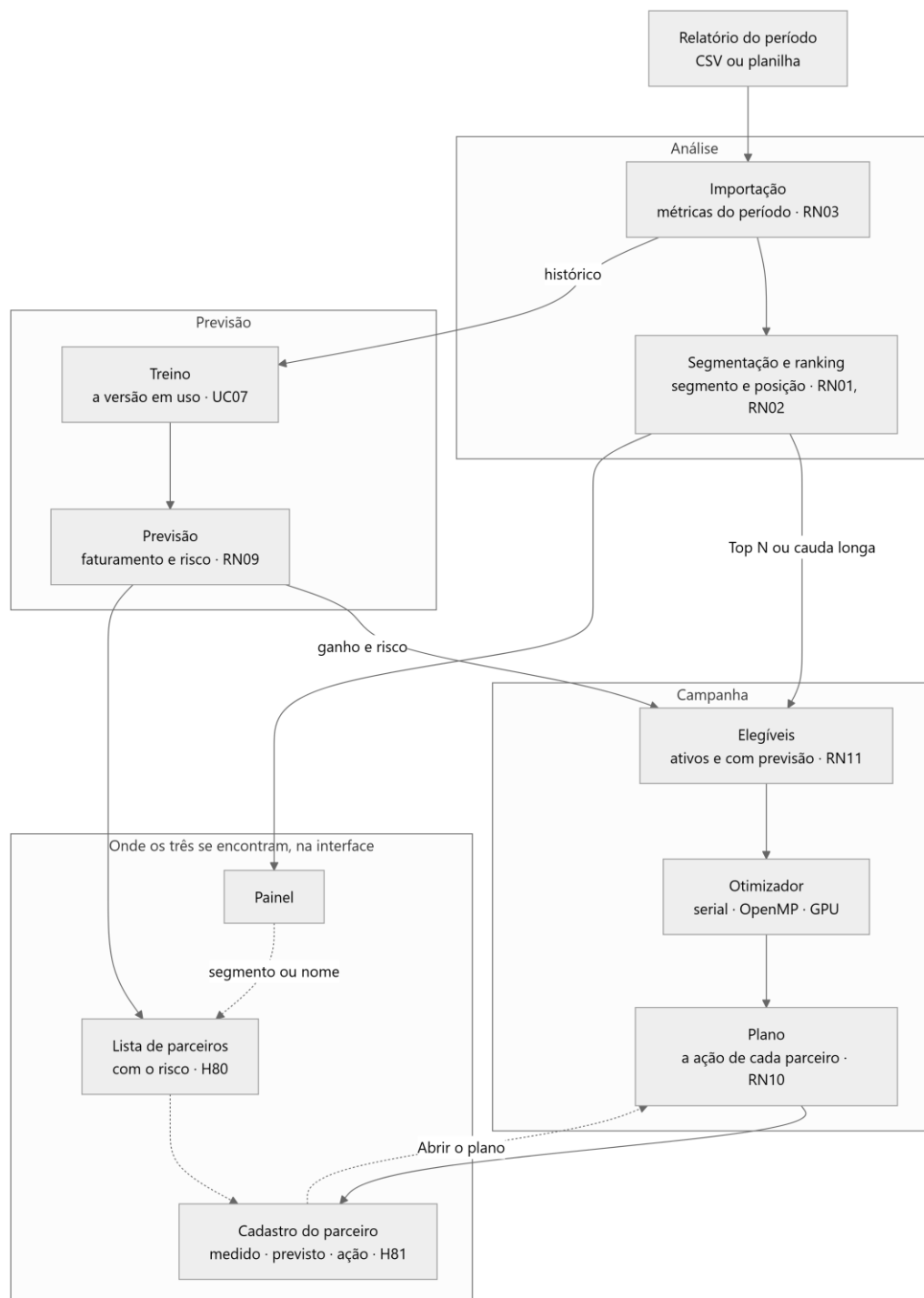


Figura 53 — O dado de um módulo para o outro, e as telas onde os três se encontram. Tracejados, os caminhos de clique.

## 4.1 Pelo dado, conferido

A transcrição da campanha confere a integração contra a aplicação no ar: o plano parte da previsão, a cauda longa é a do ranking do painel, o risco da lista é o da previsão, e o cadastro mostra a ação reservada.

ok o plano saiu da previsão da versão em uso, sobre o mesmo período-base (RN10)  
ok o risco da lista é a probabilidade de queda da previsão  
ok o cadastro mostra a ação, o custo e o ganho que o plano reservou (H81)  
ok a cauda longa do plano é quem está fora do Top 15 do painel (RN11)  
ok quem ficou fora aparece como fora, com o plano de referência

ok o segundo plano terminou viável  
ok o cadastro passa a ler o plano novo – o último concluído e viável  
ok a comparação diz que só o orçamento mudou, sobre as mesmas previsões  
ok as diferenças são do segundo plano para o primeiro  
ok o histórico traz os dois planos, o novo primeiro

## 4.2 Pela tela, por cliques

Do indicador do painel à lista de quem está em risco, ordenada pelo risco que o modelo estimou; dali ao cadastro de um parceiro, com o desempenho medido, a previsão e a ação no último plano; ao plano; e de volta ao parceiro. Ninguém passou pelo menu.

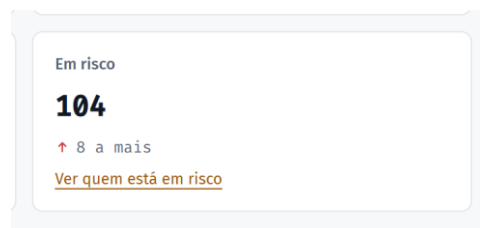


Figura 54 — O indicador do painel, com o caminho para quem está em risco.

Parceiros

104 encontrados

Exportar CSV

Novo parceiro

ESTIMATIVA

O risco de queda é a chance de o parceiro fechar o próximo período em risco, pelo modelo rede-1, com dados até 20/09/2026.

| PARCEIRO            | CATEGORIA   | SEGMENTO   | STATUS | SITUAÇÃO | FATURAMENTO  | PEDIDOS | TICKET MÉDIO | VARIAÇÃO  | RISCO DE QUEDA ↓ |
|---------------------|-------------|------------|--------|----------|--------------|---------|--------------|-----------|------------------|
| Ponto Dourado       | Restaurante | ● Em risco | Ativo  | ● Ativo  | R\$ 849,67   | 17      | R\$ 49,98    | ↓ -12,67% | 65%              |
| Celeiro Azul        | Restaurante | ● Em risco | Ativo  | ● Ativo  | R\$ 697,95   | 12      | R\$ 58,16    | ↓ -12,64% | 63%              |
| Esquina Real        | Restaurante | ● Em risco | Ativo  | ● Ativo  | R\$ 783,77   | 18      | R\$ 43,54    | ↓ -10,57% | 61%              |
| Quintal da Praça    | Mercado     | ● Em risco | Ativo  | ● Ativo  | R\$ 2.575,54 | 29      | R\$ 88,81    | ↓ -8,44%  | 54%              |
| Empório do Vale     | Lanchonete  | ● Em risco | Ativo  | ● Ativo  | R\$ 378,65   | 10      | R\$ 37,87    | ↓ -14,01% | 49%              |
| Cantina da Praça    | Restaurante | ● Em risco | Ativo  | ● Ativo  | R\$ 1.550,94 | 30      | R\$ 51,70    | ↓ -8,19%  | 47%              |
| Recanto do Vale 2   | Restaurante | ● Em risco | Ativo  | ● Ativo  | R\$ 2.966,29 | 51      | R\$ 58,16    | ↓ -10,80% | 46%              |
| Varanda Bom Prato 3 | Pizzaria    | ● Em risco | Ativo  | ● Ativo  | R\$ 2.179,39 | 38      | R\$ 57,35    | ↓ -1,64%  | 46%              |
| Casa Central        | Restaurante | ● Em risco | Ativo  | ● Ativo  | R\$ 2.955,30 | 50      | R\$ 59,11    | ↓ -11,43% | 45%              |
| Esquina do Norte    | Restaurante | ● Em risco | Ativo  | ● Ativo  | R\$ 1.693,59 | 42      | R\$ 40,32    | ↓ -4,98%  | 44%              |
| Parada Bom Prato 3  | Restaurante | ● Em risco | Ativo  | ● Ativo  | R\$ 115,41   | 3       | R\$ 38,47    | ↓ -11,77% | 43%              |

Figura 55 — A lista filtrada pelo segmento Em risco, ordenada pelo risco de queda que o modelo estimou.

**Desempenho**
14/09/2026 a 20/09/2026

Segmento

● Em risco

Faturamento

R\$ 11.593,39

Pedidos

268

Ticket médio

R\$ 43,26

Variação

↓ -11,30%

**Próximo período**

ESTIMATIVA

Faturamento previsto

R\$ 12.008,00

Chance de estar em risco

35%

Com dados até

20/09/2026

Origem

Rede neural · rede-1

Estimativa do modelo, e não medição. A chance é a de o parceiro fechar o próximo período em risco — em queda seguida, pela mesma regra da segmentação (RN09).

**Na campanha**
plano de 30/09/2026 às 18:47

Ação

Visita de relacionamento

Custo

R\$ 90,00

Ganho esperado

R\$ 1.980,44

Aplicação

05/10/2026 a 11/10/2026

O ganho esperado é estimativa, pela previsão rede-1 (RN10).

Abrir o plano

**Figura 56** — O cadastro do parceiro: os três módulos lado a lado — o desempenho medido, a previsão e a ação no plano.

**Execução > Ponto Central 3**

**Figura 57** — Aberto a partir do plano, o cadastro volta para o plano, e não para a lista.

O caminho, passo a passo, como a página o mostrou:

O último plano: execução #106, 23 ações, modelo rede-1.

1. O painel (módulo de análise)  
endereço: /  
título da aba: Painel · GIH  
trilha: (nenhuma)  
a página mostra: Em risco 104 8 a mais em relação ao período anterior Ver quem está em risco
2. Clicou em "Ver quem está em risco" e ordenou pelo risco de queda (análise + previsão)  
endereço: /parceiros?segmento=EM\_RISCO&ordenar\_por=risco&descendente=true  
título da aba: Parceiros · GIH  
trilha: (nenhuma)  
a página mostra: Ponto Dourado Restaurante Em risco Ativo Ativo R\$ 849,67 17 R\$ 49,98 -12,67% 65%
3. Clicou em Ponto Central 3 (previsão + campanha, no cadastro)  
endereço: /parceiros/310  
título da aba: Ponto Central 3 · Parceiros · GIH  
trilha: Parceiros › Ponto Central 3  
a página mostra: Na campanha plano de 30/09/2026 às 18:47 Ação Visita de relacionamento Custo R\$ 90,00 Ganho esperado R\$ 1.980,44 Aplicação 05/10/2026 a 11/10/2026 O ganho esperado é estimativa, pela previsão rede-1 (RN10). Abrir o plano
4. Clicou em "Abrir o plano" (módulo de campanha)  
endereço: /execucoes/106  
título da aba: Execução de 30/09/2026 às 18:47 · Execuções do otimizador · GIH  
trilha: Execuções › 30/09/2026 às 18:47  
a página mostra: Plano recomendado ESTIMATIVA 23 ações · R\$ 5.000,00 de R\$ 5.000,00 · ganho esperado R\$ 114.527,81
5. Clicou em Ponto Central 3, no plano – a trilha volta para a execução  
endereço: /parceiros/310  
título da aba: Ponto Central 3 · Parceiros · GIH  
trilha: Execução › Ponto Central 3  
a página mostra: Na campanha plano de 30/09/2026 às 18:47 Ação Visita de relacionamento Custo R\$ 90,00 Ganho esperado R\$ 1.980,44 Aplicação 05/10/2026 a 11/10/2026 O ganho esperado é estimativa, pela previsão rede-1 (RN10). Abrir o plano

Conferência: o cadastro de Ponto Central 3 mostra a ação que o plano #106 gravou para ele – Visita de relacionamento, R\$ 90,00: sim

## 5. Melhorias na interface

As capturas "antes" foram tiradas da branch principal antes de qualquer mudança, e as "depois", com as melhorias no ar, sobre a mesma base de demonstração.

### 5.1 O risco de queda na lista de parceiros

A lista mostrava o desempenho medido, e o risco só aparecia parceiro a parceiro, no cadastro. A coluna nova traz a chance estimada de cada um, ordenável, com a nota de que é estimativa, de que versão do modelo e até quando ela tem dados; quem não tem previsão vai para o fim, e o leitor de tela ouve o porquê. Ela sai numa consulta só, sem uma por parceiro — há teste para isso —, e também vai para a exportação.

| Parceiros           |             |               |        |          |                 |              |               |           |
|---------------------|-------------|---------------|--------|----------|-----------------|--------------|---------------|-----------|
|                     |             |               |        |          | 500 encontrados | Exportar CSV | Novo parceiro |           |
| PARCEIRO ↑          | CATEGORIA   | SEGMENTO      | STATUS | SITUAÇÃO | FATURAMENTO     | PEDIDOS      | TICKET MÉDIO  | VARIAÇÃO  |
| Cantina Azul        | Lanchonete  | ● Estável     | Ativo  | ● Ativo  | R\$ 5.161,35    | 186          | R\$ 27,75     | ↑ +15,21% |
| Cantina Azul 3      | Padaria     | ● Em ascensão | Ativo  | ● Ativo  | R\$ 11.435,17   | 377          | R\$ 30,33     | ↑ +4,88%  |
| Cantina Bom Prato   | Mercado     | ● Estável     | Ativo  | ● Ativo  | R\$ 4.894,78    | 52           | R\$ 94,13     | ↑ +44,81% |
| Cantina Bom Prato 2 | Padaria     | ● Em ascensão | Ativo  | ● Ativo  | R\$ 2.143,39    | 84           | R\$ 25,52     | ↑ +6,90%  |
| Cantina Bom Prato 3 | Restaurante | ● Em risco    | Ativo  | ● Ativo  | R\$ 1.092,99    | 22           | R\$ 49,68     | ↓ -18,39% |
| Cantina Caseiro     | Mercado     | ● Estável     | Ativo  | ● Ativo  | R\$ 2.818,56    | 41           | R\$ 68,75     | ↓ -3,08%  |
| Cantina Caseiro 3   | Restaurante | ● Estável     | Ativo  | ● Ativo  | R\$ 6.784,10    | 117          | R\$ 57,98     | ↑ +2,13%  |
| Cantina Central     | Mercado     | ● Em ascensão | Ativo  | ● Ativo  | R\$ 729,98      | 8            | R\$ 91,25     | ↑ +9,94%  |
| Cantina Central 2   | Lanchonete  | ● Em risco    | Ativo  | ● Ativo  | R\$ 3.242,68    | 100          | R\$ 32,43     | ↓ -0,92%  |
| Cantina Dourado     | Lanchonete  | ● Estável     | Ativo  | ● Ativo  | R\$ 2.128,26    | 69           | R\$ 30,84     | ↓ -10,44% |
| Cantina Express     | Lanchonete  | ● Estável     | Ativo  | ● Ativo  | R\$ 1.079,48    | 32           | R\$ 33,73     | ↑ +12,07% |
| Cantina Mineiro     | Lanchonete  | ● Em ascensão | Ativo  | ● Ativo  | R\$ 1.527,88    | 56           | R\$ 27,28     | ↑ +1,73%  |
| Cantina Mineiro 2   | Padaria     | ● Em ascensão | Ativo  | ● Ativo  | R\$ 3.553,62    | 110          | R\$ 32,65     | ↑ +16,80% |

Figura 58 — Antes: a lista de parceiros sem o risco.

## Parceiros

500 encontrados

Exportar CSV

Novo parceiro

**ESTIMATIVA** O risco de queda é a chance de o parceiro fechar o próximo período em risco, pelo modelo rede-1, com dados até 20/09/2026.

| PARCEIRO           | CATEGORIA   | SEGMENTO  | STATUS | SITUAÇÃO | FATURAMENTO   | PEDIDOS | TICKET MÉDIO | VARIAÇÃO  | RISCO DE QUEDA ↓ |
|--------------------|-------------|-----------|--------|----------|---------------|---------|--------------|-----------|------------------|
| Varanda do Norte 2 | Padaria     | ● Estável | Ativo  | ● Ativo  | R\$ 955,58    | 34      | R\$ 28,11    | ↓ -9,26%  | mais de 99%      |
| Ponto do Norte     | Restaurante | ● Estável | Ativo  | ● Ativo  | R\$ 2.677,05  | 49      | R\$ 54,63    | ↓ -3,97%  | 95%              |
| Recanto da Praça   | Petshop     | ● Estável | Ativo  | ● Ativo  | R\$ 1.614,27  | 24      | R\$ 67,26    | ↓ -12,02% | 93%              |
| Villa Mineiro 2    | Restaurante | ● Estável | Ativo  | ● Ativo  | R\$ 1.096,66  | 19      | R\$ 57,72    | ↓ -25,87% | 92%              |
| Villa Popular      | Gás e água  | ● Estável | Ativo  | ● Ativo  | R\$ 167,20    | 2       | R\$ 83,60    | ↓ -16,25% | 92%              |
| Quintal da Serra   | Restaurante | ● Top 15  | Ativo  | ● Ativo  | R\$ 23.741,23 | 515     | R\$ 46,10    | ↓ -19,67% | 92%              |
| Sabor da Praça     | Bebidas     | ● Estável | Ativo  | ● Ativo  | R\$ 1.162,44  | 18      | R\$ 64,58    | ↓ -4,85%  | 92%              |
| Parada Azul        | Mercado     | ● Estável | Ativo  | ● Ativo  | R\$ 373,23    | 5       | R\$ 74,65    | ↓ -13,41% | 91%              |
| Varanda Popular 3  | Pizzaria    | ● Estável | Ativo  | ● Ativo  | R\$ 1.478,04  | 20      | R\$ 73,90    | ↓ -9,33%  | 89%              |
| Esquina do Chef    | Lanchonete  | ● Estável | Ativo  | ● Ativo  | R\$ 2.377,98  | 60      | R\$ 39,63    | ↓ -9,30%  | 88%              |
| Parada Real 3      | Padaria     | ● Estável | Ativo  | ● Ativo  | R\$ 1.093,21  | 48      | R\$ 22,78    | ↓ -4,17%  | 88%              |

Figura 59 — Depois: a lista ordenada pelo risco de queda, do maior para o menor, com a nota da estimativa.

## 5.2 O cadastro do parceiro, com os três módulos

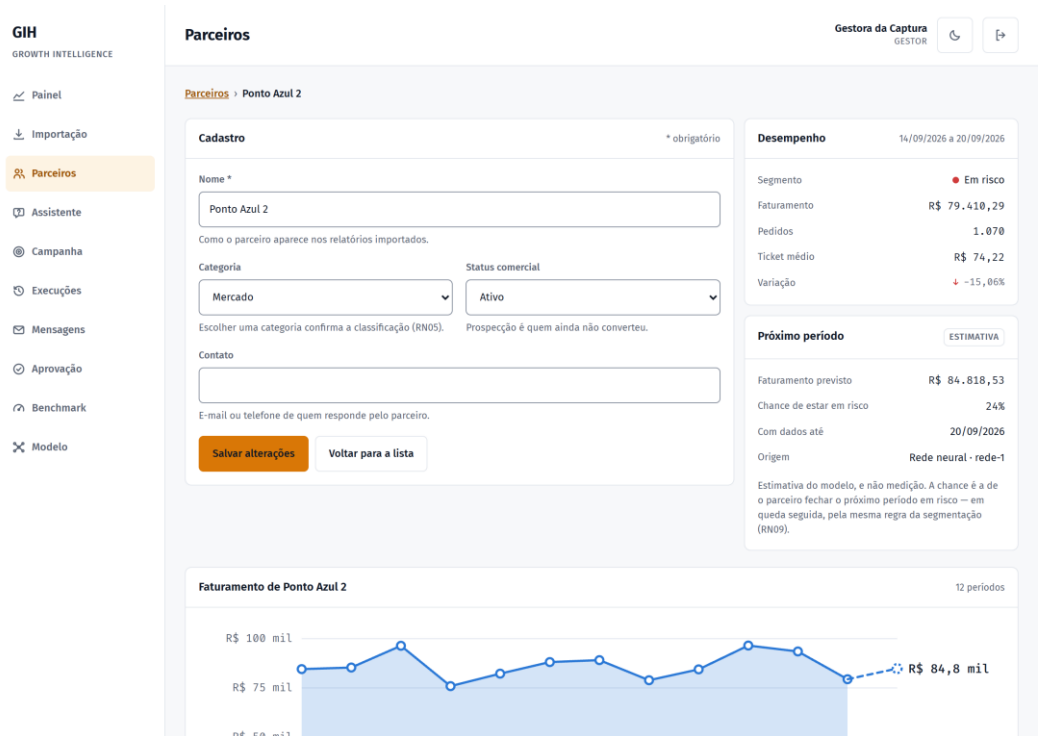


Figura 60 — Antes: o cadastro com o desempenho e a previsão, sem ligação com a campanha.



### 5.3 Do painel para os outros módulos

O ranking e a distribuição por segmento eram só leitura. Agora o nome no ranking abre o cadastro, cada segmento da distribuição abre a lista filtrada por ele, e o indicador de Em risco leva a quem está em risco.

| Ranking de parceiros |                   |                 |            |               |         |              | 25 de 500 |
|----------------------|-------------------|-----------------|------------|---------------|---------|--------------|-----------|
| POSICÃO              | PARCEIRO          | CATEGORIA       | SEGMENTO   | FATURAMENTO   | PEDIDOS | TICKET MÉDIO | VARIAÇÃO  |
| 1                    | Ponto Azul 2      | Mercado         | ● Em risco | R\$ 79.410,29 | 1.070   | R\$ 74,22    | ↓ -15,06% |
| 2                    | Quintal Verde 2   | Restaurante     | ● Top 15   | R\$ 57.841,79 | 1.129   | R\$ 51,23    | ↓ -14,11% |
| 3                    | Villa da Praça    | Petshop         | ● Top 15   | R\$ 31.039,82 | 513     | R\$ 60,51    | ↓ -3,04%  |
| 4                    | Celeiro do Chef   | Bebidas         | ● Top 15   | R\$ 28.667,26 | 382     | R\$ 75,05    | ↓ -4,56%  |
| 5 ↑ 1                | Espaço do Chef    | Lanchonete      | ● Top 15   | R\$ 27.668,36 | 955     | R\$ 28,97    | ↑ +0,90%  |
| 6 ↑ 1                | Esquina do Vale 3 | Lanchonete      | ● Top 15   | R\$ 25.429,22 | 743     | R\$ 34,23    | ↓ -7,03%  |
| 7 ↑ 1                | Casa Caseiro 2    | Açaí e sorvetes | ● Top 15   | R\$ 25.139,17 | 847     | R\$ 29,68    | ↑ +0,84%  |
| 8 ↑ 3                | Forno da Serra 2  | Gás e água      | ● Top 15   | R\$ 24.224,99 | 275     | R\$ 88,09    | ↑ +48,08% |
| 9 ↓ 4                | Quintal da Serra  | Restaurante     | ● Top 15   | R\$ 23.741,23 | 515     | R\$ 46,10    | ↓ -19,67% |
| 10 ↑ 3               | Parada Central    | Farmácia        | ● Top 15   | R\$ 20.358,11 | 370     | R\$ 55,02    | ↑ +34,86% |
| 11 ↓ 2               | Parada Mineiro 2  | Lanchonete      | ● Top 15   | R\$ 19.699,93 | 536     | R\$ 36,75    | ↑ +0,37%  |
| 12 ↑ 3               | Espaço do Vale 2  | Mercado         | ● Top 15   | R\$ 15.527,72 | 210     | R\$ 73,94    | ↑ +9,63%  |
| 13 ↑ 10              | Parada Real       | Padaria         | ● Top 15   | R\$ 15.376,65 | 494     | R\$ 31,13    | ↑ +25,06% |

Figura 62 — Antes: o ranking do painel, sem caminho para o parceiro.

### 5.4 Acessibilidade e largura, medidas

A medição das telas foi refeita depois das mudanças, num navegador de verdade (30/09/2026 23:22): as 20 telas, cada uma a 768 e a 1.440 px e nos dois temas. 19 pares de cor passam no contraste; nenhuma das 40 medidas rola a página na horizontal ou corta conteúdo; e nenhuma das 40 auditorias do axe acusa violação.

A primeira medição refeita não passou: o menu agrupado e a coluna de risco faziam a página rolar a 768 px nas telas do gestor. O defeito foi registrado e corrigido antes desta entrega (#192, seção 7).

## 6. Ajustes de navegação

### 6.1 O menu, organizado pelos módulos

O menu tinha os itens soltos, fora da ordem do fluxo. Agora eles se agrupam pelos módulos, na ordem em que o produto é usado — Análise, Previsão, Otimização, Comunicação e Administração —, e cada perfil vê só os grupos que tem: os itens continuam vindo do servidor, pelas permissões das rotas.



Figura 63 — Antes: o menu do gestor, com os itens soltos.

# GIH

GROWTH INTELLIGENCE

ANÁLISE

 **Painel**

 Importação

 Parceiros

 Assistente

PREVISÃO

 Modelo

OTIMIZAÇÃO

 Campanha

 Execuções

 Benchmark

COMUNICAÇÃO

 Mensagens

 Aprovação

Figura 64 — Depois: o menu do gestor, agrupado pelos módulos.

# GIH

GROWTH INTELLIGENCE

ANÁLISE

 **Painel**

 Importação

 Parceiros

 Assistente

OTIMIZAÇÃO

 Campanha

 Execuções

COMUNICAÇÃO

 Mensagens

 Aprovação

Figura 65 — O menu do analista.

# GIH

GROWTH INTELLIGENCE

ANÁLISE

 **Painel**

 Importação

PREVISÃO

 Modelo

OTIMIZAÇÃO

 Execuções

 Benchmark

ADMINISTRAÇÃO

 Usuários

 Configuração

Figura 66 — O menu do administrador: a administração por último, separada do resto.

GIH

 **Painel**

 Importação

 Parceiros

 Assistente

 Modelo

 Campanha

 Execuções

Figura 67 — A 768 px, o menu vira uma barra que rola na própria caixa, com um fio entre um módulo e outro.

## 6.2 O título da aba

Antes, as 16 abas visitadas diziam o mesmo: "Growth Intelligence Hub". Com várias abas abertas, não havia como saber qual era qual. Agora cada aba diz a tela e, nas de detalhe, o item aberto primeiro:

| Endereço                          | Título da aba, depois                                           |
|-----------------------------------|-----------------------------------------------------------------|
| /                                 | Painel · GIH                                                    |
| /importacao                       | Importação de relatório · GIH                                   |
| /parceiros                        | Parceiros · GIH                                                 |
| /parceiros/{id}                   | Ponto Azul 2 · Parceiros · GIH                                  |
| /modelo                           | Modelo preditivo · GIH                                          |
| /campanha                         | Campanha · GIH                                                  |
| /execucoes                        | Execuções do otimizador · GIH                                   |
| /execucoes/{id}                   | Execução de 30/09/2026 às 18:47 · Execuções do otimizador · GIH |
| /execucoes/comparar?a={id}&b={id} | Comparação · Execuções do otimizador · GIH                      |
| /benchmark                        | Benchmark · GIH                                                 |
| /mensagens                        | Mensagens · GIH                                                 |
| /aprovacao                        | Aprovação · GIH                                                 |
| /pagina-que-nao-existe            | Página não encontrada · GIH                                     |
| /usuarios                         | Usuários · GIH                                                  |
| /usuarios/{id}                    | Gestora da Captura · Usuários · GIH                             |
| /configuracao                     | Configuração · GIH                                              |

Lidos da própria página pela captura, em cada endereço — o título fica fora de qualquer captura de tela.

6.3 O endereço que não existe

Antes, um endereço inválido levava ao painel sem aviso — o registro diz que a página parou em "/" (redirecionada)". Agora ela fica no endereço pedido e diz que a página não existe, mostra o que foi pedido e oferece a volta, com o menu no lugar — e a aba diz "Página não encontrada · GIH".

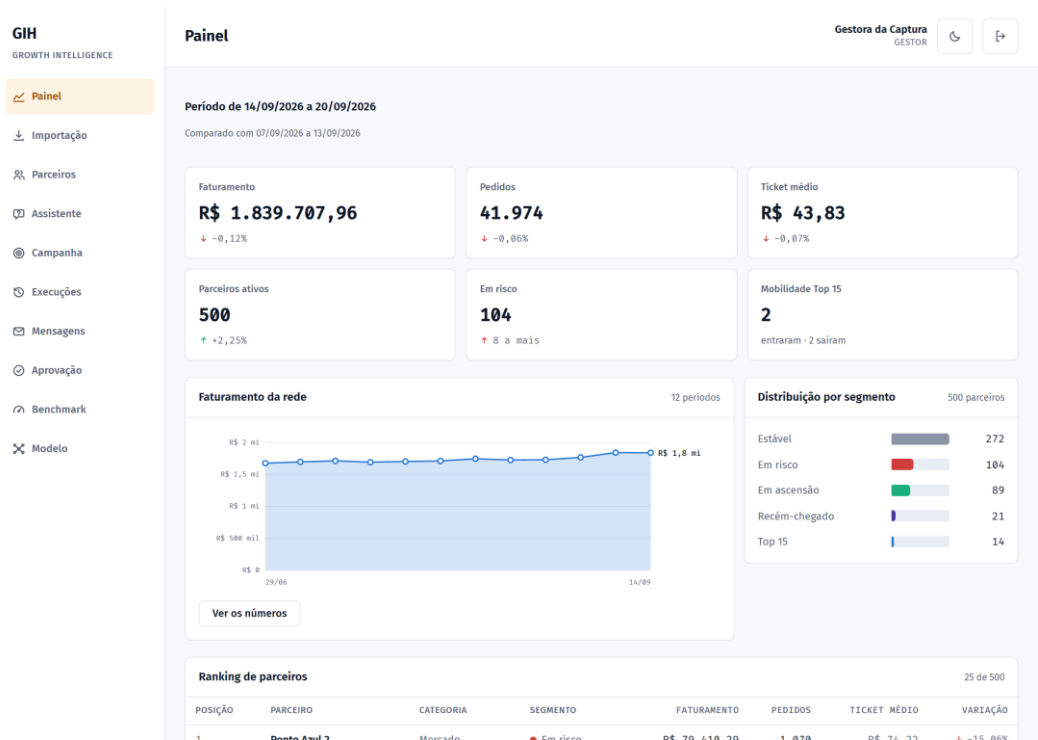


Figura 68 — Antes: o endereço inválido caía no painel, sem aviso.

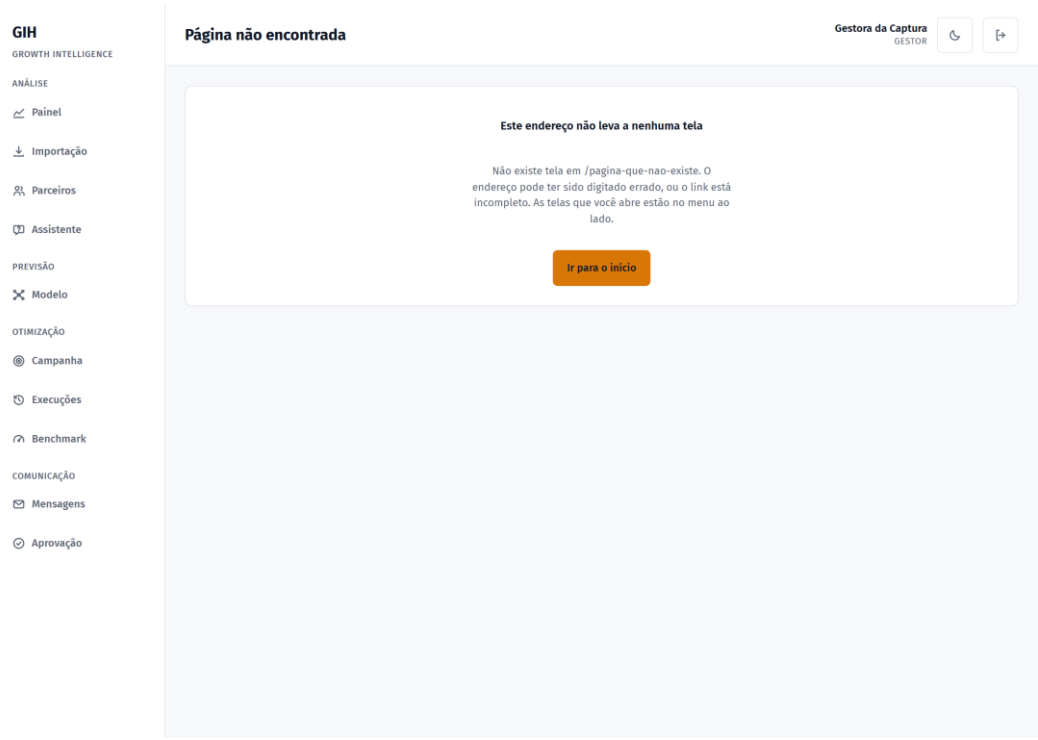


Figura 69 — Depois: a página não encontrada, com o endereço pedido e os caminhos de volta.

## 6.4 A trilha, de volta para de onde se veio

As telas de detalhe já tinham trilha, e o registro "antes" mostra isso. O que mudou é para onde ela volta: o cadastro aberto do painel, de um plano ou da comparação volta para lá, e não para a lista de parceiros, que a pessoa nem tinha aberto.

A screenshot of a breadcrumb navigation bar. It contains the text 'Painel' in a bold, orange font, followed by a right-pointing chevron '>' in a light blue font, and then 'Ponto Azul 2' in a bold, dark blue font. The entire breadcrumb is set against a light gray background.

**Painel** > **Ponto Azul 2**

Figura 70 — O cadastro aberto pelo ranking do painel: a trilha volta para o painel.

## 6.5 O mapa de navegação

O mapa completo, com todas as telas, os grupos do menu e os caminhos entre os módulos. Ele é largo demais para ser lido impresso numa página: a imagem está em alta resolução, para ampliar no PDF, e a versão vetorial está no repositório, em docs/09-sistema-visual.md.

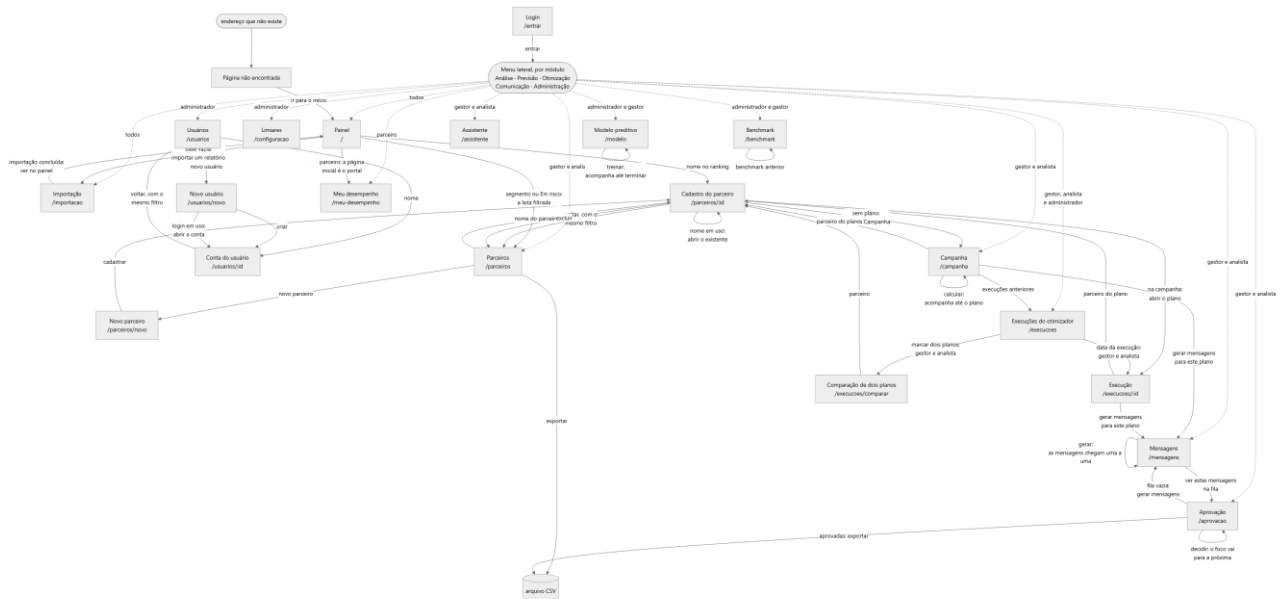


Figura 71 — O mapa de navegação, com os grupos do menu e os caminhos entre os módulos.

## 7. Testes e bugs

### 7.1 As quatro suítes

O registro foi gerado rodando as suítes, e não escrito à mão. A do otimizador passou a rodar na imagem do núcleo compilado com CUDA, com a placa da máquina — NVIDIA GeForce RTX 4060, capacidade 8.9, 8187 MB. Na máquina de desenvolvimento, sem CUDA, os testes da GPU pulariam.

| Suíte                        | Testes | Passaram | Falharam | Pulados | Tempo |
|------------------------------|--------|----------|----------|---------|-------|
| API (pytest, Postgres)       | 1213   | 1208     | 0        | 5       | 381 s |
| Modelo preditivo (pytest)    | 31     | 31       | 0        | 0       | 5 s   |
| Otimizador (pytest, com GPU) | 277    | 277      | 0        | 0       | 104 s |
| Interface (Vitest)           | 248    | 248      | 0        | 0       | 15 s  |

**Resultado:** 1769 testes, 1764 passaram, 0 falharam, 5 pulados.

Os pulados são da suíte da API: os testes contra o modelo de linguagem de verdade, que pulam quando ele não está no ar. O assistente fica para as próximas entregas, e a medição dele contra o modelo está publicada à parte (docs/medicoes/assistente.md).

### 7.2 Os quatro tipos de teste, com exemplos

#### Fluxos principais

- ok o plano respeita as restrições e é o ótimo da instância (UC08, RF30)  
api · test\_campanha.py · test\_o\_plano\_respeita\_as\_restricoes\_e\_e\_o\_otimo
- ok a busca acerta onde a escolha gulosa erra  
otimizador · test\_otimizador.py · test\_o\_guloso\_erra\_ou\_o\_genetico\_acerta
- ok a busca na GPU dá o mesmo plano do Python, numa instância grande (H54)  
otimizador · test\_gpu.py · test\_a\_busca\_na\_gpu\_e\_a\_do\_python\_numa\_instancia\_grande
- ok a lista de parceiros traz o risco da versão em uso, e de onde ele vem (H80)  
api · test\_parceiros\_risco.py ·  
test\_a\_lista\_traz\_o\_risco\_da-versao-em-uso-e-diz-de-oudele-vem
- ok o cadastro do parceiro mostra a ação do último plano (H81)  
api · test\_parceiros\_campanha.py ·  
test\_o\_parceiro\_no\_plano\_ve\_a-acao-o-custo-e-o-ganho-esperado
- ok a tela calcula, acompanha e mostra o plano  
interface · Campanha.test.jsx · calcular manda frações e reais no formato da API,  
acompanha e mostra o plano
- ok o menu agrupado por módulo, na ordem do fluxo (H79)  
interface · Casca.test.jsx · os grupos seguem o fluxo do produto, e cada um é uma  
região com o nome dele

#### Operações com o banco

- ok a execução fica gravada e auditada, com parâmetros, modo e tempo  
api · test\_campanha.py · test\_a-execucao-fica-na-auditoria-com-parametros-modo-e-tempo
- ok o histórico de execuções, da mais recente à mais antiga (RF34)  
api · test\_campanha.py · test\_o\_historico\_lista\_da-mais-recente-para-a-mais-antiga
- ok dois planos gravados, lado a lado, com o que mudou (RF35)  
api · test\_campanha.py · test\_dois\_planos\_lado\_a\_lado-com-o-que-mudou
- ok o cadastro lê o plano mais recente gravado  
api · test\_parceiros\_campanha.py · test\_vale\_o\_plano-mais-recente
- ok o risco na lista não faz uma consulta por parceiro  
api · test\_parceiros\_risco.py · test\_o\_risco-nao-custa-uma-consulta-por-linha

#### Validações

- ok campanha inviável diz a restrição e quanto falta (RN07)  
api · test\_campanha.py · test\_campanha\_inviavel-diz-quanto-falta
- ok mínimos que o orçamento não paga são inviáveis antes da busca (RF31)  
otimizador · test\_viabilidade.py ·  
test\_minimos\_que\_o-orcamento-nao-paga-nem-com-a-acao-mais-barata
- ok cota para uma categoria que não existe é recusada  
api · test\_campanha.py · test\_cota-de-categoria-que-nao-existe
- ok orçamento zero diz o mínimo, na língua de quem usa  
api · test\_erros.py · test\_orcamento\_zero-na-campanha-diz-o-minimo
- ok o erro de cada campo aparece embaixo dele  
interface · Campanha.test.jsx · os erros de campo da API aparecem embaixo do campo

#### Situações de erro

ok falha no cálculo vira FALHOU, com motivo, e libera a trava  
api · test\_campanha.py · test\_falha\_vira\_falhou\_com\_motivo\_e\_libera\_a\_trava  
ok uma segunda otimização, com outra rodando, é recusada com 409  
api · test\_campanha.py · test\_com\_uma\_otimizacao\_rodando\_a\_segunda\_e\_recusada  
ok o núcleo que não responde deixa só o serial, e a campanha segue (RNF06)  
api · test\_campanha.py · test\_nucleo\_que\_nao\_responde\_deixa\_so\_o\_serial  
ok o limite de tempo devolve o melhor plano viável, marcado como parcial  
otimizador · test\_otimizador.py ·  
test\_o\_limite\_de\_tempo\_devolve\_o\_melhor\_viavel\_e\_marca\_parcial  
ok endereço que não existe mostra a página não encontrada, com a volta (H79)  
interface · NaoEncontrada.test.jsx · diz que a página não existe, mostra o endereço  
pedido e oferece a volta

### 7.3 As validações negativas da campanha

Provocadas contra a aplicação no ar, com o esperado de cada uma conferido contra o que voltou. Cada campo recusado diz o limite, na língua de quem usa:

| Caso                                           | Esperado, e o que voltou                                                            |
|------------------------------------------------|-------------------------------------------------------------------------------------|
| O analista pede o cálculo                      | 403 com "Seu perfil não permite esta operação."                                     |
| Orçamento zero                                 | 422 com "orcamento", "Use um valor maior que 0."                                    |
| Máximo de ações zero                           | 422 com "maximo_acoes", "Use um valor a partir de 1."                               |
| Cota da cauda longa acima de 100%              | 422 com "cota_cauda_longa", "Use um valor até 1."                                   |
| Fim da aplicação antes do início               | 422 com "O fim da aplicação não pode ser anterior ao início."                       |
| Cota para categoria que não existe             | 422 com "Há cota para uma categoria que não existe."                                |
| Orçamento de R\$ 100,00 com 30% na cauda longa | 200 com ""viavel":false", ""restricao_violada":"orcamento"", "faltam", ""itens":[]" |
| Execução que não existe                        | 404 com "Otimização não encontrada."                                                |

**Resultado da transcrição de validações, com as seções das Sprints 03 a 05:** 39 de 39 respostas como esperado.

### 7.4 Os bugs desta entrega

Todo defeito vira issue com o rótulo "fix" — o que apareceu, a causa, a correção e como foi encontrado — e é fechado pelo Pull Request que o corrige. O registro é gerado dessas issues: as abertas desde o PDF da Sprint 05, fora as que o registro dela já tinha.

| Issue | Defeito                                                                                              | Situação  | Correção |
|-------|------------------------------------------------------------------------------------------------------|-----------|----------|
| #113  | O treino grava como hora de conclusão a hora em que começou                                          | Corrigido | #114     |
| #115  | Recrutar a base com a API no ar deixa erro interno nas primeiras requisições, e nenhum administrador | Corrigido | #116     |
| #189  | Orçamento zero na campanha respondia "Valor inválido para este campo.", sem dizer o mínimo           | Corrigido | #191     |
| #190  | A tela da Campanha oferecia o cálculo ao analista e escondia do gestor a edição do catálogo          | Corrigido | #191     |
| #192  | A 768 px, a página rolava na horizontal: o texto para leitor de tela escapava das caixas que rolam   | Corrigido | #193     |

**Resultado:** 5 defeitos registrados — 5 corrigidos, 0 pendentes.

### 7.5 Três defeitos que só as evidências pegaram

Todos foram corrigidos. Três vieram de gerar as evidências desta entrega, e nenhum deles os testes automáticos pegavam:

#### #190 — A tela da Campanha oferecia o cálculo ao analista e escondia do gestor a edição do catálogo

Apareceu: Na captura da tela da Campanha aberta pelo analista, o analista via o botão Calcular plano e o seletor de modo — e o clique recebia 403. Do outro lado, o gestor não via os botões Nova ação e Editar do catálogo: o catálogo aparecia só para leitura para todo mundo.

Causa: A tela segue a regra 2.4: quem pode o quê vem da API, em `pode_executar` e `pode_editar_catalogo` de `GET /api/campanha`. A API não mandava `pode_editar_catalogo` — o campo chegava indefinido, e a tela o lia como falso —, e calculava `pode_executar` só pelo estado do sistema (modelo treinado, catálogo com ação ativa, execução em andamento), sem olhar o perfil de quem pergunta. Os testes da tela passavam porque simulam uma API que manda os dois campos; nenhum teste da API conferia o estado pelo perfil.

Correção: `GET /api/campanha` passa a decidir pelo perfil de quem abre a tela: só o gestor calcula e edita o catálogo. Ao analista, `pode_executar` falso com o porquê ("Só o gestor calcula o plano; o analista consulta.") e `pode_editar_catalogo` falso. Testes da API conferem o estado para os dois perfis.

## #192 — A 768 px, a página rolava na horizontal: o texto para leitor de tela escapava das caixas que rolam

Apareceu: A medição das telas (`scripts/medir_telas.py`, H76) passou de 0 para 14 das 40 medidas com a página rolando na horizontal: todas as telas do gestor a 768 px. As do administrador e a do parceiro continuaram sem rolar. Na tela, a página inteira desliza para o lado, e não só a barra do menu ou a tabela — o que o docs/09 proíbe (RNF21: a rolagem é da caixa, nunca da página).

Causa: O texto "só para leitor de tela" (`.so-leitor`) é `position: absolute`, 1 px, recortado. Sem um ancestral posicionado, ele se posiciona em relação à página — e, dentro de uma caixa que rola na horizontal, escapa dela e alarga a página. Duas entradas da Sprint 06 puseram texto assim no fim de caixas que rolam: - o menu agrupado (H79) esconde o título de cada grupo abaixo de 860 px. Na barra do gestor, que tem mais itens, os últimos grupos ficam além dos 768 px, dentro da rolagem da barra; os títulos deles, presos à página, a alargavam. O menu do administrador cabe na largura, e por isso as telas dele passavam; - a coluna de risco (H80), a última da lista de parceiros, diz "sem previsão" ao leitor de tela; dentro da tabela, que rola na própria caixa, o texto escapava do mesmo jeito. Os testes da interface não pegam: o `jsdom` não desenha. Foi a medição no navegador que pegou.

Correção: O trilho, abaixo de 860 px, e a caixa da tabela larga (`.tabela-rolagem`) ganham `position: relative`: o texto escondido passa a pertencer à rolagem da caixa, e a página volta a não rolar. A medição volta a 0 das 40.

## #189 — Orçamento zero na campanha respondia "Valor inválido para este campo.", sem dizer o mínimo

Apareceu: `POST /api/otimizacoes` com `"orcamento": "0"` respondeu 422 com a frase genérica "Valor inválido para este campo." — sem dizer que o orçamento precisa ser maior que zero. É essa frase que a tela da Campanha mostra embaixo do campo.

Causa: A tradução das mensagens de validação (`api/app/erros.py`) cobria os limites inclusivos do `Pydantic` (`greater_than_equal`, `less_than_equal`), mas não os estritos (`greater_than`, `less_than`) nem os de número decimal (`decimal_max_digits`, `decimal_max_places`). O orçamento é `gt=0`: o tipo `greater_than` caía na frase genérica, que existe para não vaziar o texto em inglês — e ela não diz o limite (RNF20).

Correção: Os quatro tipos entram nas duas tabelas, com o limite na frase: "Use um valor maior que 0.", "Use um valor menor que 1.", "Número grande demais: use no máximo 12 dígitos." e "Use no máximo 2 casas decimais.". Um teste parametrizado confere as quatro, e outro o caso que achou a lacuna, pela rota da campanha (`api/tests/test_erros.py`).

## 8. Repositório, commits e revisões

Repositório: [github.com/PedroMiranda243/gih-fabrica-de-software](https://github.com/PedroMiranda243/gih-fabrica-de-software)

Gerado do histórico do git, lido da branch principal do repositório remoto. A janela da Sprint 05 fecha no Pull Request do PDF dela (#112), e não num dia: no mesmo 25/09, depois do PDF, já entrou trabalho desta entrega.

### Totais

```
-----
commits de trabalho          339
Pull Requests incorporados   81
commits direto na main       22   (até 15/09, antes de a main ser protegida)
```

Sprint 06 – 25/09, depois do PDF, a 17/10: 38

- #114 25/09 A conclusão do treino é gravada quando ele termina
- #116 25/09 O reset recria o administrador e reinicia a API
- #124 25/09 RN10 e RN11, a formalização do otimizador e a ADR-011
- #125 25/09 O otimizador serial em Python e a recusa de campanha inviável
- #126 25/09 A campanha na API: catálogo, cálculo do plano e recusa com o que falta
- #127 25/09 A medição do otimizador serial: contra o guloso, o teto e a enumeração
- #128 25/09 A tela de campanha: restrições, plano recomendado e catálogo
- #129 25/09 A campanha nos documentos vivos: dados, classes, casos de uso e telas
- #130 26/09 Spike: o núcleo C++/CUDA dentro da imagem da API, com a GPU (ADR-012)
- #137 26/09 H53a: o genético em C++, idêntico ao Python, chamado por processo
- #138 26/09 H53b: o genético com OpenMP, com o mesmo plano da versão serial
- #139 26/09 H55: a campanha escolhe o modo de execução, e o automático é o mais rápido
- #140 26/09 H58: o histórico de execuções do otimizador
- #147 26/09 H54a: as estruturas de dados na GPU e a transferência, conferida
- #148 26/09 H56: sem GPU o sistema funciona igual, e diz por quê
- #149 26/09 H54b: o kernel CUDA que avalia a população, igual à CPU
- #150 26/09 H54c: o genético inteiro na GPU, com o mesmo plano
- #151 27/09 H57: o benchmark na tela, com as quatro colunas
- #152 27/09 H70: testes de segurança derivados da aplicação, e dois vetores fechados
- #153 27/09 H59: dois planos lado a lado, com o que mudou
- #169 27/09 ADR-013: o modelo de linguagem local, o redator e a guarda numérica
- #170 27/09 H60 (API): gerar mensagens, com o modelo fixo quando o modelo falha
- #171 27/09 H60 (tela): gerar mensagens, com a prévia e as mensagens chegando uma a uma
- #172 27/09 H61-H63 (API): a fila de aprovação, e a decisão só do gestor
- #173 27/09 H61-H63 (tela): a fila de aprovação, com a decisão do gestor
- #174 27/09 H64: o histórico das mensagens decididas, e a exportação das aprovadas
- #175 27/09 H39: o portal do Parceiro, só com o próprio histórico
- #176 27/09 H65: o assistente com o catálogo fechado de perguntas
- #177 28/09 H66, H67, H68: a fonte, a guarda e a abstenção do assistente
- #178 28/09 H65 a H68: a tela do assistente
- #179 28/09 H76: a acessibilidade e a responsividade medidas
- #180 28/09 H72: o README que um avaliador segue sozinho
- #181 28/09 H73: a documentação técnica final consolidada, e o cronograma real
- #186 30/09 H79: a navegação por módulo, o título de cada aba e a página "não encontrada"
- #187 30/09 H80: o risco de queda na lista de parceiros, ordenável e no CSV
- #188 30/09 H81: os módulos ligados pelos próprios dados – do painel ao cadastro, do...
- #191 30/09 Evidências da Sprint 06: o terceiro módulo, a integração e a navegação, e...
- #193 01/10 A 768 px, a página não rola mais na horizontal (#192)

## 8.1 Autoria e coautoria

Os 339 commits têm o mesmo autor; os demais integrantes aparecem como coautores, pela área de cada um (João 111, Marcio 98, Ingryd 45, Thiago 22). É a pendência da seção 1: a coautoria credita, mas não distribui a autoria.

### Autoria e coautoria

Cada commit tem um autor e, por área, o colega responsável como coautor (trailer Co-authored-by, que o GitHub credita ao coautor).

|             |                                  |     |
|-------------|----------------------------------|-----|
| autor       | Pedro Miranda                    | 339 |
| coautor     | João Pedro Nunes de França       | 111 |
| coautor     | Marcio Maycom                    | 98  |
| coautor     | Ingryd Vitoria de Araújo Barbosa | 45  |
| coautor     | Thiago José Falcão de Freitas    | 22  |
| sem coautor |                                  | 74  |

## 8.2 Revisões

O registro novo lista, para cada Pull Request incorporado desde o PDF da entrega anterior, quem abriu, quem foi convidado a revisar, quem revisou e com que resultado, e quem incorporou. Ele mostra a revisão como o GitHub a registrou.

### Resumo

|                                         |    |
|-----------------------------------------|----|
| PRs incorporados                        | 38 |
| com aprovação de um colega              | 0  |
| com revisão de um colega, sem aprovação | 0  |
| sem revisão registrada no GitHub        | 38 |

### Por colega convidado

|                |              |    |           |   |                  |   |            |   |
|----------------|--------------|----|-----------|---|------------------|---|------------|---|
| mihaeldataoman | convidado em | 34 | · aprovou | 0 | · pediu mudanças | 0 | · comentou | 0 |
| ThiagojFalcao  | convidado em | 22 | · aprovou | 0 | · pediu mudanças | 0 | · comentou | 0 |
| ingrydaraujob  | convidado em | 17 | · aprovou | 0 | · pediu mudanças | 0 | · comentou | 0 |
| joaopfranca04  | convidado em | 14 | · aprovou | 0 | · pediu mudanças | 0 | · comentou | 0 |

**Resultado:** 38 PRs incorporados, 0 com aprovação registrada de um colega, 38 sem revisão

## 9. Execução e roteiro de demonstração

O ambiente sobe com um comando; com a GPU, com o arquivo dela por cima:

```
docker compose up -d # sem GPU: a CPU paralela
docker compose -f docker-compose.yml -f docker-compose.gpu.yml up -d --build # com a GPU
python scripts/resetar_banco.py --parceiros 500 # a base, segmentada e com o modelo
```

### 9.1 Roteiro para a orientação

| #  | Passo                                                                                                              |
|----|--------------------------------------------------------------------------------------------------------------------|
| 1  | Entrar como gestor: o menu por módulo, e o título da aba mudando de tela em tela                                   |
| 2  | Painel → "Ver quem está em risco" → ordenar pelo risco → abrir um parceiro: o medido, o previsto e a ação no plano |
| 3  | Campanha: os elegíveis e os excluídos; calcular com orçamento de R\$ 5.000,00, 30 ações e 30% na cauda longa       |
| 4  | O mesmo pedido no modo serial: o mesmo plano, em segundos em vez de décimos                                        |
| 5  | Orçamento de R\$ 100,00: a campanha inviável, com o que falta                                                      |
| 6  | Execuções: marcar dois planos e comparar; abrir um parceiro do plano e voltar pela trilha                          |
| 7  | Benchmark no cenário padrão: as quatro colunas, o mesmo plano e a explicação da GPU                                |
| 8  | Entrar como analista: a campanha para consulta, com o porquê no lugar do botão                                     |
| 9  | Um endereço que não existe: a página não encontrada, com a volta                                                   |
| 10 | Fechar com "docker compose down", subir de novo e mostrar o mesmo plano                                            |

O roteiro das Sprints 04 e 05 continua valendo. O Docker Desktop precisa estar aberto antes, e a verificação de ponta a ponta é a forma mais rápida de conferir que tudo subiu.

## 10. Dificuldades encontradas

### 10.1 A GPU tem um custo fixo que o tamanho do problema não paga

Iniciar o driver e criar o contexto da placa custa perto de dois décimos de segundo a cada cálculo, com 500 ou com 10.000 parceiros. Medindo só o laço, a GPU parecia a mais rápida em qualquer tamanho; medindo o processo inteiro, ela perde do OpenMP até perto de 10.000.

**O que foi feito:** o executável informa o contexto à parte, o benchmark mostra as duas coisas, e a tela explica quando a GPU não ganha. A população fica na GPU entre as gerações, que é o que faz o laço compensar.

### 10.2 Medir o Python sem medir a API junto

O benchmark roda em segundo plano, no processo da API. Ali, o Python dividiria o interpretador com as requisições da própria tela, que consulta o andamento a cada dois segundos: sairia mais lento, e o ganho das outras versões sobre ele, maior do que é.

**O que foi feito:** o Python do benchmark roda num processo à parte desde a primeira versão, com teste que confere isso; a medição não depende de a tela estar aberta.

### 10.3 No contêiner, mais threads não é mais previsível

Com as 16 threads lógicas, o OpenMP oscilava de uma medição para a outra — houve execução sem ganho nenhum: o contêiner disputa a CPU com o próprio sistema.

**O que foi feito:** o OpenMP usa uma thread por núcleo físico, e as medições reportam mediana e faixa, e não um número só.

### 10.4 Evidência que depende do "período mais recente"

A transcrição de validações importa um período longe no futuro, para não tocar na base de demonstração. A campanha usa sempre o período mais recente — e, com o futuro gravado, ninguém tinha previsão nele: a seção da campanha deu errado pela ordem, e não pelo sistema.

**O que foi feito:** a seção da campanha roda antes da importação. É a mesma lição da Sprint 05, com o modelo: quando o resultado depende do período mais recente, a ordem das seções é parte do teste.

### 10.5 Testes passando, defeito na tela

Dois defeitos desta entrega passavam em todos os testes. A tela da campanha lia um campo que a API nunca mandou, e o teste dela simulava uma API que o mandava (#190). E a página rolava a 768 px por causa de texto escondido, que o ambiente de teste não desenha (#192).

**O que foi feito:** os dois foram achados pelas capturas e pela medição no navegador. O contrato agora tem teste do lado da API, e a armadilha ficou escrita no guia do projeto, para quem vier depois.

## 11. Ajustes no planejamento, na arquitetura e na modelagem

| O que mudou                  | De / para                                                         | Por quê                                                                                                                                               |
|------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Histórias H79, H80 e H81     | Backlog de 389 para 402 pontos                                    | As melhorias de interface e de navegação e a integração pela tela, pedidas por esta entrega, viraram histórias com critério de aceite antes do código |
| Otimizador (ADR-011)         | Metaheurística (ADR-002) para genético com partidas independentes | Restrições por viabilidade, aritmética inteira e os planos gulosos na população inicial: o mesmo plano em qualquer modo, e nunca abaixo do guloso     |
| Onde o núcleo roda (ADR-012) | Compilado no Windows para executável na imagem da API             | Chamado por processo, e medido onde o sistema roda; a GPU entra por um arquivo à parte, para o sistema subir em máquina sem placa (RNF06)             |
| Modelo de dados              | 18 para 20 tabelas                                                | A execução do benchmark (H57) e o lote de mensagens (H60); a campanha ganhou os efeitos da ação e a execução em segundo plano                         |
| Quem pode o quê na campanha  | Da tela para a resposta da API                                    | O estado da campanha passou a dizer, pelo perfil de quem pergunta, se ele calcula e se edita o catálogo (#190)                                        |
| Registro de testes           | Três para quatro suítes                                           | A do otimizador, na imagem do núcleo, com a GPU                                                                                                       |
| Registro de revisões         | Nenhum para gerado                                                | A revisão cruzada, cobrada na Sprint 04, passou a ser mostrada pelo que o GitHub registra                                                             |
| Janela da Sprint 05          | Até 03/10 para até o PR do PDF                                    | O trabalho que entrou depois do PDF, no mesmo dia, é desta entrega                                                                                    |
| Arquitetura, documentada     | Versão 2.0 para 2.1                                               | A seção do dado entre os módulos, com o diagrama do fluxo                                                                                             |
| Figuras da Parte VI          | Diagramas vivos para congelados                                   | Como as da Parte II: a parte entregue não muda quando o diagrama vivo mudar                                                                           |

### 11.1 Planejamento

Esta entrega corresponde às Sprints 9 a 11 internas — o otimizador, o paralelismo em CPU e a GPU com o benchmark — mais as histórias H79 a H81. A comunicação e o assistente, das Sprints 12 e 13, já estão construídos e ficam para as próximas entregas. A tabela de correspondência do cronograma foi atualizada.

## 12. Próximos passos

| Onde                 | O que entra                                                                                                                                                   |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Próximas entregas    | A central de comunicação — mensagens por segmento, com aprovação humana obrigatória — e o assistente, que responde sobre dados já apurados ou se abstém       |
| Até a banca          | A validação do README numa máquina limpa (H72), os dois vídeos (H74, H75) e a apresentação do núcleo à equipe, com as perguntas prováveis da banca (risco R4) |
| No módulo de análise | O aval do Product Owner sobre o limiar do segmento Recém-chegado (issue #59)                                                                                  |

### 12.1 No processo

- Revisão registrada no GitHub em todo Pull Request, antes do merge — a pendência da seção 1.
- Commits de autoria de cada integrante, com a própria conta, no que cada um fizer até a entrega final.
- Medir as telas no navegador junto de cada mudança de interface, e não só na entrega: o defeito da #192 entrou com a H79 e só a medição da entrega o achou.

*Projeto acadêmico. © 2026 os autores — todos os direitos reservados. Repositório público significa visível para avaliação e portfólio; não constitui licença de uso, cópia ou redistribuição.*